Dr. Pooja's Physio — Complete Manual E2E Test Plan & Feature Guide

Feature guide and click-by-click regression suite

Application
Dr. Pooja's Physio — Next.js 16 · React 19 · Supabase · Razorpay

Document version
1.0

Audience
A tester who has never used this application before

Environment
Throwaway Supabase project · Razorpay test mode · npm run dev

Start here
Section 6 — STEP 0, Reset the test environment

Contents

Dr. Pooja's Physio — Complete Manual E2E Test Plan & Feature Guide

Document type	QA / UAT manual — feature guide plus click-by-click regression suite
Application	Dr. Pooja's Physio (Next.js 16 App Router, React 19, Supabase, Razorpay, Google Calendar/Meet)
Audience	A tester who has never used this application before
Version	1.0
Status	Pre-launch. The application has no real patients. The Debug Bar is deliberately visible in every environment.

---

1. Document purpose

This document does two jobs at once.

1. **Feature guide.** Before every group of test cases there is a plain-English explanation of what the feature is, why it exists, who uses it, which admin configuration controls it, what it interacts with, and what can go wrong. You should be able to read a section and understand the feature without opening the code.
2. **Executable test plan.** Every test case names the exact screen, the exact control, the exact value to type, and the exact result to expect. Nothing says "select a slot" or "verify the dashboard".

How to read a test case

Every case uses this fixed structure:

Field	Meaning
Test ID	Stable identifier. Quote it when reporting a defect.
Feature	The feature under test.
Role	The account you must be signed in as.
Priority	P0 critical / P1 high / P2 medium / P3 low.
Purpose	Exactly what is being verified.
Preconditions	Everything that must already exist. Usually names an earlier Test ID.
Test Data	The exact values to enter.
Steps	Numbered clicks/taps. Each step names one control and one value.
Expected Result	UI state, message text, navigation, data state, and cross-role visibility.
Cleanup	What to restore, or "leave in place for <Test ID>".

Conventions used in the steps

- **Tap** means click on desktop and tap on mobile. They are the same instruction.
- Control names are written exactly as they appear on screen, in **bold**: *Tap Sign In***.
- Values to type are written in `code`: *Enter `qa.patient.a@example.test` in the Email Address field*.
- Where a control's name is dynamic (a patient's name, a session's time), the step tells you how to identify it — for example, "the row whose **Patient** column reads `QA Patient A`".
- Money is shown to the user in rupees (₹) and stored in the database in **paise** (₹1 = 100 paise). Where a test names a stored figure, it names paise.
- All times in the application's own copy are Asia/Kolkata (IST) unless a screen says otherwise.

What "verify in the database" means

Some expected results reference database state. You do not need SQL for the ordinary path — the same fact is visible on an admin screen, and the test names that screen. Where a case genuinely needs the table (ledger append-only checks, RLS checks), it says so and gives the query, and it is marked as requiring Supabase SQL editor access.

---

2. Application overview

Dr. Pooja's Physio is a production web application for a physiotherapy practice. It has five surfaces:

1. **A public marketing site** — eight pages that explain the service and start a booking.
2. **A patient portal** — book, pay, attend, manage a health profile, answer therapist recommendations.
3. **A therapist portal** — availability, sessions, clinical records, recommendations, earnings.
4. **A hospital/partner portal** — refer patients, track referrals, see partner earnings.
5. **An admin back office** — seven sections that run the clinic: Today, Sessions, People, Money, Catalog, Logs, Settings.

The business model in one paragraph

A patient's **first purchase is always one session** — a video consultation, or (if the clinic's home-visit switch is on and their pincode is serviceable) a single home visit. Multi-session programmes cannot be bought from a price list at all: a therapist must run a session, then write a **care plan** recommending an admin-configured package, and an admin must approve it before the patient sees anything. The patient then accepts and pays from their own dashboard. That purchase creates **session credits**, which are spent one at a time as sessions are booked and delivered. Money is split between the therapist (a revenue share, earned only on delivered sessions), the referring hospital (a commission on net revenue), and the clinic.

Integrations

Integration	What it does	Failure behaviour
Supabase	Postgres database, authentication, private file storage, realtime updates	Hard dependency. Nothing works without it.
Razorpay (test mode)	Payment orders, checkout, signature verification, webhooks, refunds	A failed payment leaves an unpaid booking the patient can retry.

Integration	What it does	Failure behaviour
Google Calendar / Meet	Creates the calendar event and video link for a confirmed session	**Never blocks a booking.** Failures are recorded on the appointment and retried.

Architectural facts a tester needs

- **There is no cron job and no background worker.** Anything that must happen "when time passes" — a package expiring, a failed Meet sync being retried, risk detection — runs as a bounded sweep at the top of a relevant page render. **Consequence for testing: a time-based state change may not appear until you load the page that owns the sweep.** Each affected test names that page.
- **Email confirmation is deliberately OFF.** A sign-up returns a session immediately. If any sign-up path ever shows you a "check your email" step, that is a defect (see `SEC-AUTH-004`).
- **The public pages are ISR-cached for 300 seconds.** Against a production server (`next start`), a catalog row you just created may not appear on a public page for up to five minutes. Run the tests against `next dev`, where nothing is cached. This is not a bug in the application; it is a property of the server you point at.
- **The Debug Bar is on in every environment on purpose** and is deleted (not switched off) before real launch.

---

3. Application route map

Every route below is covered by at least one test. The rightmost column names the first test that opens it.

3.1 Public marketing pages

Route	What it is	Auth	Covered by
`/`	Home. Hero, care-area showcase, walkthrough, programmes, testimonials, mission band, connector grid	Public	`PUB-HOME-001`
`/conditions`	What we treat, plus the programme cards per condition	Public	`PUB-COND-001`
`/how-it-works`	Booking to recovery in four steps	Public	`PUB-NAV-001`
`/home-visit`	Home-visit landing page. **404s while the admin master switch is off**	Public	`PUB-HV-001`
`/team`	Therapist profiles; "Book with" carries the specialist into the wizard	Public	`PUB-TEAM-001`
`/mission`	Mission, vision, four promises, testimonials	Public	`PUB-NAV-001`
`/faq`	Admin-managed FAQ accordion	Public	`PUB-FAQ-001`
`/hospitals`	Partner pitch plus the hospital enquiry form	Public	`HOS-LEAD-001`

Route	What it is	Auth	Covered by
`/get-started`	Role hub — where a signed-in user of the wrong role is sent	Public	`SEC-ROUTE-003`

3.2 Booking

Route	What it is	Auth	Covered by
`/book`	The 3-step video-consultation booking wizard. Accepts `?category=`, `?therapist=`, and answers a stale `?package=`	Public (guest can sign up inside it)	`PAT-BOOK-001`
`/book-home-visit`	The 4-step home-visit wizard (pincode → when → who → pay)	Public	`PAT-HV-001`

3.3 Patient portal

Route	What it is	Covered by
`/patient/login`	Sign In / Register Account tabs plus Forgot password	`PAT-AUTH-001`
`/patient/register`	Standalone registration (always waits for admin approval)	`PAT-AUTH-004`
`/patient/dashboard`	Overview: four figures, activity feed, quick actions	`PAT-DASH-001`
`/patient/dashboard/book`	Booking hub inside the portal	`PAT-DASH-002`
`/patient/dashboard/suggested`	Suggested Sessions — therapist recommendations and proposed times. **Only in the sidebar when something is waiting**	`PAT-SUGG-001`
`/patient/dashboard/sessions`	All sessions, List/Calendar toggle, Upcoming/Past/Cancelled + Video/Home visit filters	`PAT-SESS-001`
`/patient/dashboard/packages`	Owned programmes and their remaining credits	`PAT-PKG-001`
`/patient/dashboard/payments`	Receipts	`PAT-PAY-010`
`/patient/dashboard/health-profile`	Health Profile: intake answers, snapshot, documents, care-plan history	`PAT-HP-001`
`/patient/dashboard/profile`	Edit Profile: photo, personal, contact, addresses, account security	`PAT-PROF-001`

3.4 Therapist portal

Route	What it is	Covered by
`/therapist/login`	Sign In / Apply to Join	`THR-AUTH-001`
`/therapist/dashboard`	Overview	`THR-DASH-001`
`/therapist/dashboard/availability`	Weekly schedule, exceptions, leave	`THR-AVAIL-001`
`/therapist/dashboard/sessions`	Sessions, join, complete, notes, recommend	`THR-SESS-001`
`/therapist/dashboard/earnings`	Earnings, payout requests, receipts	`THR-EARN-001`
`/therapist/dashboard/health-profile`	My Patients (Patients / Programmes toggle)	`THR-PAT-001`
`/therapist/dashboard/health-profile/[patientId]`	One patient's chart: intake, Pain Map, care plans, notes	`THR-HP-001`
`/therapist/dashboard/profile`	Edit Profile	`THR-PROF-001`

3.5 Hospital portal

Route	What it is	Covered by
`/hospital/login`	Sign In	`HOS-AUTH-001`
`/hospital/dashboard`	Overview	`HOS-DASH-001`
`/hospital/dashboard/refer`	Refer a Patient form	`HOS-REF-001`
`/hospital/dashboard/referrals`	Your Referrals with status	`HOS-REF-004`
`/hospital/dashboard/revenue`	Earnings (partner share)	`HOS-MONEY-001`
`/hospital/dashboard/profile`	Edit Profile	`HOS-PROF-001`

3.6 Admin back office

`/admin/login` and `/admin/dashboard`. The dashboard is one page; the screen is chosen by `?section=&tab=`. All 31 screens:

Section	Tab key	Screen	Covered by
Today	`overview`	Today	`ADM-TODAY-001`
Today	`approvals`	Approvals	`ADM-APPR-001`
Today	`risk`	Risk	`ADM-RISK-001`
Sessions	`schedule`	Schedule (calendar)	`ADM-SCHED-001`
Sessions	`all`	All Sessions	`ADM-SESS-001`

Section	Tab key	Screen	Covered by
Sessions	`new`	New Booking	`ADM-SESS-NEW-001`
Sessions	`roster`	Roster	`ADM-ROST-001`
Sessions	`delivery`	Delivery (operational rates)	`ADM-DELIV-001`
Sessions	`recommendations`	Recommendations — the clinic's review queue, plus every plan	`ADM-CARE-001`, `ADM-CARE-004`
Sessions	`new`	New Booking	`ADM-NEWB-001`
People	`patients`	Patients (+ condition requests)	`ADM-PEOP-001`, `ADM-PEOP-010`
People	`therapists`	Therapists	`ADM-PEOP-005`, `ADM-PEOP-010`
People	`partners`	Partners	`ADM-PEOP-008`, `ADM-PEOP-010`
Money	`summary`	Summary	`FIN-SUM-001`, `FIN-SUM-004`, `FIN-SUM-005`, `FIN-NAV-001`
Money	`transactions`	Transactions	`FIN-TXN-001`
Money	`payouts`	Payouts + payout requests + Cash Ledger	`FIN-PAY-001`
Money	`costs`	Costs + promo codes	`FIN-COST-001`, `ADM-PROMO-001`
Money	`breakdown`	Breakdown	`FIN-BRK-001`
Catalog	`conditions`	Conditions	`ADM-CAT-001`, `ADM-CAT-004`
Catalog	`packages`	Packages	`ADM-CAT-005`
Catalog	`areas`	Service Areas + waitlist	`ADM-CAT-010`
Catalog	`purchases`	Purchases	`ADM-CAT-014`
Logs	`all`	All Activity	`ADM-SET-033`, `ADM-SET-034`, `ADM-LOG-001`, `ADM-LOG-003`, `ADM-LOG-003a`
Logs	`retention`	Archive & Clear	`ADM-LOG-002`
Settings	`brand`	Brand & Contact	`ADM-SET-001`
Settings	`public`	Public Site	`ADM-SET-004`

Section	Tab key	Screen	Covered by
Settings	`booking`	Booking Rules	`ADM-SET-010`
Settings	`offers`	Offers & Discounts	`ADM-SET-023`, `ADM-INVITE-001`
Settings	`programmes`	Programmes & Home Visits	`ADM-SET-018`
Settings	`clinical`	Clinical Questions	`ADM-SET-020`
Settings	`access`	User Access	`ADM-SET-025`, `ADM-SET-025d`, `ADM-SET-026b`
Settings	`health`	System Health	`ADM-SET-030`
Settings	`security`	Account Security	`ADM-SET-035`

Detail routes (open as an overlay from the dashboard, and as a full page on direct navigation):

Route	Covered by
`/admin/dashboard/patients/[id]`	`ADM-PEOP-003`
`/admin/dashboard/therapists/[id]`	`ADM-PEOP-006`
`/admin/dashboard/conditions/[id]`	`ADM-PEOP-004`

3.7 System routes

Route	What it is	Covered by
`/dashboard`	Server-side role router. Sends each role to its own dashboard. The admin path never reaches a public bundle.	`SEC-ROUTE-006`
`/pending-approval`	Where an unapproved patient/therapist lands	`PAT-AUTH-003`
`/account-suspended`	Where a suspended account lands	`SEC-AUTH-006`
`/reset-password`	Password reset landing	`PAT-AUTH-006`

3.8 API routes

The application exposes 150+ POST route handlers under `/api`, grouped by audience: `admin/`, `appointments/`, `patient/`, `therapist/`, `hospital/`, `packages/`, `home-visit/`, `care-plan/`, `razorpay/`, and `medical-documents/`. Individual routes are named inside the tests that exercise them. The security section tests them directly with `curl` — `SEC-ROUTE-002` (anonymous), `SEC-ADMIN-002` (wrong role) and `SEC-TAMPER-*` (manipulated bodies).

4. Roles, gates and scopes

4.1 The four roles

`profiles.role` is a single column with one of four values. **One account carries exactly one role.** A therapist account can never also be a patient — the booking wizards refuse it, and so do the purchase routes.

Role	How an account is created	Gate before it can be used
`patient`	Self-registers (at `/patient/register`, or inside either booking wizard)	`approved` + `active`. **Exception:** a genuine payment attempt auto-approves a patient (see below).
`therapist`	Self-applies at `/therapist/login` → **Apply to Join**	Admin approval, then `active`
`hospital`	Provisioned by an admin (People → Partners → onboard)	`active`
`admin`	Promoted by hand in Supabase, or minted by a Master Admin	`active` only — **`approved` is deliberately not checked for admins**

4.2 The two flags

`profiles.approved` and `profiles.active` are enforced in **two** places, and both matter:

1. **`src/proxy.ts`** — blocks dashboard *navigation*.
2. **`requireActiveProfile`** inside self-service API routes — blocks a valid session cookie calling the API around the UI.

A test that only proves the UI hides something has not proved the rule. Every authorization test in this plan has an API-level twin.

4.3 The payment-attempt approval rule (important, and easy to mis-report)

For a **single online session**, `/api/razorpay/create-order` flips the paying patient's `approved` to `true` the moment they *genuinely attempt* checkout — on the attempt, not on a completed payment. This is deliberate: a patient whose card fails three times still lands in their dashboard with a pending appointment rather than being bounced to `/pending-approval`.

It does **not** apply to:

- home-visit purchases (`/api/home-visit/create-order` — a *completed* payment vets you), or
- standalone registration at `/patient/register` (always waits for a human admin).

4.4 Admin scopes

`profiles.admin_scope` is one of four values. It decides which **sections** an admin may open. **Every** admin route guards on scope — 99 of the 102 with `requireAdminScope(section)`, and three (`set-admin-scope`, `debug-reset`, `create-account`) with an explicit **full-only** check instead, because a section check would be too weak: a `finance` admin passing a section gate could otherwise widen its own access or mint a full admin. The sidebar hiding a section is presentation only.

Scope	Sections it can open	Cannot
`full`	Today, Sessions, People, Money, Catalog, Settings	—

Scope	Sections it can open	Cannot
`operations`	Today, Sessions, People, Catalog	Money, Settings
`finance`	Today, People, Money	Sessions, Catalog, Settings
`clinical`	Today, Sessions, People	Money, Catalog, Settings

Rules that must hold (tested in `ADM-SET-025`–`ADM-SET-029`):

- Only a `full` admin can change scopes or create another admin.
- **Nobody can change their own scope.**
- **The last `full` admin cannot be narrowed.**
- An unknown/null scope reads as `full` (so a migration can never lock everyone out).
- The **Risk** queue is `full`-only — a scoped admin's page does not even fetch it.

---

5. Test environment requirements

5.1 What you need before you start

Requirement	Value / note
A **throwaway Supabase project**	Never a project holding real data. The reset in Step 0 truncates every table.
`supabase/schema.sql` applied to it	`node scripts/run-schema.mjs`, or push to `main` and let `.github/workflows/schema-apply.yml` run it. Re-apply twice to confirm it is re-runnable.
Supabase Auth → Confirm email = OFF	The application assumes this. With it on, every sign-up path fails.
Razorpay test-mode keys	`RAZORPAY_KEY_ID` / `RAZORPAY_KEY_SECRET` from the Razorpay dashboard in **Test Mode**. Never live keys.
`RAZORPAY_WEBHOOK_SECRET` set	Without it `/api/razorpay/webhook` answers `503 {"error":"Webhook not configured"}` and a patient who pays and closes the tab leaves a paid order against an unpaid booking. Webhook tests need it.
`ALLOW_DEBUG_DATA_RESET=true`	**Server-side only.** Arms the Reset data button. Never set on a live site.
`NEXT_PUBLIC_SHOW_DEBUG_NAV`	Leave unset. The Debug Bar must be visible.
Google Calendar credentials	Optional. Without them Meet sync fails and is recorded — which is itself a test (`ADM-SET-031`).
The app running with **`npm run dev`**	Not `next start`. The public pages are ISR-cached for 300s, so a production build serves HTML that predates your fixtures.
At least one **Master Admin** (`admin_scope = 'full'`) in `profiles`	The reset keeps admin logins and refuses to run if it would leave none.
Browsers	Chrome/Edge desktop at 1440×900, plus a mobile viewport at **390 × 844**.

Requirement	Value / note
Supabase SQL editor access	Only needed for the handful of tests marked **[SQL]**.

5.2 Environments

Environment	May be reset?	Notes
Local `npm run dev` against a throwaway Supabase project	**Yes**	The default target for this plan.
Staging/preview deployment against a throwaway project	**Yes**, if `ALLOW_DEBUG_DATA_RESET=true` is set there	Public pages are cached; expect ISR delays.
Production / any project with real patients	**NEVER**	`ALLOW_DEBUG_DATA_RESET` must be unset there, which makes the reset route answer `404`.

---

6. STEP 0 — Reset the test environment

Run this first, before anything else in this plan. Every numbered section after this assumes a clean database plus the setup in Section 12 (Execution Order).

6.1 Why the database must be reset

Testing this application means filling it with throwaway patients, bookings, purchases and payouts. Several rules in the product are **one-per-thing** rules — one open care plan per patient (published or awaiting the clinic), one pending suggestion per purchase, one open risk signal per rule+subject, a unique Razorpay order id — so leftover rows from a previous run make a correct application look broken. Two known examples:

- A booking test that books a fixed slot leaves the appointment behind; the next run's identical booking is refused as a clash. That refusal is **correct behaviour**, not a defect.
- An audit-log count assertion picks up the previous run's writes.

If a test fails on a second consecutive run but passes on a fresh one, suspect leftover state before suspecting the application.

6.2 What the reset removes and keeps

The Reset data button calls `/api/admin/debug-reset`, which calls the database function `debug_reset_all_data()`. It is **one atomic `TRUNCATE`**, not a list of deletes.

Removed: every appointment, package purchase, home-visit purchase, payment, payment webhook event, payment failure log, entitlement and credit ledger row, session note and revision, pain assessment, condition profile, condition change request and access grant, patient address, medical-document metadata row, admin notes, profile change request, availability template and override, referral, B2B lead, home-visit waitlist, service area, the home-visit package catalog, testimonials, FAQs, intake question templates, payout requests and batches, business expenses, session suggestions, care plans, their versions and the clinic's reviews of them, risk signals and reviews, communication flags, contact reveal log, and the admin activity log. `site_settings` is put back to its defaults. **Every non-admin account is deleted.**

Kept:

- **Admin logins** — the function refuses to run if it would leave no admin behind.
- **Detector thresholds** (`risk_rules`) — configuration, like `site_settings`, so it is **reset to its seeded defaults** rather than emptied. Emptying it would silently disable every detector instead of restoring it.
- **The conditions catalogue** — `treatment_categories` and their `treatment_category_packages`. They are the one part of the list an admin builds by hand rather than generates by testing, so emptying them meant retyping the catalogue after every reset and left the public pages showing nothing, which reads as the clinic having shut rather than as test data being cleared. Home-visit packages, service areas, FAQs and testimonials are **not** kept.
- **Objects in the private `medical-reports` Storage bucket.** The metadata rows go; the files do not. Storage is not reachable from SQL. Clear that bucket from the Supabase dashboard if you need the space back.

> **Getting the accounts back in one command.** Every account in §8.2, §8.3, §8.8 and §8.9 is deleted by the reset — that is what "every non-admin account" means, and the full admin is the only login that survives. Rather than recreating twelve of them by hand before testing can start, run **`npm run seed:qa`** (`scripts/seed-qa-accounts.mjs`) from the repository, with `SUPABASE_SERVICE_ROLE_KEY` and `NEXT_PUBLIC_SUPABASE_URL` in `.env.local`. It creates all four admins, all three patients, all three therapists and both hospitals with the §8.1 password, prints each hospital's referral code, and is safe to re-run: an existing account keeps its id, its history and its code, and only has its password put back. **If a tester reports `Invalid login credentials` on an account this document names, that command is the first thing to try** — it repairs a missing account and a forgotten password alike. It deliberately seeds **accounts only**. Service areas, home-visit packages, therapist rosters and each patient's saved address are created by named tests (§23.2, phases 2 and 5), and seeding them would let those tests pass without running.

> **Regression worth knowing about.** An earlier version of this function predated the care-plan, evidence and risk tables and cleared none of them; three were saved by CASCADE, but `communication_flags`, `risk_signals` and `risk_reviews` survived a "delete everything". The one that actually bit a tester was `risk_signals`: it carries a partial unique index allowing at most one **open or reviewing** signal per `(rule, subject)`, so a leftover open signal held the slot and the same rule firing again wrote nothing — an empty Risk queue on a supposedly clean database, which reads exactly like a broken detector. All three are now named in the TRUNCATE list. `SETUP-RESET-001` asserts it.

6.3 The four gates (all must pass)

1. `ALLOW_DEBUG_DATA_RESET=true` in the **server** environment. Unset, the route answers **`404`, not `403`** — deliberately, so the route's existence is not confirmed.
2. A signed-in admin.
3. That admin's scope must be `full`.
4. The exact phrase `RESET ALL DATA`, typed by hand.

TEST: `SETUP-RESET-001` — Reset the test environment

Feature	Pre-launch data reset
Role	Admin (full scope)
Priority	P0

Purpose. Bring the database to a known-empty state, and prove all four gates behave.

Preconditions. `ALLOW_DEBUG_DATA_RESET=true` in the server environment. A Master Admin account exists.

Test Data. Admin: `qa.admin@example.test` / `QaTest!2024pass`. Confirmation phrase: `RESET ALL DATA`.

Steps

1. Open `http://localhost:3000/admin/login`.
2. Tap the **Email Address** field. Enter `qa.admin@example.test`.
3. Tap the **Password** field. Enter `QaTest!2024pass`.
4. Tap **Sign In**.
5. Confirm the black **Debug** bar is pinned across the top of the page.
6. In the Debug bar, tap **Reset data**.
7. Read the red warning that appears: *"Deletes people, sessions, purchases, money and settings. Admin logins and your conditions (with their programmes) survive — the rest of the catalog does not. No undo."*
8. Tap the confirmation text field (its placeholder reads `RESET ALL DATA`). Enter `reset all data` (lower case, deliberately wrong).
9. Observe the **Reset** button.
10. Clear the field. Enter `RESET ALL DATA` exactly.
11. Tap **Reset**.
12. Wait for the button to stop reading **Resetting…**.

Expected Result

- Step 7: the warning text is present and names both survivals explicitly — admin logins and the conditions catalogue.
- Step 9: the **Reset** button is **disabled** (visibly faded) while the typed phrase does not match exactly. A wrong-case phrase never arms the button.
- Step 12: the button returns to normal and a teal confirmation message appears in the bar. No error message. **It states real figures** — *"N accounts deleted, M admins kept"* with N and M non-zero where accounts existed. Two zeroes on a wipe that emptied the database is the bug where the route read `accounts_deleted` for a function returning `deleted_accounts`; it reads as a reset that did nothing.
- Step 12, the other way it failed: **`UPDATE requires a WHERE clause`** in the bar. That is pg-safeupdate, which Supabase preloads for the `authenticator` role PostgREST connects as, refusing a bare `UPDATE` inside `debug_reset_all_data()`. It cannot be reproduced by applying `schema.sql` or by running the statement in the SQL editor — both connect as `postgres`, which has no such preload — so **this gate is only ever exercised by pressing the button**, which is why it reached the deployed site unnoticed. If it reappears, a new `UPDATE` or `DELETE` in that function is missing a `WHERE`; the `TRUNCATE` is never the cause, since the guard does not cover it.
- **Open `/conditions` afterwards.** Every condition you created is still there, at its price, in its order. The public site must not come back empty.
- The page refreshes. The admin remains signed in — the session is not destroyed.
- Navigating to **People → Patients** shows an empty-state message, not a table of rows.
- Navigating to **Catalog → Conditions** shows no treatment categories.
- Navigating to **Sessions → All Sessions** shows no sessions.
- Navigating to **Logs → All Activity** shows an empty log apart from the reset's own row (the reset truncates the table, then records itself).

- Navigating to **Settings → User Access** still lists at least one admin, and your own row is there. **If this list is empty, stop immediately and restore from backup — the reset must never leave the clinic without an admin.**
- Navigating to **Today → Risk** shows an **empty** queue. **[SQL]** confirm with `select count(*) from communication_flags;` and `select count(*) from risk_signals;` — both must return `0`. A non-zero count here is the regression described above, and it will silently suppress the detector tests later in this plan.
- **[SQL]** `select rule_key, enabled from risk_rules;` still returns the eight rules, with `plan_conversion_low` and `post_consultation_dropout` back to **disabled** — thresholds are restored to their seeded defaults, not wiped.

Cleanup. None. This is the starting state for the whole plan.

TEST: `SETUP-RESET-002` — The reset is refused for a non-full admin

Feature	Pre-launch data reset — gate 3
Role	Admin (operations scope)
Priority	P1

Purpose. Prove that scope, not merely being an admin, gates the wipe.

Preconditions. `ADM-SET-026` has created `qa.admin.ops@example.test` with scope **Operations** — **run it before this test even though it belongs to a later phase.** The account cannot exist before somebody creates it, and the reset does not create it: a fresh database has exactly one admin, the one made by hand in Supabase before Step 0. Attempting this test first is answered `Invalid login credentials`, which is the account being absent rather than anything about the reset.

> **The password is not the standard one.** `create-account` **generates** it — nine random bytes, base64url — and shows it **once**, on the User Access screen, as *"temporary password `<value>`"*. It is deliberately never emailed, never written to the activity log, and never stored anywhere for an admin (the `temp_password` column exists for patients, therapists and hospitals only). **Copy it when it appears.** Lost, it cannot be recovered: set a new one in the Supabase dashboard under **Authentication → Users**, or delete the account there and create it again from User Access.

Steps

1. Sign out of the full admin account.
2. Sign in at `/admin/login` as `qa.admin.ops@example.test` with the one-time password `ADM-SET-026` showed you.
3. In the Debug bar, tap **Reset data**.
4. Enter `RESET ALL DATA` in the confirmation field.
5. Tap **Reset**.

Expected Result

- A red error appears in the bar reading exactly: `Only a Master Admin can reset data.`
- No data is deleted. **People → Patients** still lists the same rows as before step 5.
- No `admin_activity_log` row is written for a reset.

Cleanup. Sign back in as the full admin.

TEST: `SETUP-RESET-003` — The reset route is invisible when disarmed [server config]

Feature	Pre-launch data reset — gate 1
Role	Admin (full scope)
Priority	P1

Purpose. Prove the route answers `404` (not `403`) when `ALLOW_DEBUG_DATA_RESET` is unset, so its existence is not confirmed to a caller.

Steps

1. Stop the dev server. Remove `ALLOW_DEBUG_DATA_RESET` from the environment. Start the dev server again.
2. Sign in at `/admin/login` as the full admin.
3. In the Debug bar, tap **Reset data**, enter `RESET ALL DATA`, tap **Reset**.

Expected Result

- The bar shows an error. The browser **Network** tab shows `POST /api/admin/debug-reset` returning **HTTP 404** with body `{"error":"Not found"}`.
- **It must not return 403.** A 403 would confirm the route exists.
- No data is deleted.

Cleanup. Restore `ALLOW_DEBUG_DATA_RESET=true` and restart the dev server before continuing.

---

7. The Debug Bar and time simulation

7.1 What the Debug Bar is

A black bar pinned to the top of **every** page, in **every** environment. It exists because this application is pre-launch. It carries three tools:

Control	What it does
Jump to page dropdown	Navigates to any of the app's main routes, including the four protected dashboards. Entries 1–8 are the public pages; 9–15b are Get Started, both booking wizards, and each role's login and dashboard.
Simulate now (`datetime-local` input) + **Set**	Sets a simulated clock.
Reset to Real Time	Appears only while a simulation is active. Clears it.
Reset data	The wipe covered in Step 0.

The right-hand label reads *"Remove this bar before real launch"*. That is the intended end state: the bar is **deleted**, not switched off, because `NEXT_PUBLIC_SHOW_DEBUG_NAV` is a public flag and the bar names `/admin/login` and `/admin/dashboard`.

7.2 How the simulated clock actually works — read this before writing a defect

This is the single most misunderstood part of the application, and mis-reading it produces false defects.

The simulated clock is stored as an OFFSET, not as a fixed target. When you set "12 September 2026, 18:00", the app stores `target − Date.now()` in `localStorage` under `debugNowOffsetMs`. The simulated clock then keeps **ticking forward at normal speed**. It never freezes.

It is client-side only. It affects only client-rendered advisory gates that call `debugNow()` instead of `Date.now()`. It is **deliberately never wired into any server-side or API-route time check.**

Affected by the simulated clock	NOT affected (uses the server's real clock)
The `/book` wizard's date calendar and hour list (which dates/times are offered)	`/api/appointments/create`'s lead-time validation
The `/book-home-visit` wizard's date/time picker	`/api/home-visit/*` lead-time validation
Tap to Join / **Session Completed** button states on every dashboard	`/api/appointments/complete-session`'s join-window gate
Session card greying and Upcoming/Past bucketing in the browser	Refund-eligibility maths in `/api/appointments/cancel`
The therapist's own suggestion picker	`/api/therapist/suggest-session`'s lead-time check
	Payout maths, `completed_at` stamping, audit timestamps, `paid_at`

The practical consequence, stated plainly: you can use the simulated clock to make the *UI offer* a slot or a button. You cannot use it to make the *server accept* a time-gated write. If you simulate a date far in the future and then try to complete a session, the client will show you the **Tap to Join** control and the server will still answer `409` with *"This session hasn't started yet. You can mark it done once it's under way."* **That is correct behaviour, not a defect.** Tests that need a server-side time gate to pass say so explicitly and tell you to use a real near-future slot instead.

Because the storage key is `localStorage`, the simulation is **per browser profile**, and it survives navigation and reload until you reset it. Applying it triggers a **full page reload** — soft re-renders would not pick it up, because every consumer reads the clock once in a lazy initializer.

7.3 How to use it

To set a simulated time

1. Tap the **Simulate now** field in the Debug bar.
2. Type or pick the date and time **in your browser's local timezone** (the field is `datetime-local`, so it is not UTC).
3. Tap **Set**.
4. The page reloads. A **Reset to Real Time** button appears beside the field.

To verify the app is using simulated time (do this before every time-dependent test)

1. After Step 4 above, navigate to `/book`.
2. Look at the Step 1 calendar. The month shown and the highlighted "today" cell must match your simulated date, not the real date.

3. Alternatively, open the browser console and run `localStorage.getItem('debugNowOffsetMs')`. A non-zero number means a simulation is active. `null` means real time.

To reset

1. Tap **Reset to Real Time**. The page reloads and the button disappears.
2. Confirm `localStorage.getItem('debugNowOffsetMs')` returns `null`.

> **Always reset to real time at the end of a time-simulation test.** A left-over offset silently changes every later test's calendar and every join-button state, and the failures it produces look like product bugs.

7.4 Standard simulated-time scenarios

These are referenced by ID throughout the plan.

Scenario	Simulate	What it is for	Related tests
TIME-A	`2026-09-10 10:00`	Baseline booking. Online lead time is 12h, so the earliest offered slot is 10 September 22:00.	`PAT-BOOK-002`, `PAT-BOOK-003`
TIME-B	`2026-09-12 18:00`	Same rules, different day-of-week and a later hour, so the boundary lands on the *next* day. Proves the boundary is computed, not hardcoded.	`PAT-BOOK-004`
TIME-C	`2026-09-10 23:30`	Late-night boundary: no slot remains today, so the calendar's earliest bookable date must roll to 11 September.	`PAT-BOOK-005`
TIME-D	Real clock + 24h ahead of a home-visit slot	Home-visit lead time defaults to 24h — longer than online. Proves the two lead times are separate settings.	`PAT-HV-003`
TIME-E	10 minutes **before** a confirmed session's slot	The join window opens `join_window_minutes` (default 15) before the slot. **Tap to Join** must be live.	`THR-SESS-004`, `PAT-SESS-003`
TIME-F	90 minutes **after** a confirmed session's slot	Past `session_completed_after_minutes` (default 60). Every join control on every surface must read **Session Completed**.	`XR-CUTOFF-001`
TIME-G	30 hours before a paid session's slot	Outside the 24h cancellation window → full refund path.	`PAT-CANCEL-001`
TIME-H	2 hours before a paid session's slot	Inside the 24h window → no refund, and the confirm dialog must say so.	`PAT-CANCEL-002`

TEST: `DBG-TIME-001` — The simulated clock changes what the booking picker offers

Feature	Debug Bar — time simulation
Role	Public (no sign-in needed)
Priority	P1

Purpose. Prove the simulation is applied, is an offset (not a freeze), and is reversible.

Steps

1. Open `/book` with no simulation active. Note the earliest date highlighted in the Step 1 calendar and the first hour offered. Write both down.
2. In the Debug bar, set **Simulate now** to `2026-09-10T10:00` (scenario TIME-A). Tap **Set**.
3. After the reload, open `/book`.
4. Note the calendar month, the "today" cell, and the first hour offered.
5. Open the browser console and run `localStorage.getItem('debugNowOffsetMs')`.
6. Wait 60 seconds. Reload `/book`. Note the first offered hour again.
7. Tap **Reset to Real Time**.
8. Open `/book` again.

Expected Result

- Step 4: the calendar shows **September 2026**, the "today" cell is **10**, dates before 10 September are not selectable, and the earliest selectable slot is **10 September 2026 at 22:00** — twelve hours after the simulated now, matching the 12-hour online booking lead time.
- Step 5: returns a non-zero numeric string.
- Step 6: the offering is **not frozen**. The clock advanced by roughly a minute, so the picker's boundary has moved forward too (at a minute's granularity you will usually see the same 22:00 hour offered, but the *underlying* now has advanced — confirm by re-reading the "today" cell and by the fact that at a 23:00+ simulated time the day rolls over, per TIME-C).
- Step 8: the calendar returns to the real current month and the values you wrote down in step 1.
- At no point does any figure change on an **admin** money screen or an appointment's stored `slot_time` — the simulation is display/advisory only.

Cleanup. Confirm `localStorage.getItem('debugNowOffsetMs')` is `null`.

TEST: `DBG-NAV-001` — Jump to page reaches every listed route

Feature	Debug Bar — route jump
Role	Any
Priority	P2

Purpose. Smoke-test that every route in the dropdown renders without a crash, and that the protected ones enforce their gate.

Steps

1. Sign out completely (clear cookies or use a private window).
2. Open `/`.
3. Tap the **Jump to page** dropdown. Select each entry in turn, from `1. Home` to `15b. Partner Dashboard (protected)`.
4. After each, note the resulting URL and whether the page renders content or an error screen.

Expected Result

- Entries 1–8 (the eight public marketing pages) render normally — **except** `4. Home visit`, which shows a **404** page while the admin master switch `home_visit_enabled` is off. That 404 is correct and is the point of including the entry.

- `9. Get Started Hub`, `10. Booking Enquiry`, `10b. Home Visit Booking`, `11/12/14/15` login pages all render.
- Every entry ending in `b` (the four protected dashboards) redirects a signed-out visitor to that role's own login page: `/patient/login`, `/therapist/login`, `/admin/login`, `/hospital/login`.
- No entry produces a blank page, an unhandled error screen, or a stack trace.

Cleanup. None.

---

8. Test Data Library

Use these exact values everywhere. Every test in this plan refers to them by label (for example "Patient A"). All emails use the reserved `.test` TLD so nothing can be delivered to a real inbox.

8.1 Standard password

All test accounts use the same password: `QaTest!2024pass`

> **Before typing any of §8.2–§8.9 in by hand, read this.** A data reset deletes every non-admin account, so the twelve logins below stop existing the moment `SETUP-RESET-001` runs. `npm run seed:qa` recreates all of them with this password, prints both hospitals' referral codes, and can be re-run safely — see §6.2. The two accounts an admin would otherwise mint from the back office (a scoped admin in `ADM-SET-026`, a hospital in `HOS-AUTH-002`) hand out a **generated** password shown once and stored nowhere; the seeder gives them this one instead, which is why `Invalid login credentials` on any account named here is a seeding question before it is a bug.

Where a test needs a *second, different* password (a change-password test), use `QaTest!2024new`.

8.2 Admin accounts

Label	Email	Scope	Purpose
Admin Full	`qa.admin@example.test`	`full`	The main admin. Survives the reset.
Admin Ops	`qa.admin.ops@example.test`	`operations`	Proves Money and Settings are blocked.
Admin Finance	`qa.admin.finance@example.test`	`finance`	Proves Sessions and Catalog are blocked.
Admin Clinical	`qa.admin.clinical@example.test`	`clinical`	Proves Money, Catalog and Settings are blocked.

Admin Full is created by hand in Supabase before Step 0 (set `role='admin'`, `active=true`, `admin_scope='full'`), and is the only account that has the §8.1 standard password. The other three are created from **Settings → User Access** in `ADM-SET-026`, and each gets a **generated one-time password shown once on that screen** — not `QaTest!2024pass`. Write all three down as you create them: nothing stores an admin's temporary password, so a lost one is reset from the Supabase dashboard under **Authentication → Users**.

8.3 Patients

Field	Patient A (main journey)	Patient B (isolation/negative)	Patient C (hospital-referred)
Full name	`QA Patient A`	`QA Patient B`	`QA Referred Patient C`
Email	`qa.patient.a@example.test`	`qa.patient.b@example.test`	`qa.patient.c@example.test`
Password	`QaTest!2024pass`	`QaTest!2024pass`	`QaTest!2024pass`
Phone	`+91 98765 43210`	`+91 98765 43211`	`+91 98765 43212`
Date of birth	`1990-04-12`	`1985-11-30`	`1978-02-05`
Gender	`Female`	`Male`	`Female`
Address line 1	`12, 3rd Cross, Indiranagar`	`44 Residency Road`	`8, 100 Feet Road, Indiranagar`
Address line 2	`Near Metro Station`	*(leave blank)*	`Above the pharmacy`
City	`Bengaluru`	`Bengaluru`	`Bengaluru`
State	`Karnataka`	`Karnataka`	`Karnataka`
PIN code	`560038` (serviceable)	`560025` (**not** a service area)	`560038`
Emergency contact	`QA Contact A`, `+91 98765 43299`	*(leave blank)*	`QA Contact C`, `+91 98765 43298`
Referral code	*(blank)*	*(blank)*	The code from Hospital A (`HOS-AUTH-002`)
Concern	`Lower back pain`	`Knee pain after running`	`Post-stroke weakness, right side`
Booking notes	`Desk job, pain worse after sitting all day. Goal: sit through a full workday.`	`Pain on stairs for six weeks.`	`Discharged last week, needs home programme.`

Negative-test values (used to prove validation, never to create an account):

Purpose	Value
Invalid email	`not-an-email`
Too-short password	`abc12` (5 characters; minimum is 6)
Mismatched confirm password	Password `QaTest!2024pass`, confirm `QaTest!2024pas`
Invalid phone	`12345`
Invalid PIN code	`0560038` (leading zero — the pattern requires `[1-9]` first) and `56003` (5 digits)
Unknown referral code	`ZZZZZZ`
Contact-leak **block** text	`Pay me on UPI 9876543210@okhdfc instead`

Purpose	Value
Contact-leak **flag** text	`Call me on 9876543210`
Clinical text that must **not** flag	`Grade III PA mobilisation ×3 sets, 30s hold. Repeat 10 reps, twice daily.`

8.4 Clinical answers — Orthopaedic (Patient A)

The orthopaedic intake is **seven questions**. Question keys are fixed and globally unique.

Question (as shown)	Key	Answer to enter
What's the main issue you'd like help with?	`chief_complaint`	`Lower back pain that spreads into my right hip`
How long has this been going on?	`since_when`	`About four months`
Overall severity right now (0–10)	`severity`	`6`
Where does it hurt? (tap each area, rate 0–10)	`area_pain`	Tap **Lower back** → rate `7`; tap **Right hip** → rate `5`
What makes it worse?	`worsens`	`Sitting for more than an hour, and bending to pick things up`
What helps or relieves it?	`helps`	`Walking, and lying flat for ten minutes`
Anything else the therapist should know?	`notes`	`I work at a desk nine hours a day. No previous surgery.`

8.5 Clinical answers — Neurological (Patient C)

All neurological keys are prefixed `neuro_`.

Question	Key	Answer
What is the neurological condition or event, and when did it start?	`neuro_diagnosis`	`Ischaemic stroke, six weeks ago`
Which part of the body is affected?	`neuro_affected_side`	`Right side`
How do you move around indoors right now?	`neuro_mobility`	`With a walking stick and someone nearby`
Day-to-day independence right now (0–10)	`neuro_independence`	`4`
Which of these are present?	`neuro_symptoms`	Tick `Weakness on one side` and `Difficulty with balance or walking`
Falls in the last three months?	`neuro_falls`	`One`
What would you most like to be able to do again?	`neuro_goal`	`Walk to the end of my street without help`

8.6 Clinical answers — Paediatric (Patient D, if used)

Paediatric keys are prefixed `peds_`. **The two caregiver fields are a pre-step, not part of the seven-question count** — the answered counter must never include them.

Question	Key	Answer
Your name	`peds_caregiver_name`	`QA Caregiver D`
How are you related to the child?	`peds_caregiver_relationship`	`Mother`
What is the main concern about your child?	`peds_concern`	`He is not walking on his own yet at 20 months`
How was your child born?	`peds_birth_history`	`Born at 34 weeks, two weeks in special care`
Which of these can your child do on their own today?	`peds_milestones`	Tick `Sits without support` and `Pulls to stand`
Has a doctor given a diagnosis, or ordered any tests?	`peds_diagnosis`	`No diagnosis yet, an MRI is booked`
Does your child use a brace, splint, walker, wheelchair or special footwear?	`peds_equipment`	`Ankle splints on both feet`
What is hardest for your child in a normal day?	`peds_daily_difficulty`	`Standing long enough to play at the table`
What would you most like your child to be able to do in the next few months?	`peds_goal`	`Take a few steps holding my hand`

8.7 Triage answers (therapist asks these at first contact)

Triage is four questions. Its answers are stored separately from the patient's own record and are never shown to the patient.

Question	For an **ortho** outcome (Patient A)	For a **neuro** outcome (Patient C)
How old is the patient?	`18 to 64`	`65 or older`
What brought them in?	`Injury, strain or overuse`	`After a stroke, brain or spinal injury`
Any of these present?	`None of these`	`Weakness on one side` + `Difficulty with balance or walking`
Any concern about milestones…?	*(not shown — only appears when age is `Under 18`)*	*(not shown)*

Expected suggestion: **Orthopaedic** for Patient A, **Neurological** for Patient C. The suggestion is shown with its reason and is **never auto-accepted** — the therapist confirms.

8.8 Therapists

Field	**Therapist A** (main)	**Therapist B** (isolation tests)	**Therapist C** (leave / spare)
Full name	`QA Therapist A`	`QA Therapist B`	`QA Therapist C`
Email	`qa.therapist.a@example.test`	`qa.therapist.b@example.test`	`qa.therapist.c@example.test`
Password	`QaTest!2024pass`	`QaTest!2024pass`	`QaTest!2024pass`

Field	Therapist A (main)	Therapist B (isolation tests)	Therapist C (leave / spare)
Phone	`+91 90000 10001`	`+91 90000 10002`	`+91 90000 10003`
Qualifications & License / Council Reg No.	`MPT (Ortho), KSCP Reg 44821`	`MPT (Neuro), KSCP Reg 44822`	`BPT, KSCP Reg 44823`
Specialty / display note	`Spine and lower-limb rehabilitation`	`Stroke and neurological rehabilitation`	`Paediatric physiotherapy`
Experience	`9 years`	`12 years`	`5 years`
Bio	`Works with desk-based patients on posture-driven back pain.`	`Post-stroke gait and balance retraining.`	`Early-intervention paediatric care.`
Revenue share % (set by admin)	`60`	`55`	`50`
Home-visit revenue share %	`65`	*(leave unset — must fall back to 60/55)*	*(unset)*
Weekly schedule	Mon–Fri `09:00–13:00` and `14:00–18:00`	Mon–Fri `10:00–16:00`	Tue/Thu `09:00–12:00`
Date exception	`2026-09-15` : `14:00–18:00` only, reason `Clinic audit in the morning`	—	—
Leave dates	—	—	`2026-09-14` to `2026-09-18` , reason `Annual leave`
Timezone	Whatever the browser reports; the roster header states it	same	same

8.9 Hospitals / partners

Field	Hospital A (main)	Hospital B (isolation)
Organisation name	`QA Sunrise Hospital`	`QA Lakeside Clinic`
Contact person	`QA Hospital Admin A`	`QA Hospital Admin B`
Email	`qa.hospital@example.test`	`qa.hospital.b@example.test`
Phone	`+91 80400 10001`	`+91 80400 10002`
Address	`18 Airport Road`	`5 Lake View Street`
City / State / PIN	`Bengaluru` / `Karnataka` / `560017`	`Bengaluru` / `Karnataka` / `560034`
Revenue share %	`10`	`12`
Referral code	Generated at onboarding — **write it down**, Patient C needs it	Generated at onboarding

Referral payload (Hospital A → Patient C)

Field	Value
Patient Full Name	`QA Referred Patient C`
Session Type	`Online` (and `Home visit` for the second referral)
Address	`8, 100 Feet Road, Indiranagar, Bengaluru`
Preferred Language	`English`
Pincode	`560038` (required only for a home-visit referral)
Medical Issue	`Right-sided weakness following a stroke six weeks ago`
Treatment Needed	`Gait and balance retraining, twice weekly`

8.10 Treatment categories (conditions)

Create these in **Catalog → Conditions**.

Category Name	Price (₹)	Session Length (min)	Order	Button Text
`QA Back & Spine Care`	`1999`	`60`	`1`	`Book Assessment`
`QA Knee & Joint Care`	`1799`	`45`	`2`	`Book Assessment`
`QA Neuro Rehabilitation`	`2499`	`60`	`3`	`Book Assessment`

Negative values for validation tests: Price `0`, Price `-100`, Price `abc`, Session Length `0`, Order `xyz`.

8.11 Session packages (programmes)

Create in **Catalog → Packages**. Every one of these has `session_count ≥ 2`, so **none of them is directly purchasable** — they can only reach a patient through a care plan. That is the rule under test, not a limitation of the fixtures.

Field	Package P1	Package P2	Package P3 (consultation)
Category	`QA Back & Spine Care`	`QA Neuro Rehabilitation`	`QA Back & Spine Care`
Package Name	`QA Spine Recovery 6 Sessions`	`QA Neuro Rehab 8 Sessions`	`QA Single Session`
Subtitle	`Six weeks, one therapist, measured progress`	`Eight sessions of gait and balance work`	`One assessment`
Description	`A structured six-session block for persistent lower-back pain.`	`An eight-session neurological rehabilitation block.`	`A single 60-minute session.`
What We Promise (one per line)	`The same therapist every session` / `A written home programme` / `Progress measured, not guessed`	`The same therapist every session` / `Gait retraining` / `Family guidance`	`A full assessment`
Sessions Included	`6`	`8`	`1`

Field	Package P1	Package P2	Package P3 (consultation)
Bundle Price (₹)	`9999`	`17999`	`1999`
Compare-at Price (₹)	*(blank — auto-computes from the category price)*	*(blank)*	*(blank)*
Therapist Pay Basis	`Discounted package price`	`Category list price`	`Discounted package price`
Validity (days)	`90`	`120`	`30`
Session Duration (min)	*(blank — inherits 60)*	*(blank)*	*(blank)*
Minimum gap between sessions (hours)	`24`	`48`	*(blank)*
Maximum sessions per week	`3`	`2`	*(blank)*
Maximum purchases per patient	`2`	`1`	*(blank)*
Display Order	`1`	`2`	`3`
Active	ticked	ticked	ticked

Package validation negatives (expected error text in brackets):

Input	Expected message
Package Name blank	`Package Name is required.`
Sessions Included `1`	`Sessions Included must be a whole number of 2 or more.`
Bundle Price `0`	`Bundle Price must be a positive number.`
Compare-at `5000` with Bundle `9999`	`Compare-at Price can't be lower than the Bundle Price.`
Order `abc`	`Order must be a number.`

8.12 Home-visit packages

Create in **Catalog → Packages** (home-visit section).

Field	HV1 (consultation)	HV2 (programme)
Package Name	`QA Home Visit – Single`	`QA Home Visit Recovery – 4 Visits`
Subtitle	`One visit at your door`	`Four visits over a month`
Description	`A single home assessment.`	`A four-visit home rehabilitation block.`

Field	HV1 (consultation)	HV2 (programme)
Benefits (one per line)	`A physiotherapist at your door` / `Full assessment`	`The same therapist each visit` / `Family training`
Visits Included	`1`	`4`
Package Price (₹)	`2499`	`8999`
Visit Duration (minutes)	`60`	`60`
Validity (days)	`30`	`90`
Minimum gap between visits (hours)	*(blank)*	`48`
Maximum visits per week	*(blank)*	`2`
Travel fee included in price	**unticked**	**unticked**
Lock to one therapist	ticked	ticked
Active	ticked	ticked

HV1 is the only home-visit row that may be bought directly (one visit = a consultation). HV2 (`visit_count > 1`) must be refused by both `/api/home-visit/create-order` and `/api/home-visit/book-cash`.

8.13 Service areas (home visit)

Create in **Catalog → Service Areas**.

Field	Area 1	Area 2
City	`Bengaluru`	`Bengaluru`
Area name	`Indiranagar`	`Koramangala`
Travel fee (₹ per visit)	`150`	`200`
Pincodes	`560038`	`560095`
Notes	`Core service area`	`Second phase`

Not serviceable (use for the negative path): `560025`. **Invalid formats:** `0560038`, `56003`, `abcdef`.

8.14 Documents (patient uploads)

Create small dummy files locally. Content does not matter; the filename and type do.

Filename	Type to pick in the uploader	Purpose
`Spine_Report_E2E.pdf`	`Scan or X-ray`	Happy path
`Posture_Assessment_E2E.pdf`	`Lab report`	Second upload
`Discharge_Note_E2E.pdf`	`Hospital summary`	Third upload

Filename	Type to pick in the uploader	Purpose
`Referral_Letter_E2E.pdf`	`Referral letter`	Fourth
`Knee_Xray_E2E.jpg`	`Scan or X-ray`	Image path (browser re-compresses images before upload)
`Oversize_Report_E2E.pdf`	`Scan or X-ray`	**Must be larger than 10 MB.** Create with: `head -c 11000000 /dev/urandom > Oversize_Report_E2E.pdf`
`Not_Allowed_E2E.txt`	*(the picker will not accept it)*	Wrong MIME type

Caps: 10 MB per file, **20 files per patient**. Both are enforced in the upload route, which is the only writer.

8.15 Business expenses (Money → Costs)

Description	Category	Amount (₹)	Incurred on
`QA Clinic rent September`	(pick the rent/premises category)	`25000`	`2026-09-01`
`QA Software subscriptions`	(pick the software/tools category)	`4000`	`2026-09-03`
`QA Marketing test spend`	(pick the marketing category)	`6000`	`2026-08-28` (deliberately outside a September range, to test date filtering)

8.16 Free-text fixtures for the contact-leak scanner

Text	Expected tier	Where to enter it
`Pay me directly on 9876543210@okhdfc, it's cheaper`	**block** — the write is refused	Therapist's care-plan *Why this, for this patient*
`https://rzp.io/l/abcd1234 pay here`	**block**	Therapist's suggestion note
`Call me on 9876543210 before the session`	**flag** — delivered, and recorded	Therapist's suggestion note
`Email me at therapist@example.test`	**flag**	Care-plan *Anything they should do or know*
`Grade III PA mobilisation ×3 sets, 30s hold. 10 reps, 2× daily. Order ref 90210.`	**no hit** — clinical text with digits must not fire	Session note
`Call me on 9876543210` written by the **patient**	**record only** — never blocked	Patient's booking notes on `/book`

8.17 Admin settings this plan assumes

Unless a test says otherwise, leave every setting at its default. The four that matter most for setup:

Setting	Default	Note
Therapist-Suggested Sessions	**on** on a fresh database	Needed by `THR-SUGG-*` and `PAT-SUGG-*`. On a database that predates the change it stays at its old value — **confirm the toggle before running those tests**
Assign a Therapist Automatically	**off**	`ADM-SET-021` switches it on; several booking tests assume the queue behaviour while it is off
Session Balances From The Ledger	**off**	`ADM-SET-019`
Home Visit	**off**	`ADM-SET-013` switches it on for the home-visit journey

8.18 Setup aliases used in preconditions

Several tests name a setup step by a `SETUP-*` alias. Each is simply the catalog test that creates that fixture:

Alias	Is	Creates
`SETUP-CAT-001`	`ADM-CAT-001`	The three treatment categories (§8.10)
`SETUP-PKG-001`	`ADM-CAT-005`	Packages P1, P2, P3 (§8.11)
`SETUP-HVPKG-001`	`ADM-CAT-005` (home-visit section)	HV1 and HV2 (§8.12)
`SETUP-AREA-001`	`ADM-CAT-010`	Service Areas 1 and 2 (§8.13)

---

9. Payment test data and the payment model

9.1 Razorpay test mode — credentials

> **Use Razorpay Test Mode only.** Test cards fail in live mode with an "invalid card issuer" error, which is the intended safety net. Before executing this section, re-confirm the values against Razorpay's own pages, because Razorpay changes them from time to time: *Test cards: `https://razorpay.com/docs/payments/payments/test-card-details/`* Test UPI: `https://razorpay.com/docs/payments/payments/test-upi-details/` The values below were confirmed against those pages at the time of writing. **Do not substitute invented numbers.** If a value no longer works, take the current one from the pages above and note the change in your defect report.

Test UPI IDs (India / domestic)

Purpose	UPI ID
Payment **succeeds**	`success@razorpay`
Payment **fails**	`failure@razorpay`

Domestic test card

Field	Value
Card number	`4111 1111 1111 1111`
Expiry	Any **future** date, e.g. `12 / 30`
CVV	Any 3 digits, e.g. `123`
Name on card	`QA Patient A`
OTP on the simulated 3-D Secure page	Any random 4–10 digit number, e.g. `1111`

International / US test cards (only if the account is configured for them)

Network	Number
Visa (International)	`4239 5360 0631 5640`
Mastercard (International)	`5421 1393 0609 0628`
Amex (International)	`3779 849088 69514`
Discover	`6011 0833 2733 7267`
Visa (US)	`4384 7968 2770 3274`
Mastercard (US)	`5312 6865 5677 9641`
Amex (US)	`3782 8224 6310 005`
Diners (US)	`6594 7300 0000 0001`

The Razorpay checkout in test mode also shows explicit Success / Failure buttons on the card and netbanking flows. Where a test says "force a failure", using the checkout's own **Failure** button is equivalent to using `failure@razorpay` and is the quickest route.

9.2 How money actually moves in this application

Understanding this model is what makes a payment defect report useful.

One row per Razorpay order. The `payments` table has **unique indexes on `razorpay_order_id` and on `razorpay_payment_id`**. Nothing else in the database previously stopped one payment id being recorded against two rows. A unique-violation on either is not a bug to work around — it means a duplicate already exists and wants investigating.

A capture is applied in exactly one place. The database function `record_payment_capture` moves the `payments` row and the row it paid for **together**, under a real `select … for update`. It is called by the three verify routes and by the webhook. It is **idempotent by construction**: the second caller for an order finds it already captured and changes nothing. That single property is what makes all four of these safe without any of them knowing about the others:

1. a duplicate webhook,
2. Razorpay's at-least-once retries,
3. a webhook racing the browser callback,
4. a double-clicked **Pay** button.

Two confirmations, whichever arrives first. A payment is confirmed by *either* the browser callback (`/api/razorpay/verify`) *or* the server webhook (`/api/razorpay/webhook`). Both go through the same capture function.

The webhook checks its signature against the raw body (`await request.text()`), never a re-serialised parse. It inserts its `payment_webhook_events` row **before** doing any work — that insert colliding on `razorpay_event_id` *is* the deduplication.

Without `RAZORPAY_WEBHOOK_SECRET`, the webhook half does not exist. The route answers `503`. A patient who pays and closes the tab before the callback lands leaves a **paid Razorpay order against an unpaid booking**. Test `PAY-WH-004` covers exactly this.

Meet/Calendar creation is deliberately outside the capture function, because it needs an outbound Google call. Confirmation of the appointment and the Meet link stay in the route.

Cash-on-visit is real, and breaks the usual assumption. A cash home-visit purchase sits at `payment_status: 'unpaid'` for its whole life with real confirmed visits hanging off it. **Never read `payment_status` as "did money change hands" for a home visit — check `payment_mode` first.**

9.3 The payment surfaces

Flow	Order route	Verify route	What it buys
Single online session	`/api/razorpay/create-order`	`/api/razorpay/verify`	One appointment
Home visit (prepaid)	`/api/home-visit/create-order`	`/api/home-visit/verify`	A `home_visit_package_purchases` row
Home visit (cash at the door)	*(none)*	`/api/home-visit/book-cash`	Same, `payment_mode='cash'`, `payment_status='unpaid'`
Care-plan programme	`/api/care-plan/create-order`	`/api/care-plan/verify`	A `patient_package_purchases` or home-visit purchase, plus session credits
Webhook (all of the above)	—	`/api/razorpay/webhook`	Confirms whichever the order pointed at

Direct programme checkout no longer exists. `/api/packages/create-order` and `/api/packages/verify` are deleted. `/book?package=<id>` shows a "Programmes come from your therapist now" page instead of quietly selling one session to somebody who came to buy six.

9.4 Refunds

Path	Route	Behaviour
Patient cancels outside the refund window	`/api/appointments/cancel`	Full refund via Razorpay
Patient cancels inside the window	`/api/appointments/cancel`	**No refund.** The confirm dialog says so before you commit.
Admin refunds a package	`/api/admin/refund-package`	Voids **available** credits only; delivered sessions stay delivered
Admin partial refund on a session	`/api/admin/refund-session-partial`	Requires an amount > 0 and a mandatory reason
Cash refund with no Razorpay payment behind it	`/api/admin/refund-home-visit-package`	Becomes `refund_status: 'manual_pending'`, surfaced on the admin Cash Ledger until an admin confirms the cash was handed back

A refund never touches a therapist's cut — a refunded session was cancelled, so it never earned one. It **does** reverse the hospital's commission, because that is a commission on net revenue.

9.5 What a single payment must never produce

Every one of these is a separate test in Section 15:

Must never happen	Guarded by	Test
Two appointments from one payment	The appointment is created before the order; the order is minted against it	`PAY-DUP-001`
Two purchases from one payment	Unique index on `razorpay_order_id`	`PAY-DUP-002`
Two entitlements from one purchase	`ensure_entitlement_for_purchase` is idempotent	`PAY-DUP-003`
Double revenue on the Money screens	One `payments` row per order	`PAY-DUP-004`
Double therapist payout	`therapist_payout_paid_at` CAS on settle	`PAY-DUP-005`
Double partner commission	Commission derived from net revenue per appointment, not accumulated	`PAY-DUP-006`
A ledger credit granted twice	Idempotency key derived from the appointment/payment id, never random	`PAY-DUP-007`

---

10. Patient Booking Wizard: Complete E2E Flow

This section is the reference for `/book`. Read it before executing any `PAT-BOOK-*` case.

10.1 What the booking wizard is, and why it exists

`/book` is the single entry point through which a patient buys their **first** video consultation. It is a three-step wizard that does four jobs in one screen: it picks a time, it creates an account if the visitor does not have one, it records what they need help with, and it takes payment. It exists as one wizard rather than four screens because a visitor who has to register before they can see whether a suitable time exists usually leaves.

It sells **one session**. It cannot sell a programme. A programme comes from a care plan a therapist writes after seeing the patient.

10.2 Who uses it

Anyone. A brand-new visitor (creates an account inside Step 2), or a signed-in patient (Step 2 shows "Booking as …" instead of the sign-up fields). A signed-in **therapist, hospital or admin** is refused outright and shown a "wrong account" panel — see 10.10.

10.3 What information it collects

Step	Collected	Required?
1	Preferred **date**	Yes (auto-preselected)
1	Preferred **hour**	Yes (auto-preselected)
1	Preferred **language**	Yes (auto-preselected to the first admin-configured language)
2	Full Name, Email, Create Password, Confirm Password, Phone	Yes — **guests only**
2	Referral Code	Optional
2	"What would you like help with?" (treatment category)	**Yes**
2	"Continue with the same therapist?"	Optional; only shown to a returning patient with previous therapists
2	Requested specialist (from `?therapist=`)	Optional; carried in, removable
2	"Anything else we should know?" notes	Optional
2	Telehealth consent checkbox	**Yes**
3	Nothing — Step 3 is review and pay	—

10.4 How the wizard decides which **dates** are offered

A date is offerable exactly when **at least one of its hours clears the booking lead time**. The lead time is `site_settings.online_booking_lead_time_hours`, default **12**. The calendar is built from `AVAILABILITY_HOURS` (the clinic's standard hour rows) filtered by that rule.

Consequences you will see and must not report as bugs:

- **"Today" usually drops off entirely.** At 10:00, the first bookable slot is 22:00 the same day; at 23:30, no hour remains today at all and the earliest bookable date is tomorrow.
- The boundary day is **partially** available, not all-or-nothing — early hours are greyed, later ones are live.
- Changing the date **re-preselects that day's earliest eligible hour**. A previously-picked hour is not carried forward, because it may not clear the lead time on the new date.
- The picker's rule and the server's validator read the **same setting**, so the calendar can never offer a slot the server would reject. If it does, that is a genuine defect.
- The date/hour are interpreted in the **browser's local timezone**, which is the timezone Step 1 displays. The detected timezone is shown on Step 1 and stored on the appointment.

10.5 How therapist availability affects it — the rule that surprises everyone

It does not. The therapist roster (weekly schedule, date exceptions, leave) is the **clinic's planning record** — who can be *offered* a session. It deliberately does **not** filter the patient's `/book` picker, which applies the lead-time rule alone.

This is a deliberate product decision, not an oversight. A patient picks a time they want; an admin then assigns whichever therapist is actually free, and only the admin can see that. Connecting the two is a change with a deploy-sized blast radius.

So: changing a therapist's roster must never change what `/book` offers. That is a regression test (`XCFG-ROSTER-001`), not a bug.

10.6 How service areas affect it

They do not affect `/book` at all. `/book` books `visit_mode: 'online'` only. Service areas gate `/book-home-visit` exclusively.

10.7 How admin configuration reaches the wizard

Setting	Where an admin changes it	Effect on `/book`
`online_booking_lead_time_hours`	Settings → Booking Rules → **Online Booking Lead Time**	Which dates/hours the calendar offers, and the server's own validator
`online_cancellation_refund_hours`	Settings → Booking Rules → **Online Cancellation Refund Window**	The sentence on Step 3 ("Free cancellation up to N hours...") and the refund actually paid
Booking Languages	Settings → Booking Rules → Booking Languages	The chips on Step 1. An empty list degrades to `English` — booking must never present an empty language picker.
Treatment categories	Catalog → Conditions	The "What would you like help with?" dropdown, each entry showing `Title — ₹price / duration min`. An **inactive** category disappears, and a booking against it is refused server-side.
Category **price** and **duration**	Catalog → Conditions	The header price line, Step 3's Session Fee, the Razorpay amount, and the appointment's duration. **All re-derived server-side from the category row, never from the browser** — `/book` is ISR-cached, so the copy the patient filled in can legitimately be older than the one being charged.
Therapist `visible_on_team` / approval / active	People → Therapists	Whether a `?therapist=` link resolves at all

10.8 How payment is connected

1. Step 3's primary button reads **Request Booking** the first time.
2. Tapping it creates the account (guest only), runs a client-side self-overlap check, then calls `POST /api/appointments/create`. That route **re-derives** concern, duration, lead time, therapist preference and language server-side, scans the notes for contact leaks (record-only for a patient), and inserts the appointment as `status: 'requested'`, `payment_status: 'unpaid'`, `therapist_id: null`, `visit_mode: 'online'`.
3. The wizard then immediately opens Razorpay checkout via `POST /api/razorpay/create-order`. **That call flips the patient's `approved` flag to true** — on the attempt, not on success.
4. On checkout success, `POST /api/razorpay/verify` checks the signature server-side, marks the appointment paid, auto-confirms it **only if a therapist is already assigned**, creates the Meet event if so, and records the capture.

Appointments are never inserted by the browser. If you ever see a raw Postgres string such as `new row violates row-level security policy for table "appointments"` on screen, that is a P0 defect — the failure mode this route exists to prevent.

10.9 What happens on each payment outcome

Outcome	What the patient sees	What is created
Success	Step 3 is replaced by a **Payment Confirmed** panel with a **Go to Dashboard** link	Appointment `payment_status='paid'`, still `requested` until an admin assigns; a `payments` row
Failure (`failure@razorpay`)	An error message; the primary button now reads **Pay ₹… Now** (not "Request Booking"); attempt counter increments	The appointment already exists and stays `requested` + `unpaid`
Dismissed (patient closes the checkout modal)	*"Payment was not completed. You can try again below."*	Same as failure
Abandoned (patient closes the tab)	Nothing	Same. The booking sits in their dashboard as an unpaid session with a **Pay ₹… Now** button. If the webhook secret is set and the order was in fact captured, the webhook confirms it server-side anyway.
Retry	Tapping **Pay … Now** re-opens checkout against the **same** appointment. `create-order` re-checks the prior order: if Razorpay says it is already paid, the appointment is claimed as paid rather than a second order being minted.	No second appointment, no second order for a paid one
3+ failed attempts	An amber escape hatch appears: *"Having trouble paying? Your booking is saved as pending..."* with a **Go to Dashboard** link	Nothing new
Back from Step 3 to Step 2	The draft appointment is deliberately abandoned; the button returns to **Request Booking**	The unpaid appointment remains in the dashboard, exactly as if the tab had been closed

10.10 Refusals the wizard renders instead of the form

Condition	What renders
Signed in as therapist / hospital / admin	A **wrong account** panel routing each role to what is theirs (hospitals refer; admins use New Booking; a clinician wanting therapy signs out and uses a separate patient account). Checked **before** every other branch.
`?package=<id>` in the URL	*"Programmes come from your therapist now"* plus the explanation and a **Book a first session** link. Ordered **after** the wrong-account branch on purpose.
No treatment categories exist	*"No condition categories are available right now — please contact us directly to book."* and no dropdown

10.11 What happens on Back, Refresh and abandonment

Action	Result
Back on Step 2	Returns to Step 1. All Step 1 values are preserved.
Back on Step 3	Returns to Step 2, **clears the draft appointment id and the failed-attempt counter**, and the primary button becomes **Request Booking** again.
Refresh at any step	The wizard restarts at Step 1 with fresh auto-picks. **No wizard state is persisted.** Any appointment already created stays in the database and appears in the dashboard as unpaid.
Abandon (close tab)	Same as refresh.

10.12 What each role sees afterwards

Role	Where	What
Patient	`/patient/dashboard` and → Your Sessions	The session under **Upcoming**, `Requested` if unassigned, with **Pay ₹… Now** while unpaid
Admin	Sessions → All Sessions, and Sessions → Schedule	The session in the list and on the calendar. Unassigned sessions raise the **All Sessions** badge. Opening it gives the assign form, with a requested therapist preselected and marked **(requested)**.
Admin	Money → Transactions	The payment row, once paid
Therapist	`/therapist/dashboard/sessions`	**Nothing until an admin assigns them.** A `requested` unassigned session belongs to no therapist.
Hospital	Your Referrals / Earnings	Only if the patient was referred by them

---

10.13 Step-by-step screen reference

Step 1 — "When suits you?"

- **Means:** pick a time you would like. It is a *request*, not a locked slot.
- **Expected of the user:** confirm or change three auto-picked values.
- **Controls:** a month calendar (tappable day cells; ineligible days are greyed and not tappable), a row of hour chips, a row of language chips, and **Continue**.
- **On selection:** the day cell highlights; the hour list re-filters to that day's eligible hours; the "we picked this for you" hint disappears from whichever value you changed and never re-fires.
- **Validation on Continue:** date and hour must be set (*"Please select a preferred date and time."*); the slot must clear the lead time (*"Please choose a time at least 12 hours from now."*); a language must be set (*"Please select a preferred language."*).
- **Nothing is created.** No appointment, no account, no charge.

Step 2 — Your details and your concern

- **Means:** who you are and what you need.

- **Guest:** Full Name, Email, Create Password (min 6), Phone, Confirm Password, Referral Code (optional). A signed-in patient sees only a teal *"Booking as Name (email)"* strip.
- **Everyone:** the concern dropdown, optional therapist preference, optional notes, and a **required** telehealth consent checkbox.
- **Referral code** is validated on blur: `Checking code...` → either `Valid — referred by <Hospital>` in teal, or `Code not recognized — double-check it or leave blank` in red. An invalid code **blocks Continue**; a blank one does not.
- **Validation on Review Booking →**, in this order: name/email/password present and password ≥ 6 (*"Please fill in your name, email, and a password (min 6 characters)."*); email shape (*"Please enter a valid email address."*); phone shape (*"Please enter a valid phone number."*); passwords match (*"Passwords do not match. Please re-enter them."*); referral code not invalid; a category is chosen (*"Please select what you'd like help with."*); consent ticked (*"Please agree to the telehealth consent terms to continue."*).
- **Nothing is created.**

Step 3 — Review and pay

- **Means:** confirm the summary, then pay.
- **Shows:** Name, Email, Preferred Time, Language, Concern, and Session Fee, plus two notices — the Razorpay/secure-payment line and the cancellation-window line reading the admin's configured hours.
- **Controls: Back** (⅓ width) and the primary button (⅔ width) reading **Request Booking**, then **Pay ₹… Now** after an appointment exists.
- **Records created:** on **Request Booking** — the Supabase auth user (guest only) and the appointment row. On payment success — the `payments` row, the appointment's paid/confirmed state, and (when a therapist is already assigned) the Google Calendar/Meet event.

---

11. Patient test plan

11.0 Feature guide — the patient's world

A patient's journey is: **register → get approved (or auto-approve by paying) → book a consultation → pay → attend → receive a recommendation → accept and pay for a programme → spend its credits session by session**, with a **Health Profile** running alongside that the therapist opens first and the patient then maintains.

Two rules shape almost everything on the patient's screens:

1. **A screen that can only ever be empty is not in the sidebar.** Your Sessions, Your Packages, Payments and Suggested Sessions appear only once the patient actually has one. **Book a Session is the deliberate exception** — always shown, because that is how a patient gets their first of anything.
2. **The patient is read-only on their own Health Profile until a therapist writes the first record.** While locked, the "answer your questions" call to action and the answered counter are **absent, not disabled**; the dashboard banner is dropped rather than recoloured; the overview cell reads `—` on slate rather than `0%` on amber. The **reports uploader stays open**, because it is the one useful thing a patient can do beforehand.

11.1 Registration, login, approval

`PAT-AUTH-001` — Sign in as an existing patient · P0

Feature. Patient authentication. **Role.** Patient. **Purpose.** Confirm the login form authenticates and lands on the dashboard. **Preconditions.** Patient A exists and is approved and active (created by `PAT-BOOK-003` or `PAT-AUTH-002`). **Test Data.** `qa.patient.a@example.test` / `QaTest!2024pass`.

Steps

1. Open `/patient/login`.
2. Confirm the two tabs **Sign In** and **Register Account** are visible, with **Sign In** active.
3. Tap the **Email Address** field. Enter `qa.patient.a@example.test`.
4. Tap the **Password** field. Enter `QaTest!2024pass`.
5. Tap **Sign In**.

Expected Result. The button reads `Signing in...` while working. The browser navigates to `/patient/dashboard`. The sidebar shows **Overview**, **Book a Session**, **Health Profile** and **Edit Profile** at minimum, with **Back to Home** at the **foot of the nav, directly above Collapse** (above the profile footer in the mobile drawer, which has no Collapse). The public `Navbar` is **not** rendered on the dashboard. No error banner. **Cleanup.** Stay signed in for `PAT-DASH-001`.

`PAT-AUTH-002` — Register a patient account from `/patient/register` · P0

Purpose. Prove standalone registration always waits for a human admin — the payment-attempt auto-approval does **not** apply here. **Test Data.** Patient B from §8.3.

Steps

1. Open `/patient/login`. Tap **Register Account**.
2. Tap **Full Name**. Enter `QA Patient B`.
3. Tap **Email Address**. Enter `qa.patient.b@example.test`.
4. Tap the phone field. Enter `+91 98765 43211`.
5. Tap **Password**. Enter `QaTest!2024pass`.
6. Tap **Confirm Password**. Enter `QaTest!2024pass`.
7. Leave **Referral Code** blank.
8. Tap **Create Account**.

Expected Result. The button reads `Creating account...`, then the browser lands on **`/pending-approval`** — **not** the dashboard, and **not** any "check your email" screen. The page explains the account is waiting for approval. In Supabase, the `profiles` row exists with `role='patient'`, `approved=false`, `active=true`, and a `patient_code` of the form `PT####`. **Cleanup.** Leave. `ADM-APPR-001` approves this account.

`PAT-AUTH-003` — An unapproved patient cannot reach the dashboard · P0

Preconditions. `PAT-AUTH-002` complete; Patient B not yet approved. **Steps**

1. Signed in as Patient B, type `/patient/dashboard` directly in the address bar and press Enter.
2. Then call the API directly. In a terminal, with Patient B's session cookie: `curl -i -X POST http://localhost:3000/api/appointments/create -H 'Content-Type: application/json' -b '<patient B cookie>' -d '{"slotTime":"2026-12-01T10:00:00.000Z"}'`

Expected Result. Step 1 redirects to **`/pending-approval`**. Step 2 returns **HTTP 200** and creates a `requested`/ `unpaid` appointment — this is correct: `/api/appointments/create` gates on `isProfileActive`,

not approval, because an unapproved self-signup patient must be able to hold the row they are about to pay for. It grants nothing on its own. **What must be refused is a suspended account** — see `SEC-AUTH-006`. **Cleanup.** Delete the stray appointment from Sessions → All Sessions, or leave it as a fixture for `ADM-SESS-002`.

`PAT-AUTH-004` — Registration validation (negative) · P1

Steps. On `/patient/login` → **Register Account**, attempt **Create Account** once for each row, resetting the form between attempts:

Attempt	Field changed	Value	Expected message
1	Email	`not-an-email`	An invalid-email message; the account is not created
2	Password	`abc12`	A minimum-length message; not created
3	Confirm Password	`QaTest!2024pas`	A passwords-do-not-match message; not created
4	Phone	`12345`	An invalid-phone message; not created
5	Referral Code	`ZZZZZZ`	`Code not recognized — double-check it or leave blank` in red
6	Email	`qa.patient.a@example.test` (already registered)	A "user already registered" style error from Supabase; **no second profile row is created**

Expected Result. In every case the form stays on screen with the entered values intact, the message is human-readable, and **no stack trace, database column name or row id appears anywhere on screen**.

`PAT-AUTH-005` — Sign out · P2

Steps. From `/patient/dashboard`, tap the sign-out control in the sidebar/header. **Expected Result.** The session ends, the browser lands on a public page, and a farewell banner shows for the admin-configured number of seconds (`farewell_banner_seconds`, default 6; `0` means until dismissed). Navigating back to `/patient/dashboard` now redirects to `/patient/login`.

`PAT-AUTH-006` — Forgot password · P2

Steps. On `/patient/login` tap **Forgot password**, enter `qa.patient.a@example.test`, tap **Send Reset Link**. **Expected Result.** The button reads `Sending...`, then a confirmation appears. A **← Back to Sign In** link returns to the form. (Delivery cannot be checked with a `.test` address — verify the request succeeded and that the reset landing page `/reset-password` renders.)

11.1a The dashboard landing screens

The patient portal's Overview is the screen a patient lands on after every sign-in, and the one that answers "how am I doing / what needs me / what do I do next" in that order — a strip of four figures, the activity feed, then quick actions. It is assembled by `loadPatientDashboard()`, so every figure on it is derived from rows the same loader already read: none of it can disagree with the screen it links to. The booking hub under **Book a Session** is the portal's own front door to the two things a patient may buy directly — **one** video consultation, or **one** visit at home.

`PAT-DASH-001` — The patient Overview · P1

Feature. Patient dashboard Overview (`/patient/dashboard`). **Role.** Patient. **Purpose.** Confirm the four figures, the feed, the quick actions and the conditional banners render, that every figure links to rows that agree with it, and that the first-visit tour appears exactly once. **Preconditions.** Signed in as Patient A (`PAT-AUTH-001`). Run this **twice**: once immediately after `PAT-AUTH-002`/`PAT-BOOK-003` (nothing completed yet) and once after `THR-SESS-005` has completed a session, so both the empty and the populated shapes are seen. **Test Data.** None entered.

Steps

1. Open `/patient/dashboard`.
2. On a brand-new account only: read the welcome tour that opens over the screen. Confirm it reads `1 of N`, has **Skip**, **Next** and (from step 2) **Back**, and that the highlighted ring sits over the sidebar entry each step names.
3. Tap **Next** until the last step, then tap **Done**.
4. Reload the page.
5. Read the header: `Welcome back, QA Patient A` with `Your virtual physical therapy dashboard` under it.
6. Read the four figures across the strip, left to right.
7. Tap the **Next session** figure.
8. Tap the browser's Back button, then tap the **Package sessions left** figure.
9. Tap Back again. Read the activity feed under the strip.
10. Read the four quick actions at the foot: **Book a session**, the health-profile action, **Your sessions**, **Your payments**.

Expected Result.

- The tour appears **only on the first visit**. Step 3 writes `profiles.onboarding_seen_at` through `/api/patient/dismiss-onboarding`, so the reload at step 4 shows no tour, and it never returns. **Skip** does the same thing as **Done** — both dismiss it permanently.
- The strip reads: **Next session** (date + time and the therapist's name, or `—` with `Nothing booked yet`), **Sessions done** (count, `Your history builds up here` at zero), **Package sessions left** (count, `You don't own a package yet` when there are none), and **Health profile**.
- **Health profile before the therapist's first record reads `—` on a slate accent** with `Your therapist fills this in at your first session` — **not** `0%` on amber. An amber zero here is the `PAT-HP-001` defect: it asserts the patient is behind on something nobody has asked them for.
- Step 7 navigates to `/patient/dashboard/sessions`; step 8 to `/patient/dashboard/packages`. Every figure is a link and lands on the rows it counted.
- With no history the feed reads `Bookings, payments and health-profile updates show up here as they happen.` Once there is history, items still waiting on the patient (`needsYou`) are **pinned above** everything else regardless of date, and no single title repeats more than the cap allows.
- The health-profile quick action changes wording with state: `See your health profile` / `Your therapist fills this in at your first session` while locked, `Finish your health profile` / `N questions left, about 2 minutes` while partly answered, `Review your health profile` while complete.
- **Conditional banners.** While the profile is the patient's to write and unfinished, an amber banner reads `You're N of M questions into your health profile.` (or the `Add to your health profile…` wording at zero) with a progress bar and `Finish it →`. While a recommendation or a proposed time is waiting, a teal card reads `Your therapist has recommended a programme` or `Your therapist has proposed a time` and links to `/patient/dashboard/suggested` — the cards themselves live there, never duplicated here.

- If invites are enabled (`ADM-SET-023`'s neighbour, `invite_rewards_enabled`), the invite card renders **below** the overview, never as a banner over it, and its claim field is gone once any session has been paid for.
- No console error. No horizontal scroll at 390 × 844.

Cleanup. None. Stay signed in.

`PAT-DASH-002` — The booking hub inside the portal · P1

Feature. `/patient/dashboard/book`. **Role.** Patient. **Purpose.** Prove the portal's booking hub offers exactly the two things a patient may buy directly, and nothing that must come from a recommendation. **Preconditions.** `SETUP-CAT-001` has created the three categories. For the home-visit half: `ADM-SET-013` on, `SETUP-AREA-001` and `SETUP-HVPKG-001` done (HV1 = 1 visit, HV2 = 4 visits). **Test Data.** None entered.

Steps

1. In the sidebar tap **Book a Session**.
2. Read the two group headings and their blurbs.
3. Read every card under **Single online consultation**.
4. Read every card under **Home visits**.
5. Tap the card titled `QA Back & Spine Care`.
6. Tap the browser's Back button, then tap the HV1 card.
7. Return to `/patient/dashboard/book`. Ask an admin to switch **Home Visit** off (`ADM-SET-013`), then reload this screen.

Expected Result.

- Heading **Book a Session**, subtitle `Video consultations and home visits, in one place.`
- Group 1 is `Single online consultation` — `One video session with a therapist, booked for a specific concern.` One card per **active** treatment category, each showing its title, description, `N min · online` and its consultation price.
- **Each group's heading reads as a level above the cards, not as one of them.** A filled square tile in the mode's own colour — **teal** with a video glyph for online, **amber** with a house glyph for home visits — then the heading in the display face, a count chip (`2 OPTIONS`), the one-line blurb, and a hairline that starts in that colour and fades out. The two headings used to be `text-sm font-bold` slate, the identical weight and ink the card titles below them use, so the sections did not separate and the line telling a patient which one comes to their house disappeared into the list. If a heading is the same size and weight as the card titles under it, that regression is back.
- The count chip must **agree with the cards beneath it** — three cards, `3 OPTIONS`; one card, `1 OPTION` singular.
- Group 2 is `Home visits` — `A therapist comes to you. We'll check your pincode before anything is charged.` **HV1 (1 visit) appears. HV2 (4 visits) does not.** A multi-visit home package is a programme and comes from a care plan; if HV2 is on this screen that is the P0 `PAT-HV-005` defect. Each card carries `Single visit · N min at home` and either `Travel included` or `Travel charged separately, by area`.
- **No session package (programme) appears anywhere on this screen**, at any price, with or without a Buy control.
- Step 5 navigates to `/book?category=<id>` with that category preselected. Step 6 navigates to `/book-home-visit?package=<id>`.
- After step 7 the **Home visits** group is **absent** entirely and the sidebar's home-visit affordances go with it. With no categories **and** no home-visit packages the whole card reads `Nothing is available to book right now — please check back shortly.`

Cleanup. Switch **Home Visit** back on if later home-visit tests follow.

11.2 The booking wizard — happy paths

`PAT-BOOK-001` — Reach the wizard from the public site · P1

Steps

1. Open `/`.
2. Tap **Book a session** in the header (or the **Book now** card in the connector grid at the foot of the page).
3. Observe the URL and the wizard header.

Expected Result. The URL is `/book`. The dark header reads **Step 1 of 3** and **Book Virtual Physical Therapy Session**. Because no concern is chosen yet, the subtitle reads *"HD Video Call & Custom Rehab Plan — pricing shown once you pick a concern"*.

`PAT-BOOK-002` — Step 1 auto-picks and the lead-time boundary (scenario TIME-A) · P0

Preconditions. `SETUP-CAT-001` has created the three treatment categories. Online lead time is the default 12 hours. **Test Data.** Simulate `2026-09-10T10:00`.

Steps

1. In the Debug bar set **Simulate now** to `2026-09-10T10:00` and tap **Set**.
2. After the reload, open `/book`.
3. Read the calendar: note the month heading, which day is marked as today, and which day cells are greyed.
4. Read the hour chips: note the first one offered.
5. Read the language chips: note which one is selected.
6. Tap the day cell **11**.
7. Read the hour chips again.
8. Tap the hour chip **09:00**.
9. Tap **Continue**.

Expected Result

- Step 3: heading `September 2026`; **10** is today; every cell before 10 is greyed and not tappable.
- Step 4: the earliest offered hour on 10 September is **22:00** (10:00 + 12h). Hours before 22:00 are absent or disabled. A "we picked this for you" hint marks the auto-chosen date, hour and language.
- Step 5: the first admin-configured booking language is selected — `English` by default.
- Step 7: after moving to 11 September, the **full** hour list is offered (every hour on 11 September clears a 12h lead time from 10:00 on the 10th), and the hour has been **re-preselected to that day's earliest**, not carried over from 22:00.
- Step 9: the wizard advances to **Step 2 of 3**. **No appointment exists yet, no account is created, and nothing is charged.**

Cleanup. Stay in the wizard for `PAT-BOOK-003`.

`PAT-BOOK-003` — Guest books and pays successfully (the main journey) · P0

Purpose. The single most important path in the application: a brand-new visitor becomes a paying patient with a booked session. **Preconditions.** `PAT-BOOK-002` left the wizard on Step 2 with 11 September 2026

09:00 and `English` selected. Razorpay test keys configured. **Test Data.** Patient A from §8.3. Payment: UPI `success@razorpay`.

Steps

1. Tap **Full Name**. Enter `QA Patient A`.
2. Tap **Email**. Enter `qa.patient.a@example.test`.
3. Tap **Create Password**. Enter `QaTest!2024pass`.
4. Tap the phone field. Enter `+91 98765 43210`.
5. Tap **Confirm Password**. Enter `QaTest!2024pass`.
6. Leave **Referral Code** blank.
7. Tap the **What would you like help with?** dropdown. Select `QA Back & Spine Care — ₹1,999 / 60 min`.
8. Tap the **Anything else we should know?** box. Enter `Desk job, pain worse after sitting all day. Goal: sit through a full workday.`
9. Tap the telehealth consent checkbox.
10. Tap **Review Booking →**.
11. On Step 3, read every summary row.
12. Tap **Request Booking**.
13. When the Razorpay checkout opens, choose **UPI**. Enter `success@razorpay`. Submit and complete the simulated approval.
14. Wait for the checkout to close.

Expected Result

- Step 10: the wizard advances to **Step 3 of 3**. The dark header subtitle now reads `₹1,999 INR • 60-Min HD Video Call & Custom Rehab Plan`.
- Step 11: **Name** `QA Patient A`; **Email** `qa.patient.a@example.test`; **Preferred Time** 11 September 2026, 09:00, in local format; **Language** `English`; **Concern** `QA Back & Spine Care`; **Session Fee** `₹1,999 INR`. Two notices are present: the Razorpay/secure line and *"Free cancellation up to 24 hours before your slot. Cancelling within 24 hours of the slot isn't eligible for a refund."*
- Step 12: the button shows `Please wait...`; the Supabase account is created and signed in **with no email-confirmation step**; the appointment is created; Razorpay checkout opens.
- Step 14: Step 3 is replaced by a green tick, **Payment Confirmed**, the sentence about confirming the slot and sending the link, and a **Go to Dashboard** button.
- **Data state:** exactly **one** appointment for Patient A — `status='requested'`, `payment_status='paid'`, `therapist_id=null`, `visit_mode='online'`, `duration_minutes=60`, `concern='QA Back & Spine Care'`, `preferred_language='English'`, timezone recorded. Exactly **one** `payments` row for that Razorpay order. Patient A's `approved` is now **true** (flipped by `create-order`).
- **Cross-role:** Admin → Sessions → All Sessions lists it as unassigned and raises the badge; Admin → Money → Transactions lists the ₹1,999 payment; **no therapist sees it yet**.

Cleanup. Leave everything. This is the root fixture for most of the plan.

`PAT-BOOK-004` — The lead-time boundary is computed, not hardcoded (scenario TIME-B) · P1

Steps. Simulate `2026-09-12T18:00`, tap **Set**, open `/book`. **Expected Result.** Today is **12** September. The earliest bookable slot is **13 September at 06:00** — 18:00 + 12h is 06:00 the next day, and no hour remains on the 12th. The 12th is either fully greyed or offers no hour chips; the 13th is preselected. **Cleanup.** Reset to real time.

`PAT-BOOK-005` — Late-night rollover (scenario TIME-C) · P1

Steps. Simulate `2026-09-10T23:30`, open `/book`. **Expected Result.** The earliest bookable date is **11 September** (23:30 + 12h = 11:30 on the 11th, so the 11th's later hours qualify while the 10th has none left). The 10th is greyed. The auto-picked hour is the 11th's earliest hour at or after 12:00. **Cleanup.** Reset to real time.

`PAT-BOOK-006` — A signed-in patient books again · P1

Preconditions. Patient A signed in, with at least one completed session so a previous therapist exists. **Steps.** Open `/book`, complete Step 1, and read Step 2. **Expected Result.** Step 2 shows a teal strip *"Booking as QA Patient A (qa.patient.a@example.test)"* and **no** name/email/password/phone/referral fields. A **"Continue with the same therapist?"** dropdown appears, defaulting to `No preference — any available specialist` and listing `QA Therapist A`. The consent checkbox and concern dropdown are still required.

`PAT-BOOK-007` — Booking a named specialist from `/team` · P1

Steps

1. Open `/team`.
2. Tap the card for `QA Therapist A`.
3. In the popup, tap the **Book with …** action.
4. Read the URL, then advance to Step 2.

Expected Result. The URL is `/book?therapist=<uuid>`. On Step 2 a teal panel reads **Requested: QA Therapist A**, their credentials beneath, and the sentence *"We'll book you with them if they're free at your chosen time. If not, another specialist takes the session and you'll see who before it starts."* It has a **Remove** button. The "Continue with the same therapist?" dropdown is **hidden** while this panel is shown. It must **never** be worded as a confirmed booking with that therapist.

> **Environment note.** This resolution happens **in the browser** against `public_therapist_profiles`, because `/book` is ISR-cached. In a sandbox whose Chromium has no outbound network, the chip never renders and this test fails on a working feature. Check first: run `fetch(location.origin)` and then a direct fetch of `<SUPABASE_URL>/rest/v1/` from the page console. "Failed to fetch" there, with the same call succeeding from Node, means the environment — not the feature.

`PAT-BOOK-008` — A hidden or suspended therapist link degrades silently · P2

Preconditions. An admin has switched `QA Therapist C`'s **Show on team** off (or suspended them). **Steps.** Open `/book?therapist=<Therapist C uuid>`, advance to Step 2. **Expected Result. No** requested-therapist panel appears; the booking proceeds normally with no preference attached. No error is shown. A hand-typed or stale link must never fail a booking.

11.3 The booking wizard — negative, boundary and duplicate

`PAT-BOOK-010` — Step 1 validation · P1

Attempt	Action	Expected message
1	Deselect the language (if the UI allows) and tap **Continue**	`Please select a preferred language.`

Attempt	Action	Expected message
2	Hand-edit the URL/state to an in-the-past slot and tap **Continue**	`Please choose a time at least 12 hours from now.`

Expected Result. In both cases the wizard stays on Step 1 and no appointment exists.

`PAT-BOOK-011` — Step 2 validation, in order · P1

Attempt **Review Booking →** once per row, as a guest:

#	Missing/invalid	Expected message
1	Name blank	`Please fill in your name, email, and a password (min 6 characters).`
2	Password `abc12`	same as above
3	Email `not-an-email`	`Please enter a valid email address.`
4	Phone `12345`	`Please enter a valid phone number.`
5	Confirm password mismatched	`Passwords do not match. Please re-enter them.`
6	Referral code `ZZZZZZ` (blurred, shown invalid)	`That referral code isn't recognized. Please double-check it or clear the field to continue without one.`
7	No concern selected	`Please select what you'd like help with.`
8	Consent unticked	`Please agree to the telehealth consent terms to continue.`

Expected Result. Each message appears in the red banner at the top of the card, the entered values survive, and the wizard does not advance. **No account and no appointment is created by any failed attempt.**

`PAT-BOOK-012` — Overlapping booking is refused (client and server) · P0

Preconditions. Patient A already has a `requested` or `confirmed` session at 11 September 2026 09:00, 60 minutes. **Steps.** Signed in as Patient A, book the **same** slot again through `/book` and tap **Request Booking**. Then repeat the attempt directly against the API with Patient A's cookie and the same `slotTime`. **Expected Result.** Both refuse with: `You already have a session scheduled around this time. Please pick a different slot, or check your dashboard for existing bookings.` The API returns **409**. **No second appointment row is created.** (The client check exists for fast feedback; the server check is the one that binds.)

`PAT-BOOK-013` — An inactive category is refused server-side · P1

Steps

1. Load `/book` and select `QA Knee & Joint Care` on Step 2, but do **not** submit.
2. In a second browser as Admin Full, open Catalog → Conditions and set `QA Knee & Joint Care` to inactive.
3. Return to the wizard and tap **Request Booking**.

Expected Result. The booking is refused with `That concern isn't available any more. Please pick another one.` (HTTP 409). No appointment is created. This proves the route re-derives the category rather than trusting the ISR-cached page. **Cleanup.** Re-activate the category.

`PAT-BOOK-014` — A stale `?package=` link is answered, not ignored · P0

Steps. Open `/book?package=00000000-0000-0000-0000-000000000000`. **Expected Result.** Instead of the wizard, a panel headed **"Programmes come from your therapist now"** with the explanation *"A course of treatment is arranged by your therapist after they've seen you… Book a first session and they'll recommend the right programme."* and a **Book a first session** link to `/book`. **The wizard must not silently sell one session to somebody who came to buy six.**

`PAT-BOOK-015` — Refresh mid-wizard loses no money and creates nothing · P2

Steps. Complete Steps 1 and 2, reach Step 3, then press **F5**. **Expected Result.** The wizard restarts at Step 1 with fresh auto-picks. No appointment exists (none was created — creation happens on **Request Booking**). No charge.

`PAT-BOOK-016` — Back from Step 3 abandons the draft cleanly · P2

Steps. Reach Step 3, tap **Request Booking**, dismiss the Razorpay modal, then tap **Back**. **Expected Result.** The wizard returns to Step 2. The primary button on Step 3 reads **Request Booking** again (not "Pay … Now"), and the failed-attempt counter is reset. The unpaid appointment created by the first attempt **remains** in the patient's dashboard with a **Pay ₹1,999 Now** button — it is not deleted, and a second **Request Booking** creates a *second* appointment only if the slot does not overlap the first (it will overlap, so it is refused — see `PAT-BOOK-012`).

`PAT-BOOK-017` — Double-tap on Request Booking · P0

Steps. On Step 3, tap **Request Booking** twice in rapid succession (double-click). **Expected Result.** The button disables on the first tap (`Please wait...`). Exactly **one** appointment row exists afterwards. Exactly **one** Razorpay order is minted.

11.4 Payment outcomes from the patient's side

`PAT-PAY-001` — Payment fails, then retries successfully · P0

Steps

1. Book a fresh consultation as Patient A for 12 September 2026 10:00, `QA Knee & Joint Care`.
2. On Step 3 tap **Request Booking**.
3. In the checkout choose **UPI**, enter `failure@razorpay`, submit.
4. Read the wizard.
5. Tap **Pay ₹1,799 Now**.
6. This time enter `success@razorpay` and complete.

Expected Result

- Step 4: an error banner appears; the primary button now reads **Pay ₹1,799 Now**; the appointment already exists as `requested` / `unpaid`.
- Step 6: **Payment Confirmed** panel. **Still exactly one appointment.** The `payments` table has one row for the successful order. A failed attempt is logged (`payment_failure_log`) and appears nowhere on a patient screen.
- Money → Transactions shows **₹1,799 once**, not twice.

`PAT-PAY-002` — Dismissing the checkout · P1

Steps. Tap **Request Booking**, then close the Razorpay modal with its ×. **Expected Result.** `Payment was not completed. You can try again below.` The button reads **Pay … Now**. The unpaid appointment is visible at `/patient/dashboard/sessions` under Upcoming with a **Pay … Now** button.

`PAT-PAY-003` — Reassurance on the first failure, escape hatch on the third · P1

Steps. Fail or dismiss the payment **once** and read the card. Then fail it twice more. **Expected Result.** After the **first** failure a slate panel reads *"Nothing was lost — your booking is saved and still held as unpaid. You can try again above, or pay later from your dashboard."* A patient whose card was declined has no other way of knowing the booking survived, and the likeliest next action is closing the tab. After the third, that line is replaced by an amber panel: *"Having trouble paying? Your booking is saved as pending — you can come back and pay any time from your dashboard."* with a **Go to Dashboard →** link. Following it lands on `/patient/dashboard` with the session listed as unpaid. **The patient is not bounced to `/pending-approval`** — the first genuine attempt already approved them.

`PAT-PAY-004` — Abandon then pay from the dashboard · P0

Steps. Create an unpaid booking, close the tab, sign in again, go to **Your Sessions**, tap **Pay ₹… Now** on that row, and complete with `success@razorpay`. **Expected Result.** The same appointment becomes paid. No second appointment. The dashboard row's **Pay** button is replaced by the session's normal state. One `payments` row.

`PAT-PAY-005` — Retry after the order was already captured · P1

Purpose. Prove `create-order` re-checks a prior order rather than minting a second one. **Steps.** Pay successfully, then (before the page updates) tap **Pay … Now** again, or reload and tap it. **Expected Result.** The route finds the prior order already `paid` at Razorpay, claims the appointment as paid, and returns a message saying the booking is already paid rather than opening a second checkout. **No second Razorpay order id is written to the appointment.**

11.5 Home visit (patient)

> **Preconditions for this whole sub-section:** `ADM-SET-013` has switched **Home Visit enabled** on, `SETUP-AREA-001` has created Area 1 (`560038`, ₹150/visit), and `SETUP-HVPKG-001` has created HV1 (1 visit) and HV2 (4 visits).

`PAT-HV-001` — The public home-visit page appears only when enabled · P1

Steps. With the master switch **off**, open `/home-visit`. Then switch it on in Settings → Programmes & Home Visits → Home Visit and reload. **Expected Result.** Off: a **404** page, and the **Home visit** entry is absent from the header nav, the footer Explore column, the home page connector grid and every "Where to go next" strip. On: the page renders with the admin-configured heading and subheading, and the entry reappears everywhere.

`PAT-HV-002` — Serviceable pincode → address → book and pay · P0

Test Data. Pincode `560038`; address from Patient A §8.3.

Steps

1. Open `/book-home-visit`. Confirm the header reads **Step 1 of 4**.
2. Tap the **Pincode** field. Enter `560038`.

3. Tap **Check**.
4. Read the confirmation line.
5. Fill the address form: Address line 1 `12, 3rd Cross, Indiranagar`; line 2 `Near Metro Station`; City `Bengaluru`; State `Karnataka`; PIN `560038`.
6. Tap **Continue**.
7. On **When suits you?** pick a date at least 24 hours out and an arrival time.
8. Tap **Continue**.
9. On **About you**, choose the package `QA Home Visit — Single — 1 visit` and complete the identity fields (or confirm the "booking as" strip if signed in).
10. Tap **Continue**.
11. On **Review and pay**, read every line of the price breakdown.
12. Choose **Pay online**, tap the pay button, and complete with `success@razorpay`.

Expected Result

- Step 4: a teal line — *"Yes — we visit Indiranagar, Bengaluru. Travel to this area is ₹150 per visit."*
- Step 7: the copy states home visits need at least **24 hours'** notice (the `home_visit_lead_time_hours` setting, deliberately longer than the online 12).
- Step 11: the breakdown shows **programme price**, **travel**, and **total** as three separate figures — `₹2,499` + `₹150` = `₹2,649`. **The button must charge exactly the total shown.** Quoting one figure and charging another is a P0 defect.
- Step 12: a confirmation screen. A `home_visit_package_purchases` row exists with `payment_mode='online'`, `payment_status='paid'`, `travel_fee_paise=15000`, a `default_address_id`, and a **snapshot** of the package.
- The visit's address is **snapshotted onto the appointment** (`visit_address_*`), not referenced live — editing the saved address later must not rewrite a delivered visit.

`PAT-HV-003` — Unserviceable pincode → waitlist · P1

Steps. Open `/book-home-visit`, enter `560025`, tap **Check**. Then enter `QA Patient B` and `+91 98765 43211` and tap **Tell me when you do**. **Expected Result.** An amber panel: *"We don't visit 560025 yet."* plus *"Leave your number and we'll tell you the moment we do. Nothing has been charged."* No address form appears and no package can be chosen. After submitting: *"Thanks — we'll be in touch."* The panel also offers a link to an **online consultation**, which is available anywhere. Admin → Catalog → Service Areas shows the waitlist entry, and its badge count increases.

`PAT-HV-004` — Pincode validation · P2

Value	Expected
`56003`	`Enter a valid 6-digit pincode.`
`0560038`	`Enter a valid 6-digit pincode.` (must not start with 0)
`abcdef`	`Enter a valid 6-digit pincode.`
(blank)	The **Check** button does nothing harmful; a validation message appears

`PAT-HV-005` — A multi-visit home package cannot be bought directly · P0

Steps. In the Step 3 package picker, attempt to select `QA Home Visit Recovery — 4 Visits`. If the UI offers it, proceed and attempt payment. Also call the API directly: `POST /api/home-visit/create-order` with HV2's id. **Expected Result.** The multi-visit package is **not offered** for direct purchase, and the API refuses it.

Only a **one-visit** home package is directly purchasable — that is the patient's home consultation. HV2 can only reach a patient through a therapist's care plan.

`PAT-HV-006` — Cash on visit · P1

Preconditions. Allow cash on visit is on. **Steps.** Repeat `PAT-HV-002` but choose **Pay at the visit** on Step 4. **Expected Result.** The booking completes with **no Razorpay checkout**. The purchase row has `payment_mode='cash'` and `payment_status='unpaid'` — **for its whole life**, with a real confirmed visit hanging off it. The patient's Payments screen must not describe this as an outstanding failure. Admin → Money → Payouts → Cash Ledger will show the collection once the therapist records it.

`PAT-HV-007` — Cash refused when the switch is off · P2

Steps. Admin turns **Allow cash on visit** off. Patient repeats the flow. **Expected Result.** The cash option is absent. Calling `/api/home-visit/book-cash` directly returns `Paying at the door isn't available right now — please pay online instead.`

11.6 Sessions, joining and cancellation

`PAT-SESS-001` — The Sessions screen, its filters and its view switch · P1

Preconditions. Patient A has at least one upcoming, one past and one cancelled session, and at least one home visit. **Steps.** Open `/patient/dashboard/sessions`. Tap **Upcoming**, **Past**, **Cancelled** in turn. Then tap the **Video** and **Home visit** filters. Then switch **List** → **Calendar** and back. **Expected Result.** One list, not two. The Video/Home visit filter appears **only** because this patient has both kinds; a patient with only video sessions must not see it. The Calendar view renders **the same server-rendered cards**, so the two views can never disagree about a session. Filter chips are hidden entirely when only one of them would have rows behind it. The list ends with a pager showing "x–y of n", a per-page control, and Previous/Next that grey out at the ends.

`PAT-SESS-002` — An unassigned session reads as Requested · P2

Expected Result. A paid but unassigned session shows a `Requested` status pill and **no therapist name**. It must not claim a therapist it does not have.

`PAT-SESS-003` — Tap to Join appears only inside the join window (scenario TIME-E) · P1

Preconditions. A `confirmed` session with an assigned therapist and a Meet link. Join window before/after default to 15 minutes. **Steps.** With the real clock well before the slot, look at the session card. Then simulate a time 10 minutes before the slot and reload. **Expected Result.** Before the window: the control is inert and does not offer a call. Inside the window: **Tap to Join** is live and opens the Meet link. **Reminder:** this control reads the *simulated* clock; the server's own gates do not.

`PAT-SESS-004` — After the cutoff every surface says Session Completed (scenario TIME-F) · P1

See `XR-CUTOFF-001` for the cross-role version. Simulate 90 minutes after the slot (past the default 60-minute `session_completed_after_minutes`). **Expected Result.** The control reads **Session Completed** on the patient's card — and on the therapist's and the admin's, identically.

`PAT-CANCEL-001` — Cancel outside the refund window → full refund (scenario TIME-G) · P0

Preconditions. A **paid**, `confirmed` session whose slot is more than 24 hours away in **real** time (the refund maths runs server-side, so a simulated clock will not move it). **Steps.** Open the session card, tap

Cancel Session, read the dialog, optionally enter a reason, confirm. **Expected Result.** The dialog reads *"Cancel this session and refund the payment? You can add a reason (optional):"*. After confirming, the card shows `Cancelling...` then a cancelled state. The appointment becomes `cancelled` with `refund_status='processed'` and the refund amount recorded. Admin → Money → Summary shows the refund reducing **Net Revenue** while **Gross Revenue** is unchanged. **No therapist cut is reversed** — a cancelled session never earned one. If the patient was hospital-referred, the **partner commission is reduced**, because it is taken on net.

`PAT-CANCEL-002` — Cancel inside the window → no refund, and the dialog says so (scenario TIME-H) · P0

Preconditions. A paid session whose slot is **less than 24 hours** away in real time. **Steps.** Tap **Cancel Session** and read the dialog **before** confirming. **Expected Result.** The dialog reads *"Cancel this session? It's within 24 hours of your slot, so this won't be refunded. You can add a reason (optional):"*. After confirming, the appointment is `cancelled` with **no refund**. Money → Summary is unchanged for refunds. This forfeited amount stays in the clinic's share — it must not deduct a therapist cut, because the session was never completed.

`PAT-CANCEL-003` — Home visits use their own refund window · P1

Repeat `PAT-CANCEL-001` / `002` against a home visit. **Expected Result.** The window used is `home_visit_cancellation_refund_hours` (default 24, independently configurable). Changing the online window in Settings must **not** change the home-visit dialog, and vice versa.

`PAT-SESS-006` — Rate a completed session · P2

Steps. After a therapist marks a session complete, open the patient's card and submit a rating of `5` with feedback `Very clear explanation, the exercises helped.` **Expected Result.** The rating is stored once. A second attempt returns `You've already rated this session.` A rating of `0` or `6` is refused with `Rating must be between 1 and 5`. A no-show session refuses rating entirely: `This session was marked as a no-show — there's nothing to rate.`

11.7 Health Profile (patient side)

`PAT-HP-001` — Before the therapist's first record, the patient is read-only · P0

Preconditions. Patient A exists with **no** condition profile yet. **Steps.** Open `/patient/dashboard/health-profile`. Then return to `/patient/dashboard` and look at the overview. **Expected Result**

- On the Health Profile screen: **no** "answer your questions" call to action and **no** answered counter — they are **absent, not greyed out**.
- The **reports uploader is present and usable** — it is the one useful thing the patient can do beforehand.
- On the dashboard overview: the health-profile cell reads **`—` on a slate background**, never `0%` on amber, and there is **no amber banner**.
- The waiting panel **names the session it is waiting on**, by date — *"Your session on 11/09/2026 is when this gets filled in"* before the session, and *"Expected at your session on 11/09/2026. If it still isn't here in a day or two, tell us and we'll chase it."* after it. A locked screen with no date on it leaves the patient unable to tell whether the wait is normal or whether they have been forgotten. A patient with no session yet simply sees no date line — never a placeholder.
- Attempting the API directly (`POST /api/patient/condition-profile/save-draft` with the patient's cookie) is **refused** — the lock is enforced in `submit`, in `save-draft`, and by an insert policy on

`condition_change_requests`. A UI-only lock would be cosmetic.

`PAT-HP-002` — After the therapist's first fill, the wizard unlocks · P0

Preconditions. `THR-HP-002` has triaged Patient A as Orthopaedic and written the first record. **Steps.** Open `/patient/dashboard/health-profile`. **Expected Result.** The seven orthopaedic answers are shown **as answers, not as inputs**. The severity gauge, the pain-area chips and the progress line render. The answered counter appears. Attribution is correct: the counter must **not** claim the patient answered questions the clinician wrote — answers written by the therapist are attributed to the therapist.

`PAT-HP-003` — The intake wizard is one question at a time · P1

Steps. Tap the call to action to review/update answers. **Expected Result.** A **pop-up wizard showing one question at a time**, never a seven-field form rendered on the dashboard. Each question shows its `helpText` in the patient's own words. Progress through all seven with the §8.4 answers.

`PAT-HP-004` — Submitting a change goes to review · P1

Steps. Change `severity` from `6` to `4` and submit. **Expected Result.** A confirmation that the clinic will review it. The word used for the reviewer on a **patient** screen is **"the clinic"** — never "admin", never "us". A second submission while one is pending is refused: `Your last change to your health profile is still being checked by the clinic.`

`PAT-HP-005` — The record exports as a PDF, not JSON · P1

Steps. Tap the export/download control on the Health Profile. **Expected Result.** A typeset **PDF** downloads, named `QA Patient A_PT####.pdf` (name + patient code). It contains the intake answers and the Pain Map data. **Session notes are excluded from every format.** `?format=json` still serves raw JSON for portability but is not linked anywhere in the UI. **Boundary.** A patient whose name contains non-Latin characters (set the name to `पूजा शर्मा` temporarily) must still export successfully — the PDF encoder transliterates rather than throwing a 500.

`PAT-DOC-001` — Upload a report · P0

Steps. On the Health Profile, open the reports panel. Tap the upload control. Choose `Spine_Report_E2E.pdf`. Pick the type `Scan or X-ray`. Submit. **Expected Result.** The file appears in the list with its type label and upload date. Only **metadata** is stored in the database — the file itself is in the private `medical-reports` bucket. Tapping the file opens it through `/api/medical-documents/view`, which mints a **120-second signed URL**. Copying that URL and opening it after two minutes must **fail** — the link is deliberately short-lived.

`PAT-DOC-002` — Upload limits · P1

Attempt	Expected
`Oversize_Report_E2E.pdf` (>10 MB)	Refused before or during upload with a size message; nothing is stored
`Not_Allowed_E2E.txt`	The file picker does not accept it (`accept=".pdf,.jpg,.jpeg,.png,.webp,.heic,.heif"`); a forced API call is refused with `Upload a PDF or a photo (JPG, PNG, WEBP or HEIC).`
No type selected	`Pick what kind of report this is.`
No file chosen	`Choose a file to upload.`
A 21st file	Refused — the cap is **20 files per patient**, enforced in the upload route

`PAT-DOC-003` **— Deleting and re-uploading is the only correction path · P2**

Steps. Delete `Posture_Assessment_E2E.pdf`, then upload it again. **Expected Result.** Deletion succeeds and the row disappears. **There is deliberately no edit/update path** — correcting a report means delete + re-upload, so the row and the stored object can never describe different things.

11.8 Suggested Sessions, care plans and credits

`PAT-SUGG-001` **— The Suggested Sessions entry appears only when something waits · P1**

Steps. Before any therapist has suggested anything, check the sidebar. Then have `THR-SUGG-001` send a suggestion, and reload. **Expected Result.** Before: **no** Suggested Sessions entry. After: the entry appears **above** Your Sessions — something waiting on the patient's answer outranks a list of what is settled.

`PAT-SUGG-002` **— Accept a therapist-proposed time · P0**

Preconditions. `THR-SUGG-001` has proposed a slot on an active programme with credits remaining. **Steps.** Open `/patient/dashboard/suggested`. Read the card. Tap **Accept this time**. **Expected Result.** The card resolves to a booked state. An appointment is created via the package-session path: **auto-assigned to the locked therapist, auto-confirmed, and given its own Meet link**. `sessions_used` on the purchase increases by exactly **one**. A `reserve` / `consume` ledger entry is written with an idempotency key derived from the appointment id. The therapist's dashboard shows the new session. **A declined suggestion would have claimed nothing** — a suggestion is its own row, never an appointment in a new status.

`PAT-SUGG-003` **— Decline · P2**

Steps. Tap **Decline** on a pending suggestion. **Expected Result.** The card shows `Declined`. No appointment. No credit moves. The therapist's control shows the declined state.

`PAT-SUGG-004` **— Button spam and a dropped connection · P0**

Steps. Tap **Accept this time** three times rapidly. Then, on a second suggestion, throttle the network to Offline and tap **Accept**. **Expected Result.** Spam: exactly **one** appointment and **one** credit consumed. The control guards with a synchronous ref, so the second tap never reaches the server. Offline: the card **does not clear optimistically** — the patient is left exactly where they were, with an error, and can retry. The partial unique index allows at most one pending suggestion per purchase, so a double tap cannot create two.

`PAT-SUGG-005` **— A lapsed suggestion cannot be accepted · P1**

Preconditions. A pending suggestion whose slot is now inside the booking lead time. **Expected Result.** The card reads **Suggestion lapsed** and offers no Accept. **Nothing writes an "expired" status** — `status` records explicit human actions only; lapse is computed at read time. Attempting acceptance via the API is refused.

`PAT-CARE-001` **— Read a therapist's recommendation · P0**

Preconditions. `THR-CARE-001` has written a care plan recommending Package P1, **and `ADM-CARE-004` has approved it.** Until the clinic approves, the patient must see nothing — check that first: an unapproved recommendation absent from this screen is the feature working, not a missing fixture. **Steps.** Open `/patient/dashboard/suggested` (and `/patient/dashboard/health-profile`). **Expected Result.** An offer card showing the programme name, **session count**, **price**, validity, the therapist's *"Why this, for this patient"* text written **to** the patient, and their instructions. The same rows render on the therapist's chart and on the patient's Health Profile — one record, two readers. **There is no price field the therapist could have set**: the price comes from the admin's catalog row.

`PAT-CARE-002` — Accept and pay for a recommended programme · P0

Steps. Tap the buy/accept action on the offer card. Complete Razorpay with `success@razorpay`. **Expected Result**

- The price charged is **re-derived server-side** from the live catalog row. It must equal the amount shown on the card.
- A `patient_package_purchases` row is created with a **frozen `package_snapshot`** and `sessions_granted = 6`.
- **Session credits are granted: exactly 6.** Not 5, not 12.
- The care plan's status becomes `accepted`. **The thread is now closed** — a later recommendation opens a *new* plan with `supersedes_id` set.
- The payment does **not** refresh into an empty screen. In place of the offer card the patient reads *"Payment received — N sessions are yours"*, who will run them, and one next step. **If the screen goes blank after paying, that is a P0 defect** — it is the highest-intent moment in the product.
- **Your Programmes** appears in the sidebar (not "Your Packages") and lists the programme with `6 sessions`, `0 used`, `6 remaining`.
- A second attempt to buy the same plan is refused: `You've already bought this plan.`

`PAT-CARE-003` — A recommended home-visit programme quotes travel correctly · P0

Preconditions. A care plan recommending HV2 (4 visits), patient address in a ₹150 area. **Expected Result.** The offer card shows **programme + travel + total** as three figures, with travel charged **per visit**: `₹8,999 + (₹150 × 4) = ₹9,599`. A card that printed `₹8,999` on a button that charged `₹9,599` would be a P0 defect — travel is per visit, not per purchase. Additionally: if an admin switches **Home Visit enabled** off, `/api/care-plan/create-order` must refuse the purchase with `Home visits aren't available right now. Please talk to your therapist.` A recommendation written before the service stopped must not stay purchasable.

`PAT-SCHED-001` — The scheduler opens answered, not empty · P0

Feature. Everything needed to lay out the run is already known — how many sessions, how often the clinician recommended, the programme's gap and weekly rules, the lead time, the validity. Handing the patient a blank calendar asks them to redo arithmetic somebody has already done, immediately after paying.

Preconditions. A paid 6-session programme recommended at **2 a week**, with `min_gap_hours` 48 and `max_sessions_per_week` 2.

Steps. Tap **Choose my times** (straight after payment) or **Schedule sessions** (from Your Programmes). Read what is already selected. Change one date. Read the panel again.

Expected Result

- Dates are **already chosen** — as many as the bulk limit allows — spaced roughly 3 days apart, none inside the 12-hour lead time, none past the programme's expiry, and no more than 2 in any calendar week.
- Every proposed session is at the **same time of day**. A course at one hour is one thing to remember rather than six.
- The panel says *"We've picked N times for you — spaced 2 a week, the way your therapist recommended."*

- After an edit the heading becomes **Your times** and a **Put the suggested times back** link appears. The suggestion must **not** silently reapply itself — that is the calendar overruling the person using it.
- Nothing is booked until Confirm. The proposal is a suggestion; the server re-checks every rule on submit.

`PAT-SCHED-002` — A clashing slot can be fixed without starting over · P1

Steps. Deliberately pick two sessions 24 hours apart on a programme with a 48-hour minimum gap, and confirm. **Expected Result.** *"N of M sessions scheduled"*, with the too-close one marked and the reason naming the gap. An amber line confirms the others **are** booked and nothing was charged twice. A **Pick another time** button returns to the calendar with only what is still unspent — the booked ones are gone from the selection, since re-offering them would invite booking the same slot twice.

`PAT-SCHED-003` — Unbooked sessions keep asking · P0

Feature. The one thing a patient can buy and then receive nothing for. Paying is the hard part and it is done; what is left is a calendar step on a screen they have to think to visit. **Steps.** Pay for a programme and tap **I'll do it later**. Open `/patient/dashboard`. **Expected Result.** The activity feed carries a **needsYou** item — *"N sessions still to book"* — naming the programme and linking to Your Programmes. It stays until the balance is spent and disappears when it is. The figure must agree with the Your Programmes card exactly; a feed claiming a balance that screen disagrees with is a P0.

`PAT-CARE-004` — A declined or withdrawn recommendation · P2

Steps. Decline a recommendation from the offer card. Separately, have an admin withdraw one (`ADM-CARE-002`). **Expected Result.** Declining closes it for the patient; withdrawing removes it from the patient's offers with the plan marked withdrawn. **A purchased plan can never be withdrawn** — the honest lane is a refund.

`PAT-PKG-001` — Spend credits by booking package sessions · P0

Preconditions. `PAT-CARE-002` complete: 6 credits, therapist locked. **Steps.** Open `/patient/dashboard/packages`. Use the bulk scheduler to book **3** sessions, respecting the package's rules (min gap 24h, max 3/week). **Expected Result.** Three appointments are created, each **auto-assigned to the locked therapist, auto-confirmed, each with its own Meet link** — never through the admin's per-session assign flow. The widget reads `6 sessions · 3 used · 3 remaining`. Three `reserve` ledger entries exist, each keyed `reserve:<appointment_id>`. **Boundary.** Attempting a 4th session in the same week is refused by the package's max-per-week rule. Attempting more than `package_bulk_schedule_max` (default 8) slots in one request is refused with `Too many slots in one request.` Two slots less than 24h apart are refused by the minimum-gap rule.

`PAT-PKG-002` — Credits cannot be overdrawn · P0 [SQL]

Steps. Book all 6 sessions, then attempt a 7th. **Expected Result.** Refused with `Every session in this programme is already used.` In the database, a ledger row that would overdraw fails the CHECK constraint and takes its transaction with it — **an impossible balance is impossible, not merely unwritten**.

`PAT-PKG-003` — A locked therapist's clash never fails the booking · P1

Preconditions. The locked therapist already has a session at the requested time. **Expected Result.** The booking still succeeds, but the session lands **`requested` and unassigned** in the admin queue rather than failing. The patient is not told to pick another time.

`PAT-PKG-004` **— An expired programme · P2**

Preconditions. A purchase whose `expires_at` has passed. **Steps.** Load `/patient/dashboard` (the expiry sweep runs at the top of that render). **Expected Result.** The purchase's status moves `active → expired` on that page load — **there is no cron**; the sweep is lazy and idempotent. The widget stops offering booking. A ledger `expiry` entry records the balance that lapsed.

11.9 Payments screen, profile and addresses

`PAT-PAY-010` **— Receipts · P2**

Steps. Open `/patient/dashboard/payments`. **Expected Result.** Every paid session and purchase is listed once, with amount, date and what it was for. A cash-on-visit home purchase appears with its cash status and is **not** presented as a failed payment. Amounts match Money → Transactions exactly.

`PAT-PAY-011` **— The patient is told about their refund · P0**

Preconditions. For one patient: a session cancelled outside the window and refunded in full, a **completed** session partially refunded by `₹500` with a stated reason, a cash home visit at `manual_pending`, a refund that failed at the gateway, and a session cancelled **inside** the window. **Steps.** Open `/patient/dashboard/sessions`, then `/patient/dashboard/payments` and open each receipt's detail, then `/patient/dashboard` and read the feed. **Expected Result.** In the patient's own words on every surface: `₹1,200 refunded`, `₹500 refunded`, `₹500 coming back to you`, `Refund didn't go through — please contact us`. The one cancelled inside the window says **`No Refund`** with the window on hover and **no refund line at all** — it must never be announced as a refund. The full and partial refunds carry the date and the clinic's stated reason as given, plus "It can take a few working days to reach your account". The **partial refund on a completed session** appears on the card, on the Payments row and in its detail — its receipt still reads `Completed`, because a delivered session that was partly refunded is not a refunded one. The failed refund is a **pinned** `needs you` feed item. Nothing on any of these screens shows an admin-voice string (`Hand back ₹500`, `Refund failed`, `No refund due`).

`PAT-PAY-012` **— A refund the patient's database predates · P1**

Steps. Against a database whose `appointments` predate `refunded_at`, open a refunded session and its receipt. **Expected Result.** The amount and the state still read correctly; the **date line is simply absent** rather than showing a guessed one, and no screen errors. The columns are read in their own query, so a database missing the migration loses the refund date and **not** the sessions list.

`PAT-PROF-001` **— Edit Profile sections · P2**

Steps. Open `/patient/dashboard/profile`. Walk the five sub-sections: **Photo**, **Personal Details**, **Contact Details**, **My Addresses**, **Account Security**. **Expected Result.** Each is reachable from the sidebar's child list. Instant fields (photo, some details) save immediately; **gated** fields (the ones an admin must approve) submit a profile change request instead of writing directly, and say so. The request appears in Admin → Today → Approvals.

`PAT-ADDR-001` **— Address book · P2**

Steps. In **My Addresses**, add the Patient A address, then add a second, then remove the second. **Expected Result.** Both are saved and selectable at home-visit checkout. Removing one does **not** alter any visit already booked — a visit's address is snapshotted onto the appointment at purchase.

`PAT-EMPTY-001` **— Empty states across the patient portal · P2**

Preconditions. A freshly approved patient with nothing at all. **Expected Result.** The sidebar shows only **Overview**, **Book a Session**, **Health Profile**, **Edit Profile**, plus **Back to Home** at the foot of the nav. Sessions, Packages, Payments and Suggested Sessions are **absent**. The Overview shows a friendly empty feed and quick actions, not a blank panel or a zero-filled table.

---

12. Therapist test plan

12.0 Feature guide — the therapist's world

A therapist applies, waits for admin approval, sets a roster, is assigned patients, delivers sessions, writes clinical records, recommends treatment, and gets paid a revenue share on what they **delivered**.

Five rules shape almost every screen:

1. **Availability is three separate things and reads as three things.** A **weekly schedule** (what they normally work, expressed as working *periods*, not hourly cells), **exceptions** (one date that differs), and **time off** (`profiles.on_leave`, off the roster entirely). Leave never clears the schedule. An exception never edits the weekly template.
2. **Availability never touches an appointment.** Removing hours a session is booked into names who is affected and says the session stays as booked. Nothing cancels, moves or flags it. The booking wins.
3. **A therapist suggests; the patient books.** A therapist can propose a time on a programme locked to them. No slot is held. Only the patient's acceptance spends a credit.
4. **A therapist picks a package, never a price.** There is no price, session-count or discount field on a care plan version. Those columns do not exist.
5. **A patient's phone is masked, their email is not loaded at all**, and revealing the number is logged. Every cross-role free-text field is scanned.

12.1 Application, approval and access

`THR-AUTH-001` — Apply to join · P0

Steps

1. Open `/therapist/login`. Confirm two tabs: **Sign In** and **Apply to Join**.
2. Tap **Apply to Join**.
3. Tap **Full Name**. Enter `QA Therapist A`.
4. Tap **Email Address**. Enter `qa.therapist.a@example.test`.
5. Tap the phone field. Enter `+91 90000 10001`.
6. Tap **Qualifications & License / Council Reg No.** Enter `MPT (Ortho), KSCP Reg 44821`.
7. Tap **Password**. Enter `QaTest!2024pass`.
8. Tap **Confirm Password**. Enter `QaTest!2024pass`.
9. Tap **Submit Application**.

Expected Result. The button reads `Submitting...`, then the form returns to the **Sign In** tab with a confirmation that the application is with the clinic. The `profiles` row exists with `role='therapist'`, `approved=false`, `active=true`. **No "check your email" step appears anywhere.** The application shows in Admin → Today → Approvals and raises that tab's badge. **Cleanup.** Leave for `ADM-APPR-002`.

`THR-AUTH-002` — An unapproved therapist is held at the door · P0

Steps

1. Sign in at `/therapist/login` as `qa.therapist.a@example.test` before approval.
2. Then, with that session cookie, call `POST /api/therapist/save-availability` directly with any valid body.

Expected Result. Step 1 lands on **`/pending-approval`**, not the dashboard. Step 2 returns **403** with `Your account is not active — it is either awaiting admin approval or has been suspended.` **The API refusal is the one that matters** — a valid cookie must not be able to call around the UI.

`THR-AUTH-003` — Sign in after approval · P0

Preconditions. `ADM-APPR-002` approved Therapist A. **Expected Result.** Sign-in lands on `/therapist/dashboard`. The sidebar shows **Overview**, **Availability**, **Sessions**, **Earnings**, **My Patients**, **Edit Profile**, with **Back to Home** at the foot of the nav directly above Collapse (with children Photo / Public Details / Credentials / Account Security).

12.1a The dashboard landing screens

The therapist Overview is where a clinician starts every shift: what is on today, what is owed, and what they have not written up yet. Edit Profile is the screen that draws the line between a detail they own outright and a credential a patient relies on.

`THR-DASH-001` — The therapist Overview · P1

Feature. `/therapist/dashboard`. **Role.** Therapist. **Purpose.** Confirm the header states what this therapist is paid and rated, that the four figures agree with the screens they link to, and that every quick action lands somewhere real. **Preconditions.** `THR-AUTH-003`. Best read after `THR-SESS-005` has completed at least one session, so **Notes to write** and **Owed to you** are non-zero.

Steps

1. Open `/therapist/dashboard`.
2. Read the header block: name, credentials, `Your Revenue Share: N%`, `Your Rating: …`.
3. Read the four figures: **Today**, **Upcoming**, **Notes to write**, **Owed to you**.
4. Tap **Notes to write**.
5. Tap Back, then tap **Owed to you**.
6. Tap Back. Read the activity feed.
7. Tap the quick action **Set your availability**.
8. Tap Back, then tap each of **Your assigned sessions**, **Patient health profiles** and **Earnings and payouts** in turn, returning each time.

Expected Result.

- Header: `Welcome, QA Therapist A`, the credentials string, `Your Revenue Share: 60%` (whatever `ADM-PEOP-006` set), and `Your Rating: No ratings yet` until a patient has rated one — then `4.0 (1 rating)`. If an admin has hidden this therapist's rating, the line ends `— hidden from public pages`.

- Greeting `Your practice today`. The headline names the next session and its patient, or reads `No sessions booked yet — keep your availability open and the clinic assigns work to it.`
- **Today** counts today's sessions with `Next at H:MM` beneath, **Upcoming** counts confirmed and awaiting-assignment work, **Notes to write** counts delivered sessions with nothing recorded (amber above zero, emerald at zero with `Every delivered session is written up`), **Owed to you** is a rupee figure with `Not yet requested` / `Payout request under review` / `Payout request sent`.
- Steps 4 and 5 land on `/therapist/dashboard/sessions` and `/therapist/dashboard/earnings`. The counts there match the figures exactly.
- Empty feed reads `Assignments, completed sessions and payouts show up here as they happen.` A session delivered with no note written is a `needsYou` item pinned above dated items.
- **Step 7 must land on `/therapist/dashboard/availability`, with the weekly schedule editor on screen.** Availability is its own route; a quick action that reloads the Overview and changes nothing is a defect — report it against this test ID.
- Step 8's three links land on `/therapist/dashboard/sessions`, `/therapist/dashboard/health-profile` and `/therapist/dashboard/earnings`.
- No figure on this screen exposes a patient's phone number or email — see `THR-SESS-003`.

Cleanup. None.

`THR-PROF-001` — Edit Profile: what saves instantly and what needs approval · P1

Feature. `/therapist/dashboard/profile`. **Role.** Therapist. **Purpose.** Prove the line between a detail a therapist owns and a credential patients rely on: the first saves on the spot, the second becomes an admin review request and locks the field until it is decided. **Preconditions.** `THR-AUTH-003`. An admin is available to decide the request in `ADM-APPR-004`. **Test Data.** Bio `Works with desk-based patients on posture-driven back pain.` · Languages `English, Kannada, Hindi` · Years of Experience `15` (a deliberate wrong value — it is declined, never approved, so Therapist A stays at the §8.8 value of `9`) · Specialist In `Spine, hip and knee rehabilitation`.

Steps

1. In the sidebar tap **Edit Profile**.
2. Under **Public Details**, tap **Short Bio**. Replace it with `Works with desk-based patients on posture-driven back pain.`
3. Tap **Languages Spoken**. Enter `English, Kannada, Hindi`.
4. Tap **Save**.
5. Reload the page and confirm both values survived.
6. Under **Credentials & Specialization**, tap **Years of Experience**. Enter `15`.
7. Tap **Specialist In**. Enter `Spine, hip and knee rehabilitation`.
8. Tap **Request Changes**.
9. Read the two fields you just changed.
10. Tap **Withdraw** beside **Specialist In**.
11. Repeat steps 7–8 for **Specialist In** only. Leave it pending.
12. Sign in as a full admin and open **Today → Approvals**. Decline **both** requested fields with the note `Send the council registration number first.`
13. Sign back in as the therapist and reopen **Edit Profile**.
14. Under **Account Security**, tap **Send password reset email**.

Expected Result.

- **Public Details save instantly.** The button reads `Saving...` then `Save`; the values survive the reload at step 5. No admin sees anything.
- **Credentials do not.** Step 8 shows `Your request has been submitted for admin review.` Each requested field is replaced by its **new** value on a slate panel with an amber `Pending Review` chip and a **Withdraw** link, and **cannot be edited again** until the request is decided. The live public profile still shows the **old** value — `/team` and the patient's therapist card must not change yet.
- Step 10's **Withdraw** returns the field to an editable input immediately, with the old value.
- After step 12's decline the field is editable again and carries `Last request declined: Send the council registration number first.` in red. Nothing on the public profile ever changed.
- The note under the credential fields reads `Changes to these fields need admin approval before they take effect.`
- **Profile photo** uploads on the spot (no review) and appears on `/team` once the therapist is visible there.
- Step 14 sends the reset by email and says so; the password is never typed on this screen.
- Negative: a **negative** Years of Experience is refused by the field's `min=0`; an empty **Full Name** submits nothing.

Cleanup. Leave the bio and languages as set — §8.8 quotes them. **No credential request may be left pending**, or `ADM-APPR-004` starts with rows it did not create and Therapist A's experience drifts from `9`.

12.2 Availability — the roster

> **Feature guide.** The editor is the same component on the therapist's own screen and on the admin's Roster. It edits **periods** ("Monday 9 AM – 1 PM and 2 PM – 6 PM"), and converts them to the hour rows the tables have always stored. Every existing schedule — including a sparse exception written one cell at a time by the old grid — must read back as exactly the same hours. A weekly save is a **compare-and-swap under a real row lock**, versioned by `therapist_schedule_state`.

`THR-AVAIL-001` — Set a weekly schedule with two periods a day · P0

Steps

1. Open `/therapist/dashboard/availability`.
2. Read the header line — it states the schedule's timezone.
3. On **Monday**, tap the "working" toggle so the day is on.
4. Tap the Monday start-time control. Set `09:00`. Tap the end-time control. Set `13:00`.
5. Tap **Add hours** on Monday. On the new period, set `14:00` to `18:00`.
6. Tap the copy control on Monday and copy Monday to Tuesday, Wednesday, Thursday and Friday.
7. Tap **Save**.

Expected Result. After saving, a "Saved" indication appears and the week summary reads Mon–Fri `9:00 AM – 1:00 PM, 2:00 PM – 6:00 PM`. Reloading the page shows exactly the same periods — **the stored hour rows round-trip to the same periods**. Saturday and Sunday remain off. **Cross-check:** Admin → Sessions → Roster, opening `QA Therapist A`, shows the identical schedule in the identical editor.

`THR-AVAIL-002` — A day with no hours must be explicit · P1

Steps. Turn Monday's working toggle on, remove all its periods, and tap **Save**. **Expected Result.** Refused with `Add at least one set of hours, or mark the day unavailable.` The save does not go through with an ambiguous day.

`THR-AVAIL-003` **— Removing hours a session is booked into · P0**

Preconditions. Therapist A has a confirmed session on a Wednesday at 10:00. **Steps.** Edit Wednesday's period to `14:00–18:00` (dropping 10:00) and tap **Save**. **Expected Result.** Before saving, the editor **names the affected sessions** and states that **the session stays as booked**. After saving, the schedule changes and **the appointment is unchanged** — not cancelled, not moved, not flagged. The patient's dashboard and the admin's Schedule still show it at 10:00. **Availability and appointments are separate systems and the booking wins.**

`THR-AVAIL-004` **— A stale save is refused; a double-clicked Save is a no-op success · P0**

Purpose. Prove the compare-and-swap. **Steps**

1. Open the availability screen in two browser tabs (Tab 1 and Tab 2), both showing the same schedule.
2. In Tab 1, change Monday to `10:00–14:00` and tap **Save**. Wait for "Saved".
3. In Tab 2 (still holding the old version), change Monday to `08:00–12:00` and tap **Save**.
4. Separately, in a fresh tab, change Tuesday and **double-click Save**.

Expected Result

- Step 3: refused with **HTTP 409** and a message telling the therapist to reload the latest. Tab 1's save is not overwritten.
- Step 4: **both identical requests succeed** — two identical requests carrying the same stale version are one logical change, so this is a **no-op success**, not a 409. Exactly one change is stored.

`THR-AVAIL-005` **— A therapist reads their exceptions but cannot write them · P0**

Steps. On the availability screen, look for a control that creates a date exception. Then call `POST /api/admin/set-availability-exception` with the therapist's cookie. **Expected Result.** The therapist's own screen **shows** any exception on their record but offers no control to create one. The admin route returns **403 Forbidden**. Writing a date exception is an admin capability and stays one.

`THR-AVAIL-006` **— Leave leaves the schedule intact · P0**

Steps

1. In the Leave panel, set leave from `2026-09-14` to `2026-09-18` with reason `Annual leave`. Save.
2. Look at the weekly schedule.
3. Remove the leave.
4. Look at the weekly schedule again.

Expected Result. The weekly schedule is **unchanged** in both step 2 and step 4 — leave never clears it, and there is nothing to restore on the way back because nothing was removed. `profiles.on_leave` reflects the state. The admin's Roster shows the therapist as on leave for those dates.

`THR-AVAIL-007` **— An exception owns only its own date · P1**

Preconditions. An admin has set a `2026-09-15` exception of `14:00–18:00` for Therapist A (`ADM-ROST-003`). **Expected Result.** 15 September shows `14:00–18:00`; every **other** Tuesday still shows the weekly template's hours. The weekly template itself is unchanged.

`THR-AVAIL-008` **— Roster changes never move a booking or the patient's picker · P0**

This is the regression that guards the whole design. See `XCFG-ROSTER-001`.

12.3 Sessions

`THR-SESS-001` — Assigned sessions appear; unassigned ones do not · P0

Steps. Sign in as Therapist A and open `/therapist/dashboard/sessions` **before** the admin assigns anything. Then have `ADM-SESS-003` assign Patient A's paid session to Therapist A, and reload. **Expected Result.** Before assignment: the session is **absent**. A `requested`, unassigned session belongs to no therapist. After assignment: it appears under Upcoming with the patient's name, the concern, the slot and its status. If Google credentials are configured, it also carries a Meet link.

`THR-SESS-002` — Sessions is one list with filters · P2

Expected Result. Video sessions and home visits are in **one** Sessions screen with Upcoming/Past/Cancelled and a Video/Home-visit filter that appears only for a therapist who has both. There is no separate "Home visits" sidebar entry.

`THR-SESS-003` — A patient's phone is masked and their email is absent · P0

Preconditions. `contact_masking_enabled` is on (the default). **Steps.** Open an assigned session card and read the contact area. Then open the browser's **View Source / Network** and search the page HTML for the patient's full phone number and for `qa.patient.a@example.test`. **Expected Result**

- On screen: the phone reads in the masked form `+91 ••••• ••210` — country prefix and last three digits only. **The email is not shown at all.**
- **In the page source, the plaintext phone number must not appear anywhere**, because masking happens where the rows are loaded, not in the component. The email must not appear either — it is not loaded onto therapist surfaces.
- A patient with no number shows `No number on file`.

`THR-SESS-004` — Reveal a contact inside the join window · P0

Preconditions. A confirmed **video** session whose slot is within the join window in **real** time (the route uses the server clock). Reason: this route's window check is server-side. **Steps.** Tap **Show number**. Read the number. Then repeat on a session whose slot is a week away, and on a cancelled session. **Expected Result**

- Inside the window: the full number is revealed, and a `contact_reveal_log` row is written. **The log write is not best-effort** — if it fails, the reveal is refused with `Could not show the number just now. Please try again.` A reveal with no trace is the one outcome this route must not produce.
- Outside the window: refused with the route's own explanation (403).
- Cancelled session: refused.
- A **home visit** is revealable **any time on the visit's own day**, not merely in a join window — verify this separately.
- Admin → Settings → User Access shows the reveal log. It is **admin-read-only and append-only by trigger**: attempting to update or delete a row raises, even with the service role.

`THR-SESS-005` — Completing a session is gated two ways · P0

Purpose. `status='completed' && payment_status='paid'` is the exact condition that makes a therapist's revenue share payable, so the route refuses two things.

Steps

1. On an **unpaid, unprogrammed, no-cash** confirmed session, tap the complete control.
2. On a **paid** confirmed session whose slot is still in the future, tap the complete control.

3. Simulate a time after the slot and try step 2 again.
4. On a paid confirmed session whose join window has genuinely opened in **real** time, tap the complete control, read the dialog, and confirm.

Expected Result

- Step 1: refused (409). Nothing may be completed with no payment, no programme behind it and no cash recorded. A cash home visit must collect first.
- Step 2: refused (409) — nothing may be completed before the join window in which it could have been started.
- Step 3: **still refused.** The simulated clock is client-side only; this gate is server-side. **This is correct behaviour, not a defect.**
- Step 4: the dialog reads *"Mark this session as done? You'll be asked to rate it next."* After confirming, the session becomes `completed`, `completed_at` is stamped, and the therapist's Earnings figure increases by their share.
- A session that is not `confirmed` is refused with `Only confirmed sessions can be marked completed.`
- A second completion attempt returns `This session was already updated — please refresh and try again.` (409). **No double payout.**

`THR-SESS-006` — Mark a no-show · P2

Expected Result. The session is completed with `no_show=true`. It shows as `Marked as no-show` and the patient's rating control is suppressed with `This session was marked as a no-show — there's nothing to rate.`

`THR-SESS-007` — Record cash on a home visit · P0

Purpose. The person holding the cash must not also decide how much the clinic knows about. **Steps.** On a cash-on-visit home visit, tap the collect-cash control. Observe what the form asks for. Then inspect the network request body. **Expected Result.** The therapist **asserts that money changed hands**; they do **not** type an amount. The request body carries **an appointment id and nothing else** — the total is reconstructed server-side from the purchase using the same per-visit maths that booked it. If the UI ever offers an editable amount field here, that is a P0 defect. The honest exception (a patient short of cash) belongs to **Admin → correct cash amount**, which requires `money` scope, a mandatory reason, a CAS on the figure being replaced, and writes a `cash.correct_amount` audit row — and refuses a visit whose cash has already been remitted. A second attempt returns `This visit's payment has already been recorded.`

`THR-SESS-008` — Session notes · P1

Steps

1. Before a session has taken place, open the note dialog.
2. After completion, open it and fill: **What did you treat today?** `L4-L5 segmental mobilisation, glute med activation`; **Techniques and dosage** `Grade III PA mobilisation ×3 sets, 30s hold`; **How did the patient respond?** `Reported easing within the session, straight-leg raise improved`; **Home exercise prescribed** `Cat-camel ×10, twice daily. Walking 15 min.`; **Plan for the next session** `Progress to loaded hinge if pain stays under 3/10; reassess SLR`; **Anything to watch** `Reports night pain — reassess if it persists`. Save.
3. Edit the note within 24 hours and change one field.
4. Sign in as Patient A and search their entire portal and their exported PDF for the note text.

Expected Result

- Step 1: refused — `You can write the note once the session has taken place.`

- Step 2: saved. It appears on the therapist's chart. **Completion is never blocked on a note** — the nudge is a "Notes to write" figure on Overview and a feed item.
- Step 3: succeeds, and a `session_note_revisions` row records what it replaced. After 24 hours, editing is refused.
- Step 4: **the note text appears nowhere the patient can see, including the PDF export.** `session_notes` has no patient select policy and must never get one.
- Submitting with required fields blank: `Fill in what you treated, how the patient responded, and the plan for next time.`
- An unknown field in the payload: `Note contains unknown fields.`

12.4 Clinical records

`THR-PAT-001` — My Patients, with a view switch not a second entry · P2

Steps. Open `/therapist/dashboard/health-profile`. Toggle **Patients** ↔ **Programmes**. **Expected Result.** The same patients, arranged two ways — by name, and by package purchase. **Programmes is a toggle, not a sidebar entry**, and it only appears for a therapist who actually has package patients.

`THR-HP-001` — Triage a new patient · P0

Purpose. The therapist owns the **first fill**, and it is **not reviewed**. **Preconditions.** Therapist A is assigned to Patient A, who has no condition profile.

Steps

1. Open Patient A's chart at `/therapist/dashboard/health-profile/<patientId>`.
2. Tap the control that starts triage.
3. Answer **How old is the patient?** → `18 to 64`.
4. Answer **What brought them in?** → `Injury, strain or overuse`.
5. Answer **Any of these present?** → tick `None of these`.
6. Observe whether the milestones question appears.
7. Read the suggestion panel.
8. Confirm the suggested condition type.

Expected Result

- Step 2: the dialog shows **all four questions at once, with headings** — the clinician's surfaces deliberately invert the patient's one-question-at-a-time pacing, because a clinician filling this after every assignment wants to scan it.
- Step 6: the milestones question is **not shown** — it only appears when age is `Under 18`.
- Step 7: **Orthopaedic** is suggested, **with its reason stated**, and is **not auto-accepted**. The therapist confirms or overrides.
- On confirming, an already-`approved` `condition_change_requests` row is written, so the triage appears in the ordinary Review History with no new concept and no queue.
- **Nowhere on a patient-facing screen do the words "triage", "onboarding" or "specialty" appear.** A patient is shown the care ("Orthopaedic care"), never the category word.

`THR-HP-002` — Write the patient's first record (live, no review) · P0

Steps. Continue from triage into the seven orthopaedic questions and enter the §8.4 answers. Submit. **Expected Result.** The record is written **live** — there is no approval queue in front of it. The patient's own

Health Profile **unlocks** immediately (see `PAT-HP-002`). The route needs only that the therapist is **assigned** to the patient. If the therapist is not assigned, it is refused with `You aren't assigned to this patient.`

`THR-HP-003` — Editing a live record needs an approved grant · P0

Purpose. The line is **create versus edit**. **Steps.** With the record now live, attempt to edit the patient's answers on their behalf. Then request access, have an admin approve it (`ADM-PEOP-004`), and try again. **Expected Result.** Before the grant: refused with `You don't have an approved access grant for this patient's health profile.` The **Request access to edit** card sits **inside the Pain Map card** — beside the thing it gates, not three sections above it — and states what is readable regardless and what needs approval. After the grant: the edit submits and goes to admin review. A second request while one is open: `You already have a pending or approved request for this patient.`

`THR-HP-004` — Re-triage merges, it never replaces · P0 [SQL or careful UI check]

Purpose. This is the single most damaging possible regression in the clinical layer. **Steps.** With Patient A holding a full orthopaedic record, re-triage them as **Neurological** and complete the neuro question set with the §8.5 answers. Then inspect the profile. **Expected Result.** The neuro answers are stored **alongside** the orthopaedic ones in the same flat blob. **Every orthopaedic key survives** (hidden on screen, never deleted). The chart now renders the neuro summary card, the neuro snapshot strip and the neuro progress line. The Pain Map is **not** rendered. **If any orthopaedic answer is gone, stop and raise a P0** — the approve path must merge, never write the proposed data outright.

`THR-HP-005` — Pain Map is orthopaedic and stays so · P0

Steps

1. On an **orthopaedic** patient, open the body-map surface. Tap **Lower back** on the figure. Record an exam through the dialog.
2. On the **neurological** patient (Patient C), look for the body map. Then call `POST /api/therapist/pain-assessments/submit` for that patient directly.

Expected Result

- Step 1: the region is chosen by **tapping the figure** (or a chip inside the dialog), never a `<select>`, and the chosen region stays in the dialog header while the clinician types. Questions are grouped, not listed flat. The exam posts **live with no review**, and is **append-only** — a re-assessment is a new row, so the UI can show a trend against the previous visit. Recording one requires only that the therapist is **assigned**.
- Step 2: the non-ortho page **does not merely hide the map — it never queries `pain_assessments` at all**, and the submit route answers **400**.
- Every user-facing exam figure is printed **out of ten**, never as a raw percentage. `Last exam found 34%` beside `How you rate it 6/10` is a defect: both must read on the same 0–10 scale.

`THR-HP-006` — Read access needs no request · P2

Expected Result. An **assigned** therapist can read the patient's intake and Pain Map with no grant. Only *editing the patient's own account of their history* needs one.

12.5 Recommendations (care plans) and suggested sessions

`THR-CARE-001` — Write a care plan from the session note dialog · P0

Preconditions. A **completed** session that **this therapist ran** for Patient A. Package P1 exists and is recommendable.

Steps

1. Open the completed session's note dialog.
2. In the **Recommend treatment** panel, tap the **Condition** dropdown.
3. Read the list of conditions offered, and how they are grouped.
4. Select the session's own condition, then tap the session-count chip for `6 sessions`.
5. Read the four read-only figures shown beneath.
6. Tap **How often, per week** and select `2 a week`.
7. Tick **Needs hands-on treatment**.
8. Tap **Why this, for this patient**. Enter `Your range has improved but the pain returns after a day at your desk. A structured block will hold the gains.`
9. Tap **Anything they should do or know**. Enter `Keep up the walking between sessions. Book the first one within a fortnight if you can.`
10. Read the line above the submit button.
11. Submit.

Expected Result

- Step 3: **conditions are grouped by condition type** — Orthopaedic, Neurological, Paediatric — from `treatment_categories.specialty`. A category an admin has not tagged appears under **General** and still works — that is where the clinic's general consultation sits, and "General" is deliberately **not** a fourth condition type: a patient's own health profile is only ever ortho, neuro or paediatric. The panel **never shows a programme by name**: a clinician answers "which condition" and "how many sessions", and those two pick the catalogue row.
- Step 3: **only programmes for this session's own condition are offered.** `QA Neuro Rehab 8 Sessions` must not be reachable for a `QA Back & Spine Care` session.
- Step 4: a **Delivered as** toggle (Video sessions / Home visits) appears **only** where the clinic sells both against that condition. A toggle with one option is not a decision the clinician has.
- Step 5: **Sessions \`6\`**, **Price \`₹9,999\`**, **Valid for \`90 days\`**, **Each session \`60 min\`** — all read-only, all from the admin's catalog row. **There is no price field, no session-count field and no discount field anywhere in this panel.** If one exists, that is a P0 defect: "the therapist set their own price" must be a thing the schema cannot express.
- Step 6: the frequency dropdown is capped by `care_plan_max_frequency_per_week` (default 5) and offers `Leave open`.
- Step 10: with `care_plan_requires_approval` **on** (the default), the panel says *"Goes to the clinic first. Your patient sees it once it is approved."* With it off, it says the patient sees it on their dashboard. **The copy must match the setting** — telling a clinician their patient can already see something sitting in a queue is a P0 defect.
- Step 11: the plan lands `status = 'pending_review'`, append-only and attributed. `care_plan_versions.source_appointment_id` is NOT NULL and is **re-derived from the appointment, not trusted from the body**. `care_plan_versions.expires_at` is **null** — the offer window is stamped at approval, not now.

- **The patient sees nothing.** Not a greyed-out card: the recommendation is absent from Suggested Sessions and from their Health Profile until an admin approves it (`ADM-CARE-004`).

`THR-CARE-006` — The Overview names who is waiting to hear · P1

Feature. Every programme a patient can buy comes from a recommendation written after a completed session, so that one step is the whole distance between a delivered consultation and a course of treatment. It used to be carried only by an aggregate count, which reads as a score rather than as a list of people.

Steps. Complete a session for Patient A and write no recommendation. Open `/therapist/dashboard`. **Expected Result.** The activity feed carries a **named** item — *"QA Patient A is waiting to hear what next"* — pinned by `needsYou` and linking to that patient's chart. It disappears once a recommendation is written, or once the patient's plan is accepted. A patient who **already has** a live or purchased recommendation must **not** appear; a patient whose plan was declined or withdrawn **should**, because that thread is open again. At most four are shown, most recently seen first, alongside the note nudge.

`THR-CARE-008` — Writing a second one while the first is still queued · P1

Feature. Submitting again is allowed — it lands as a new version on the same thread, which is right when a clinician has genuinely changed their mind. Doing it *without being told* is how the same plan gets submitted twice by someone who assumed the first had failed.

Steps. With a recommendation already waiting for the clinic, open another completed session's note dialog for the same patient. **Expected Result.** The panel says *"You have already recommended a programme for this patient and the clinic has not decided yet — nothing has gone wrong, and your patient has not been asked for anything. Writing another replaces it."* The button reads **Replace it**, not **Add a recommendation**.

`THR-CARE-002` — A recommendation needs a completed session this therapist ran · P0

Steps. Attempt to submit a care plan (a) for a patient this therapist is not assigned to, (b) against a session run by Therapist B, (c) against a session that is not completed. **Expected Result.** (a) `That isn't your patient.` (403). (b) and (c) refused by the route's own re-derivation. This is what makes "recommend to everyone and see who bites" **impossible rather than discouraged**.

`THR-CARE-003` — A purchased plan is never re-versioned · P0

Preconditions. Patient A has **bought** the recommendation (`PAT-CARE-002`). **Steps.** Attempt to write a new recommendation for the same patient. **Expected Result.** The purchased thread is **closed**. A new recommendation opens a **new plan** with `supersedes_id` set — it does not add a version to the purchased one. Editing a purchased plan would change the description of something already paid for. `care_plans_one_open_per_patient` means the patient never sees two competing live recommendations — and covers a **queued** plan too, so a submission waiting on the clinic blocks a second one exactly as a published one does.

`THR-CARE-004` — Versions are append-only by trigger · P1 [SQL]

Steps. In the Supabase SQL editor, attempt `update care_plan_versions set clinical_rationale='changed' where id='<id>';` and `delete from care_plan_versions where id='<id>';` **Expected Result.** Both **raise**. Only `is_current` and a **first** `expires_at` may change — the offer window is stamped once, at approval, and moving one already set raises too. This is enforced by trigger, not by RLS — every route writes with the service role, which bypasses RLS entirely.

`THR-CARE-005` — Withdraw one's own recommendation · P2

Steps. Tap the withdraw control on an unpurchased recommendation, then on one still **waiting for the clinic's approval**. **Expected Result.** Both close; the patient's offer disappears where there was one. **A purchased plan cannot be withdrawn at all.**

`THR-CARE-007` — The clinic's decision reaches the therapist · P0

Feature. A recommendation turned down silently is one that never happened, and the therapist is the only person who can put it right — they rewrite.

Preconditions. `ADM-CARE-005` has turned down this therapist's recommendation with the reason `This patient still has four unused sessions on their current plan.`

Steps. Open `/therapist/dashboard`. Then open that patient's chart. **Expected Result.**

- The activity feed carries a **needsYou** item: *"The clinic turned down your recommendation for QA Patient A"*, with the admin's reason as its detail. It is the **only** care-plan feed item marked `needsYou` — an approval is the expected outcome and marking it so would train the therapist to ignore the badge.
- An **approval** and an **approve-with-changes** also appear, not marked `needsYou`: a submission that vanishes into a queue and never reports back teaches a clinician to stop trusting the queue.
- On the chart, the thread reads **Not approved with the clinic's reason printed beneath it**, in red. The reason is the actionable half — "Not approved" says the recommendation is gone, and only the reason says what to write instead — and the feed item scrolls away while the chart is where a clinician goes to rewrite. A queued thread reads **Waiting for the clinic to approve**. Both are visible to the clinician and **neither is visible to the patient**.
- The thread being closed frees the one-open-plan slot, so a fresh recommendation can be written straight away.

`THR-SUGG-001` — Suggest a session · P0

Preconditions. `therapist_suggestions_enabled` is **on** (it is on by default now; confirm in Settings → Programmes & Home Visits). A programme locked to this therapist with credits remaining.

Steps

1. On the programme's card, tap the suggest control.
2. Pick a date and time comfortably beyond the booking lead time.
3. Enter the note `Let's keep the two-a-week rhythm while it is working.`
4. Tap **Send suggestion**.
5. Tap **Send suggestion** twice more, rapidly, on a second programme.

Expected Result

- Step 4: the control changes to **Waiting on the patient** with a **Withdraw suggestion** option. **No slot is held** — a hold would need releasing, releasing would need a sweep, and there is no scheduled worker. The therapist's calendar is re-checked at acceptance instead.
- No appointment is created and **no credit is spent** — `sessions_used` is unchanged.
- Step 5: exactly **one** suggestion. The submit is guarded by a **synchronous ref** (a `disabled` attribute lands a render too late), and a partial unique index allows at most **one pending suggestion per purchase**.
- Negative cases: a slot too soon → `That time is too soon to book. Pick a later slot.`; a programme not locked to this therapist → `That isn't your programme.`; an inactive/unpaid programme → `That programme isn't active.`; a slot after expiry → `That time is after the programme expires.`; no credits → `Every session in this programme is already used.`; the therapist already busy → `You already have

`a session at that time.` (409); a note over 500 characters → `Your note must be 500 characters or less.`
- With the toggle off: `Suggesting sessions is switched off.` (403).

`THR-SUGG-002` — Withdraw a suggestion · P2

Expected Result. The pending suggestion is closed. The patient's Suggested Sessions entry disappears (and the sidebar entry disappears if nothing else waits).

12.6 The contact-leak scanner (therapist side)

> **Feature guide.** Every string one role writes and another reads is scanned. **Two tiers, deliberately**: a `block` hit (UPI handle, payment link, payment app) refuses the write; a `flag` hit (phone, email, social handle, bare URL) is delivered and recorded. The two tiers exist because this text is **clinical** — a scanner that treats digits as suspicious fires on every dose and every exercise prescription, and a check that cries wolf is a check nobody reads. Phone matching is the Indian mobile shape specifically (ten digits starting 6–9, optional `0` / `91`), not a loose digit run.

`THR-LEAK-001` — A payment handle is blocked · P0

Steps. In **Why this, for this patient**, enter `Pay me directly on 9876543210@okhdfc, it's cheaper` and submit. **Expected Result.** The write is **refused**. The care plan is not created. A `communication_flags` row records the attempt with `blocked=true` and the offending content.

`THR-LEAK-002` — A payment link is blocked · P0

Steps. In a suggestion note, enter `https://rzp.io/l/abcd1234 pay here`. **Expected Result.** Refused, recorded.

`THR-LEAK-003` — A phone number is delivered and recorded · P1

Steps. In a suggestion note, enter `Call me on 9876543210 before the session`. **Expected Result.** The suggestion **is created** and the patient sees the note. A `communication_flags` row exists with tier `flag` and `blocked=false`. Admin → Settings → User Access shows it.

`THR-LEAK-004` — Clinical text with digits does not fire · P0

Steps. In a session note, enter `Grade III PA mobilisation ×3 sets, 30s hold. 10 reps, 2× daily. Order ref 90210.` **Expected Result. No flag at all.** If this fires, the scanner is broken in the way that matters most — a false positive on ordinary clinical text is what makes the whole control useless.

`THR-LEAK-005` — The patient direction is record-only · P1

Steps. As Patient A, put `Call me on 9876543210` in the `/book` notes field and complete the booking. **Expected Result.** The booking **succeeds**. A flag is recorded. **A 400 at the last step of checkout costs a real booking, and a patient is not who this control exists to catch.**

`THR-LEAK-006` — The scan mode switch · P1

Steps. Admin sets `contact_scan_mode` to `flag_only`, then to `off`. Repeat `THR-LEAK-001` after each. **Expected Result.** `flag_only`: the UPI handle is **delivered** and recorded rather than refused. `off`: nothing is scanned or recorded. The setting is read in its **own** call and **fails open** — if the read fails, writes are allowed. (`contact_masking_enabled` is the opposite: it fails **closed**. Both defaults are deliberate and opposite.)

`THR-LEAK-007` — The evidence tables cannot be edited · P0 [SQL]

Steps. Attempt `update communication_flags set content='x'` and `delete from contact_reveal_log` in the SQL editor. **Expected Result.** Both **raise**. Append-only **by trigger**, not merely by RLS — every route here writes with the service-role client, which bypasses RLS entirely. *An evidence record the evidenced party could edit is not evidence.*

12.7 Earnings and payouts

`THR-EARN-001` — Earnings shows delivered work only · P0

Preconditions. Therapist A has a revenue share of `60`, one **completed paid** session at ₹1,999, and one **paid but not completed** session at ₹1,799. **Steps.** Open `/therapist/dashboard/earnings`. **Expected Result.** The earned figure counts **only the completed** session: `₹1,999 × 60% = ₹1,199.40 → 119940 paise` (rounded). The paid-but-not-completed session contributes **nothing** — a therapist's share is earned by **delivering**, not by being booked. The screen answers both "what am I owed" and "what have I been paid" — Earnings and Payout Receipts are **one screen**, not two entries.

`THR-EARN-002` — Home-visit share and travel · P0

Preconditions. Therapist A has `home_visit_revenue_share_percent = 65`; Therapist B has none set. Each completes one home visit of ₹2,499 with a ₹150 travel fee. **Expected Result.** Therapist A: `2499 × 65% + 150 = ₹1,774.35`. Therapist B falls back to their **online** share (55%): `2499 × 55% + 150 = ₹1,524.45`. **The travel fee is paid through in full and is never revenue** — a therapist must never fund their own transport.

`THR-EARN-003` — Request a payout · P1

Steps. Tap **Request Payout**. **Expected Result.** The button becomes **Request Pending**, then **Under Review** once an admin starts review. A second request returns `You already have a pending payout request.` With no revenue share set: `Ask admin to set your revenue share % before requesting a payout.` The online method is refused with `Online payouts aren't available yet — use cash for now.`

`THR-EARN-004` — Cash held nets off the payout · P0

See `FIN-PAY-003` for the full cross-check. From the therapist's side, the figure shown as payable must be **net of cash they are still holding**.

12.8 Therapist authorization (negative)

`THR-SEC-001` — Therapist A cannot touch Therapist B's schedule · P0

Steps. With Therapist A's cookie, call `POST /api/therapist/save-availability` with a body naming Therapist B's id. Then call `POST /api/admin/save-therapist-availability` for Therapist B. **Expected Result.** Both refused. The therapist route writes **only** the caller's own schedule (the id in the body is ignored or rejected); the admin route returns **403 Forbidden**. Verify in the admin Roster that Therapist B's schedule is byte-identical afterwards.

`THR-SEC-002` — Therapist A cannot read an unassigned patient · P0

Steps. With Therapist A's cookie, open `/therapist/dashboard/health-profile/<Patient B id>` (a patient assigned only to Therapist B). Then call `POST /api/therapist/pain-assessments/submit` and `POST`

`/api/therapist/care-plan/submit` for Patient B. **Expected Result.** The page shows no clinical data. The routes return **403** with `You're not assigned to this patient.` / `That isn't your patient.`

`THR-SEC-003` — A therapist cannot book as a patient · P0

Steps. Signed in as Therapist A, open `/book`. **Expected Result.** Instead of the form, the **wrong-account** panel renders, naming the role and routing them to what is theirs. A direct `POST /api/appointments/create` with the therapist's cookie returns **403** with `This account can't book sessions. Sessions are booked under a patient account.` The same applies to all four purchase routes.

`THR-SEC-004` — A therapist cannot reach the admin dashboard · P0

Steps. Signed in as Therapist A, navigate to `/admin/dashboard`. **Expected Result.** Redirected to **`/get-started`** — **never** to `/admin/login`, which would confirm the back office exists and name its door.

`THR-SEC-005` — A suspended therapist is locked out immediately · P0

Steps. Admin sets Therapist A inactive. With the therapist's still-valid cookie, load the dashboard and then call any therapist API route. **Expected Result.** The dashboard redirects to **`/account-suspended`**. The API returns **403** `Your account is not active.` A live session cookie must not outlive suspension.

---

13. Hospital / partner test plan

13.0 Feature guide — the partner's world

A hospital is a **referral source**, never a clinical actor. It is **provisioned by an admin** (there is no partner self-signup that produces a working account — the public `/hospitals` page collects an *enquiry*, which becomes a B2B lead an admin converts). Once provisioned it can refer patients, watch their referrals move through a status pipeline, and see the commission it has earned.

The lifecycle end to end:

```
/hospitals enquiry  →  admin converts the lead  →  hospital account + referral code
      →  hospital submits a referral  →  admin reviews it
      →  admin assigns a therapist  →  admin sends an invite
      →  the patient registers (via the invite, or by typing the referral code at booking)
      →  the patient books and pays  →  the session is delivered
      →  the partner commission is computed on NET revenue
      →  it appears on the hospital's Earnings and on the admin's Money screens
```

The commission rule, precisely: the hospital's cut is `round(net_revenue_paise × hospital_share_percent / 100)` per appointment, where `net = paid − processed_refund`. It is taken on **net**, so a refund reverses it. It is **not** taken on a session whose therapist share is unknown — that whole appointment is excluded from the split and surfaced as a named count instead. **Never guess a percentage to make the numbers tie.**

Data isolation: a hospital sees its own referrals and its own commission, and nothing else. It never sees another hospital's rows, another patient's clinical data, or any admin screen.

13.1 Provisioning

`HOS-LEAD-001` — The public enquiry form · P1

Steps

1. Open `/hospitals`.
2. Scroll to the enquiry form.
3. Fill it with Hospital A's details from §8.9 and submit.

Expected Result. A confirmation appears. A `b2b_leads` row is created. It shows in Admin → People → Partners and raises that tab's badge. **No account is created and no login works yet** — an enquiry is not a partner. **Negative:** submitting with a blank required field, or an invalid email, is refused with a readable message and creates nothing.

`HOS-AUTH-002` — Admin provisions the hospital · P0

(Executed by the admin; verified here because it produces the partner's credentials.)

Steps

1. As Admin Full, open **People → Partners**.
2. Find the lead `QA Sunrise Hospital`. Tap the onboard control.
3. Enter **Organisation Name** `QA Sunrise Hospital`, **Contact Person** `QA Hospital Admin A`, **Email** `qa.hospital@example.test`, **Revenue Share %** `10`.
4. Submit.
5. **Write down the generated password and the generated referral code.**

Expected Result. An account is created with `role='hospital'`, `active=true`, a generated `referral_code`, and `revenue_share_percent=10`. The one-time password is shown **once, in the response** — and **never written into the `admin_activity_log`'s `details`**, because that log is readable by every admin and a generated password there would be a credential leak. An `admin_activity_log` row records *who onboarded whom and when*, with no password. **Negative:** a share of `-5` or `150` is refused with `Revenue share must be a number between 0 and 100`. A missing email is refused. Re-submitting the same email is refused rather than creating a second account.

`HOS-AUTH-001` — Hospital sign-in · P0

Steps. Open `/hospital/login`, sign in with `qa.hospital@example.test` and the generated password. **Expected Result.** Lands on `/hospital/dashboard`. The sidebar reads **Overview**, **Refer a Patient**, **Your Referrals**, **Earnings**, **Edit Profile**, with **Back to Home** at the foot of the nav directly above Collapse (children: Logo, Organisation Details, Contact Preferences, Account Security). The money word on this sidebar is **Earnings** — matching the therapist. It must not read "Revenue & Payouts" or any third name for the same thing.

`HOS-AUTH-003` — A suspended hospital is locked out · P1

Steps. Admin toggles the hospital inactive. With the hospital's cookie, load the dashboard, then call `POST /api/hospital/withdraw-referral`. **Expected Result.** Dashboard redirects to `/account-suspended`. The API returns **403**.

13.2 Referrals

`HOS-REF-001` — Submit an online referral · P0

Steps

1. Open `/hospital/dashboard/refer`.
2. Tap **Patient Full Name**. Enter `QA Referred Patient C`.
3. Tap **Patient Phone Number**. Pick the country and enter `9876543210`.
4. Under **Session Type**, select `Online`.
5. Tap **Address**. Enter `8, 100 Feet Road, Indiranagar, Bengaluru`.
6. Tap **Preferred Language**. Enter `English`.
7. Tap **Medical Issue**. Enter `Right-sided weakness following a stroke six weeks ago`.
8. Tap **Treatment Needed**. Enter `Gait and balance retraining, twice weekly`.
9. Submit.

Expected Result. A teal confirmation: *"Referral submitted — our team will review and reach out."* The form resets and the Session Type returns to `Online`. The referral appears under **Your Referrals** with status **Pending Review**. It appears in Admin → People → Partners and raises the badge. **The Pincode field is not required for an online referral.** The phone field clears with the rest of the form. **Negative:** submitting with the phone blank or malformed is refused with `Enter the patient's phone number so our team can reach them.` The number is required because the clinic **rings this patient before sending the registration link** — see `ADM-REF-001`.

`HOS-REF-002` — A home-visit referral requires a pincode · P1

Preconditions. The admin master switch **Home Visit enabled** is on (otherwise the Session Type option is absent, which is itself the expected behaviour when it is off — verify that first). **Steps.** Repeat `HOS-REF-001`, selecting `Home visit`, and submit with the **Pincode** field blank. Then submit `56003`. Then submit `560038`. **Expected Result.** Blank and `56003` are both refused with `Enter the patient's 6-digit pincode for a home visit referral.` `560038` succeeds. With the master switch off, the **Home visit** option is not offered at all — a partner must not be offered a delivery mode the platform has not turned on.

`HOS-REF-003` — Duplicate referral prevention · P1

Steps. Submit the identical referral (same patient name, same medical issue) a second time. Also double-tap the submit button on a fresh referral. **Expected Result.** The double-tap produces exactly **one** row. For a genuine repeat submission, the second row appears in the list — **and the admin's queue makes the duplication visible** so it can be declined rather than silently creating two patient journeys. *(See §19 — the exact duplicate-detection behaviour on a genuinely re-typed referral is one of the items flagged for confirmation.)*

`HOS-REF-004` — The referral status pipeline · P0

Steps. Watch one referral through every state, driving each transition from the admin side.

Stage	Driven by	Status shown to the hospital
Just submitted	Hospital	**Pending Review**
Therapist assigned	Admin → assign referral	**Therapist Assigned**
Invite sent	Admin	**Invite Sent**
Patient registered	Patient uses the invite / referral code	**Registered**

Stage	Driven by	Status shown to the hospital
Refused	Admin → decline referral (reason required)	**Declined**

Expected Result. Each transition is reflected on the hospital's **Your Referrals** screen. The hospital sees **status only** — never the patient's clinical record. Declining without a reason is refused with `A reason is required to decline.`

`HOS-REF-005` — Withdraw a referral · P2

Steps. On a **Pending Review** referral, tap the withdraw control. Then try to withdraw one whose status is **Invite Sent**. **Expected Result.** The pending one is withdrawn. The invited one is refused with `An invite has already been sent for this referral, so it can't be withdrawn`. The same rule guards the admin's decline path with its own wording.

`HOS-REF-006` — Referral attribution reaches the patient · P0

Steps

1. As Patient C, open `/book`.
2. Complete Step 1, then on Step 2 enter the Patient C details and, in **Referral Code**, enter Hospital A's code. Tab out of the field.
3. Read the validation line.
4. Complete the booking and pay with `success@razorpay`.

Expected Result. Step 3 shows `Checking code...` then, in teal, `Valid — referred by QA Sunrise Hospital`. After the booking, Patient C is linked to Hospital A. The hospital's **Your Referrals** shows the referral as **Registered**. **An unknown code (`ZZZZZZ`) blocks Continue** with `That referral code isn't recognized…`; a **blank** code does not block anything.

`HOS-REF-007` — Registration through the invite link · P1

Steps. Use the invite link the admin copied (Admin → Partners → **Copy invite link**) in a private window, and register. **Expected Result.** The register card is pre-associated with that referral. The account is created with a session immediately (no email step). The referral becomes **Registered**. Attribution is set without the patient having to type a code.

13.3 Partner money

`HOS-MONEY-001` — Earnings reflects delivered, paid, non-refunded work · P0

Preconditions. Patient C (referred by Hospital A, share `10%`) has **one completed paid** online session at ₹2,499 and **one paid, cancelled-and-refunded** session at ₹1,999. Therapist A's share is configured.

Steps. Open `/hospital/dashboard/revenue`.

Expected Result

- The completed session contributes `2499 × 10% = ₹249.90 → 24990 paise`.
- The refunded session's net is `₹1,999 − ₹1,999 = ₹0`, so it contributes **₹0** — a refund reverses the partner's commission, because the commission is a cut of money **kept**.
- The figures on this screen match Admin → Money → Breakdown's `hospitalCutPaise` for the same range exactly.
- Balances are **not date-filtered**; flows are. The screen labels which is which.

`HOS-MONEY-002` **— A patient whose therapist share is unset is excluded, not guessed · P0**

Preconditions. Therapist B has **no** revenue share configured; Patient C had a completed paid session with Therapist B. **Expected Result.** That appointment is **excluded from the split entirely** — it contributes to Gross, Refunds and Net on the admin screens, but to **none** of therapist cut, hospital cut or clinic share. Admin → Money shows it in the named excluded count and excluded revenue. **No commission is estimated for the hospital.**

`HOS-MONEY-003` **— A hospital with a share of 0 is not the same as one with none · P1**

Purpose. These are two different states and must not collapse. **Expected Result.** A patient **not** hospital-referred → hospital cut of 0, and the appointment stays in the split. A patient referred by a hospital whose share is **not configured** → the appointment is **excluded** from the split. If a 0% hospital cut and an unconfigured hospital produce identical figures, the split maths has lost the distinction and that is a P0 defect.

13.4 Hospital isolation and authorization

`HOS-SEC-001` **— Hospital A cannot see Hospital B's referrals · P0**

Steps. Sign in as Hospital A. Open **Your Referrals**. Then, with Hospital A's cookie, call `POST /api/hospital/withdraw-referral` with a referral id belonging to Hospital B. **Expected Result.** The list contains only Hospital A's rows. The API returns **403/404** and Hospital B's referral is unchanged.

`HOS-SEC-002` **— A hospital cannot open a patient's clinical record · P0**

Steps. As Hospital A, navigate directly to `/therapist/dashboard/health-profile/<Patient C id>` and to `/patient/dashboard/health-profile`. Then call `POST /api/medical-documents/view` with a document id belonging to Patient C. **Expected Result.** Both pages redirect to `/get-started` (a signed-in user of the wrong role is never shown another role's dashboard). The document route is refused — the metadata row does not come back under the hospital's own RLS-scoped client, and **the row coming back is the authorization**.

`HOS-SEC-003` **— A hospital cannot reach the admin dashboard · P0**

Expected Result. `/admin/dashboard` redirects to **`/get-started`**, never to `/admin/login`.

`HOS-SEC-004` **— A hospital cannot book as a patient · P0**

Steps. As Hospital A, open `/book`. **Expected Result.** The wrong-account panel renders and tells the hospital to **refer** instead. A direct `POST /api/appointments/create` returns **403**.

`HOS-PROF-001` **— Edit Profile · P2**

Steps. Open `/hospital/dashboard/profile` and walk **Logo**, **Organisation Details**, **Contact Preferences**, **Account Security**. **Expected Result.** The page is named **Edit Profile** — not "Account Security", which named one section of the page rather than the page. Changing the organisation name updates what the admin's Partners screen shows. A password change signs the partner out of other sessions or requires re-authentication, per the security section's behaviour.

`HOS-DASH-001` **— Overview · P2**

Expected Result. The same shape as every other dashboard: a strip of four figures, then the activity feed, then quick actions — in that order. Items still waiting on the partner are pinned to the top of the feed and counted.

14. Admin test plan — Part A (Today, Sessions, People, Catalog)

14.0 How the admin dashboard works

The whole back office is **one page** at `/admin/dashboard` making roughly forty queries. The screen you see is chosen by `?section=&tab=`. Three facts follow, and each has a testable consequence:

1. **Tab state is written with the History API, never a router navigation.** Moving between two already-rendered screens must **not** re-run the page's queries. If switching tabs shows a loading spinner or takes seconds, that is a defect.
2. **A deep link server-renders its screen.** `/admin/dashboard?section=money&tab=payouts` must paint Payouts directly — not paint Today first and jump.
3. **An unknown tab key falls back to the section's first screen.** So a stale link *looks* like it works while quietly landing somewhere else. Build links with the typed helper; never hand-write one.

Realtime. The dashboard subscribes to two channels: operational tables (bookings, payouts, profiles, care records) on a **short** cooldown, and catalog/settings tables on a **much longer** one. It fires on the **leading edge** — the first change lands immediately and only the burst behind it is collapsed. A refresh re-runs every query on every screen, which is why the cooldowns exist.

Every mutating admin route writes an `admin_activity_log` row, after the route's compare-and-swap claim, so the log cannot record a settlement or cancellation that lost its race. The write is **best-effort and never throws** — an audit failure must not block the action it describes. The log has a select policy and **deliberately no insert policy**: the service role is the only writer, so it is append-only from any session. **A generated password never appears in `details`.**

14.1 Today

`ADM-TODAY-001` — The Today screen · P0

Feature. "Everything waiting on you, in one list." Figures, queues and the activity feed on **one** screen, because splitting them meant an admin checked one and missed the other.

Steps. Sign in as Admin Full. The dashboard opens on **Today → Today**. **Expected Result.** A strip of four figures, the inbox queues, and the activity feed — in that order. Each inbox row shows a **count**, a **one-line hint** saying why it matters, and links to the section/tab where that work is actually done. Rows representing **money at stake** render in red as urgent. The **Today** badge equals the sum of the inbox counts. Every row's destination must exist: tapping each row must land on a real screen, never on a section's first tab by accident.

`ADM-TODAY-005` — A count opens the rows it counted · P1

Feature. A figure or a queue row that opened an unfiltered list made the reader redo the filtering by hand, and made the number look wrong. The link carries a `?view=` preset the target screen applies on arrival.

Steps

1. With at least one unassigned session on record, note the **Unassigned sessions** figure on Today.
2. Tap it.
3. Read the **All Sessions** heading count and the status filter.
4. Tap **Sessions today**, then **Cash to remit**, from Today.

5. Return to **Today** in the sidebar, then open **Sessions → All Sessions** from the sidebar.
6. Sign in as **Admin Clinical** (no Money scope) and read the same strip.

Expected Result

- Tapping **Unassigned sessions** opens All Sessions with the status filter on **Needs a therapist**, the header count equal to the figure, and the list starting at **page 1**. Every other filter — mode, payment, therapist, patient, date range — is cleared, including any remembered on this device: a remembered filter that hid rows the figure counted is the same bug in a subtler form.
- **Sessions today** filters the date range to today; **Cash to remit** opens Money → Payouts filtered to therapists with a balance.
- The preset is **one-shot**. Returning through the sidebar drops `view` from the URL, and re-opening All Sessions shows the filters as the admin last left them, not the preset again.
- On All Sessions itself, the **No therapist**, **Today** and **Home visits** figures filter the list **in place** (no page navigation) and tapping the applied one clears it.
- An admin whose scope cannot open the target section sees the figure **without a link** — never a link into a 403.

`ADM-TODAY-006` — A queue's age is real, and survives a refresh · P0

Feature. The feed items that roll a queue up — signups waiting for approval, change requests to review, sessions with no meeting link — used to stamp themselves with the moment the page rendered. This dashboard re-renders on every realtime event, so they reset to **just now** constantly, and a signup that had been waiting three days read as having just arrived.

Steps

1. Create a patient signup and leave it unapproved. Note the real time.
2. Wait a few minutes, then open **Today** (or **Today → Activity** on a scoped desk) and read the time under *"1 signup waiting for approval"*.
3. Press **Refresh**. Read it again. Then have a second admin act so a realtime refresh fires, and read it a third time.
4. Leave a second signup unapproved a day later, so two are waiting, and read the time again.
5. Approve them all and let a **change request** and a **failed Meet sync** be the waiting items instead; check both the same way.
6. Leave the tab open for ten minutes and watch the item.

Expected Result

- Step 2: the age of the **signup**, not of the page — a few minutes, matching step 1.
- Step 3: **unchanged by both refreshes.** If it reads *just now* after a refresh, this defect is back.
- Step 4: the time is the **oldest** of the two, not the newest. A queue's age is the age of what has waited longest — that is the whole reason the figure is worth showing.
- Step 5: both behave identically. The sync queue is dated by the **session's slot time**, since a session without a link is urgent by when it is due.
- Step 6: it ages normally (`5m ago` → `15m ago`), which is the one way this number is allowed to move.

`ADM-TODAY-002` — Inbox counts are live · P1

Steps. In a second browser, have a patient book a session. Watch the admin's Today screen without reloading. **Expected Result.** The unassigned count and the badge update within the operational channel's cooldown. The **first** change appears immediately (leading edge); a burst of ten bookings collapses into one refresh.

`ADM-APPR-001` — Approve a pending patient · P0

Steps

1. Open **Today → Approvals**.
2. Find the row for `QA Patient B`.
3. Tap **Approve**.

Expected Result. The row leaves the queue and the badge decreases by one. `profiles.approved` becomes true. Patient B can now sign in and reach `/patient/dashboard` instead of `/pending-approval`. An `admin_activity_log` row records the approval with the actor, the target and the timestamp. **Approvals live under Today, beside the inbox that counts them — never on the patients directory.** A queue is not a person. **The screen states what it is deciding**, because the two halves are not the same decision: a therapist here is a credentials check, while a patient here registered *without* booking — anyone who genuinely attempts a payment is approved automatically at that moment. Approving a patient from this list changes what they can see, never whether they can pay. Confirm that line is present; without it a new admin cannot tell what they are being asked to judge.

`ADM-APPR-002` — Approve a therapist · P0

Same as above for `QA Therapist A`. **Expected Result.** The therapist can sign in and reach the dashboard, and their availability routes stop returning 403.

`ADM-APPR-003` — Decline an account · P1

Steps. Tap **Decline** on a pending row, leaving the reason blank; then with a reason. **Expected Result.** Blank is refused (`A reason is required to decline.`). With a reason, the row leaves the queue, the account does not become approved, and the decline is audited.

`ADM-APPR-004` — Profile change requests · P1

Preconditions. `PAT-PROF-001` submitted a gated field change. **Steps.** Approve one request; decline another with a reason. **Expected Result.** Approving writes the new value onto the profile; declining does not. Both are audited. A stale decision on an already-decided request is refused with `This request has already been reviewed`.

`ADM-RISK-004` — Each desk reads its own signals; the trails stay closed · P0

Steps. With at least one open signal from a money rule (`cash_variance`) and one from a sessions rule (`contact_leak`), open **Today → Risk** as **Finance**, then as **Operations**, then as **Admin Full**. **Expected Result.** Finance sees the money signals and **not** `contact_leak`; Operations and Clinical see the sessions signals and **not** `cash_variance`; Admin Full sees both. Each scoped screen carries `Only the signals your desk can act on are shown here.` — a filtered queue that looks complete is worse than one that says what it is. A scoped desk can **review** its own signals (the route is `today`-scoped, which every desk manages). **What stays closed: Flagged messages**, the **contact-reveal trail** and the **thresholds** render for **Admin Full only**. Those quote what a colleague wrote and name every patient contact a therapist opened — the reading the whole queue used to be shut for — and a threshold is configuration. If a scoped admin can see any of the three, that is a P0.

`ADM-RISK-001` — The Risk queue · P1

Feature. Suspicious patterns surface here, written by a **bounded lazy sweep after the Today render** — a wall-clock budget checked between rules, and a five-minute minimum interval, because realtime refreshes this page on every booking.

Steps. Open **Today → Risk**. Read the list, then open one signal. **Expected Result**

- **A flag is never an accusation and never carries a penalty.** Nothing is suspended, held or hidden because a rule fired. **The Risk tab carries no action buttons** — acting on a finding means going to the screen that owns that action and doing it deliberately, with its own audit row. If an action button appears here, that is a defect.
- Each signal shows a severity (`Low` / `Worth a look` / `Look now`), a status (`Needs a look` / `Being reviewed` / `Nothing in it` / `Acted on`), and **links to the rows behind it** — `evidence` stores row ids, not a score, because an admin who can only see a verdict cannot disagree with it.
- The eight rules are `contact_leak`, `completion_without_payment`, `early_completion`, `cash_variance`, `contact_reveal_volume`, `manual_adjustment_volume`, `plan_conversion_low`, `post_consultation_dropout`. **The last two ship disabled** — a threshold invented before anyone knows the clinic's normal rate fires on everyone or on nobody.

`ADM-RISK-002` — Reviewing a signal requires a real note · P1

Steps. Review a signal with a note of `ok` (2 characters), then with `Checked the two sessions, both legitimate.` **Expected Result.** The short note is refused — the minimum is **ten characters**, enforced by a CHECK, because "dismissed" with no reason reads the same as "not read". Reviews are **append-only**. Closing a signal frees its slot, so a repeat after a dismissal is raised **fresh** — that is correct, it is new information.

`ADM-RISK-003` — Thresholds are editable, and the editor is Master Admin's · P1

Steps. Edit a rule's threshold on the tab as a Master Admin. Then sign in as Admin Ops and open Today → Risk. **Expected Result.** The threshold saves and the next sweep uses it. As Admin Ops, the **threshold editor is absent** — what fires at all is a clinic-wide decision — and so are the flagged-message and reveal-log trails, which name a colleague and quote what they wrote. The signals that desk can act on are still listed and reviewable (`ADM-RISK-004`).

14.2 Sessions

`ADM-SCHED-001` — Schedule (calendar) · P1

Steps. Open **Sessions → Schedule**. Navigate months. Tap a day with sessions. Tap one session. **Expected Result.** The calendar shows sessions by day. Tapping a day opens its panel; tapping a session opens the **same `SessionDetailDrawer`** that All Sessions opens. **There is one detail surface, not two.**

`ADM-SESS-NEW-001` — "Book for a patient" lands on the booking form · P1

Feature. An admin who opens `/book` is shown the wrong-account card rather than the wizard — one account carries one role, so an admin cannot be the patient a booking is for. Its **Book for a patient** button sends them to New Booking under Sessions. It used to carry `?section=sessions` with **no tab**, which resolves to the section's *first* screen, so the one button in the product named "Book for a patient" landed on the Schedule month grid.

Steps

1. Signed in as a **Master Admin**, open `/` then **Book**, and tap **Book for a patient**.
2. Repeat as **Operations**, then **Clinical**.
3. Repeat as **Finance**.
4. On the screen Finance lands on, press **Dismiss**, then move to another screen and back.

5. As a limited desk, deep-link straight to `?section=today&tab=activity`, then navigate away and press **Back**.

Expected Result

- Steps 1–2: **New Booking**, with the form on screen — not the calendar.
- Step 3: Finance lands on the nearest Sessions screen they *can* open, with **one amber line** saying *"New Booking is not part of your access, so this is the nearest screen you can open."* Finance reads Sessions and cannot change one, so booking is genuinely not theirs — `/api/admin/create-booking` refuses them too. What must not happen is landing on a different screen with nothing said.
- Step 4: the line goes and does not come back — it describes the link they arrived on, not the screen they chose next.
- Step 5: **Today → Activity both times.** The shell's own URL handler resolves with the same scope rules the server used; while it did not, a limited desk's deep link and Back button both fell through to Today's overview.

`ADM-SESS-001` — All Sessions is one filterable list · P0

Feature. All Bookings, Session Story, the calendar's day panel and the home-visit queue were four lists over the same rows. They are now **one** list plus the calendar, both opening the same drawer. **If you find yourself wanting a second list of sessions, the answer is a filter.**

Steps. Open **Sessions → All Sessions**. Apply a status filter, a mode filter and a date range. Reload the page. Then change the page size and reload again. **Expected Result**

- One list containing every session, video and home visit alike. Home-visit specifics (address, travel fee, cash) are a **panel inside the drawer**, not a parallel screen.
- Filters are **remembered per browser** — but **the date range is not**, because it goes stale.
- The list ends in the standard pager: a **Show N per page** field (default **25** on this screen — it is the one an admin lives on, where ten rows made every working day a paging exercise; other lists keep the shared default of 10, remembered per browser under this list's own key), Previous/Next that grey out at the ends, and an "x–y of n" count. There is **no arbitrary row cap with a "Show all" escape hatch** — that was the old behaviour, and "Show all" then painted every row anyway, which is the thing the pager replaced.
- Filtering, sorting, totals and **both exports** always run over the **whole filtered set** — only what is painted is paged. Otherwise a range total would start describing a page.

`ADM-SESS-002` — Assign a therapist · P0

Steps

1. Open a paid, unassigned session's drawer.
2. Read the assign form.
3. Select `QA Therapist A`.
4. Submit.

Expected Result. The drawer's control reads **Assign a therapist**, never "Reschedule / Reassign" — nothing has been reassigned on a session nobody has ever been assigned to. If the patient requested a specialist, that therapist is **preselected and marked "(requested)"**, with the line "Patient requested this therapist" above the picker. An unassigned row on **All Sessions** and on the **Schedule** day panel carries a **Tap to assign** chip, and the reschedule control below the assign form is the one to use when the time has to move too. A **home visit** is assigned from the visit panel in the same drawer, not from a second copy of the online form. On assignment the session becomes `confirmed`, a Google Calendar/Meet event is created (if credentials are configured), the therapist now sees it, and the patient's card shows the therapist's name.

The unassigned badge decreases. **Negative:** assigning a therapist who already has an overlapping session is refused with `This therapist already has another session that overlaps this time slot.` A session already over is refused with `This session is already over and can't be modified`. A stale submit returns `This session's status changed — please refresh and try again.`

`ADM-SESS-003` — Meet sync failure is recorded, retried and capped · P1

Feature. Google sync **must never block a booking**. Failures are recorded on the appointment, re-attempted by a **lazy sweep at the top of the admin dashboard render**, and retried by hand from **Session Links**. Because that sweep makes outbound calls from inside a page render, it is capped **three ways**: a wall-clock timeout per attempt, a few appointments per sweep, and an attempts-per-appointment counter.

Steps. Remove or invalidate the Google credentials. Assign a therapist to a paid session. Then open **Settings → System Health**. Tap **Retry** on the failed row several times. **Expected Result.** The assignment **succeeds** and the session is confirmed — the booking is never blocked. `google_calendar_sync_error` is recorded and the row appears under **Session Links**, raising that tab's badge. Retrying increments the attempt counter; at the cap the row stays flagged as **needing a person** rather than being retried forever. A **manual Retry resets the counter.** The **Google Connection** card above Session Links turns red at the same time and says the account is no longer connected — see `ADM-SESS-003c`, and check it first, because that panel is what tells you whether these are a few unlucky sessions or the whole integration being down. Two overlapping attempts must not both create an event — each claims the appointment first, with a staleness window so a render that dies mid-attempt releases its row. Retrying a session that is not confirmed-with-a-therapist is refused with `Only confirmed sessions with an assigned therapist can retry Meet sync`. A concurrent retry returns `A sync attempt for this session is already running. Try again in a moment.` **Note:** a **home visit still gets a calendar event even when `google_meet_enabled` is off** — that toggle gates the Meet conferencing only, not event creation, because the invite email is the only outbound notification this platform sends.

`ADM-SESS-003d` — A home visit is never listed as a failed sync · P1

Feature. A home visit deliberately gets **no Meet link** — there is nothing to join, the therapist travels to the address — so its calendar event carries the address and access notes instead. "Has no Meet link" therefore cannot mean "sync failed", and `src/lib/meetSyncState.ts` is the one place that decides: a home visit is synced when its **event** exists, an online session when its **Meet link** does.

Steps. Book and confirm a **home visit** with a therapist assigned, and let its calendar event be created. Open **Settings → System Health → Session Links**. Then, on a session that genuinely has no event, press **Retry** twice and count the events on the clinic's Google Calendar. **Expected Result.** The confirmed home visit is **absent** from Session Links — it is not a failure. Pressing Retry on a session that *is* listed and does succeed reports **success**, not `Retry failed`, whichever delivery mode it is. Most importantly, a session that already has a calendar event **never gets a second one**: `createMeetEventForConfirmedAppointment` refuses on `google_event_id`, at every door — the automatic sweep, the manual Retry, and the three booking paths. **Negative:** this is a regression test with a real history. Before it, every confirmed home visit sat in Session Links permanently, Retry answered `502 Retry failed` even on the runs that worked, and each click minted a duplicate calendar event that emailed the patient and the therapist again — three reached one patient. A **cancelled** session still clears `google_event_id`, so a legitimate re-creation after deletion must still go through.

`ADM-SESS-003c` — A dead Google connection says so, and stops burning retries · P1

Feature. Every Calendar and Meet call uses one refresh token. When it dies — revoked, or (much the commonest cause) the Google Cloud OAuth consent screen left on **Testing**, where Google expires refresh tokens after **seven days** — *every* session fails at once. Before this the only sign was a list of per-session errors under Session Links with a Retry button that could never work, which reads as bad luck rather than as an outage. **Settings → System Health → Google Connection** answers it directly, by spending the token: a token can be set and still be dead, so its presence is not the test.

Steps. Set `GOOGLE_CALENDAR_REFRESH_TOKEN` to a revoked or nonsense value and open **Settings → System Health**. Then unset the three `GOOGLE_CALENDAR_*` variables and reload. Then restore working credentials and reload. **Expected Result.** With a dead token the panel is **red**, reads **"The Google account is no longer connected."**, explains that the usual cause is the consent screen being on Testing, and names the fix — set it to **In production** and re-run `scripts/get-google-refresh-token.mjs`. With the variables **unset** the panel is **amber**, not red, and reads **"Google is not set up."** naming the missing variables — an owner who has not wired Google up yet has chosen that, and it is not a fault. With working credentials the panel is **white** and says sessions get an invite and a link automatically; if the saved permission predates the open-access feature it says so and points at the same script. A **network failure is not reported as a dead token** — it reads "Google could not be reached" and says it may be temporary. **Negative:** while the connection is down, the automatic sweep **must not** raise `google_calendar_sync_attempts` or `meet_access_attempts` on any row — retrying cannot succeed, and spending the caps means the sessions that most needed retrying are already retired to "needs attention" by the time the credential is restored. The **manual Retry button is unaffected** and still works, since an admin pressing it is asking deliberately. The panel itself must never break the page: a Google outage costs that one panel and nothing else on the screen.

`ADM-SESS-003b` — Patient and therapist join without being admitted · P1

Feature. Meet holds anyone it does not recognise in a waiting room until the meeting's owner admits them, and a patient signs in with whatever Google account they have — so without this, every session needed the clinic's own account to let both parties in, one at a time. Each new session's meeting is opened at creation. Settings → Booking Rules → **Join Without Approval** is the switch; Settings → System Health → **Waiting Room** lists the sessions where it did not take.

Steps. With the switch **on**, confirm a paid session. Open its Meet link in a browser signed in as a Google account that is *not* on the invite. Then open **Settings → System Health**. **Expected Result.** The link goes **straight into the call** — no "asking to be let in", and nobody has to admit anyone. `meet_access_open` is `true` and the session is **not** in the Waiting Room panel. The Join button's caption reads `Opens straight into the call — sign in to Google if asked.` once the join window is open, and `Opens N minutes before your session.` before it. **Negative:** with a refresh token minted before the `meetings.space.settings` scope (or the Google Meet API not enabled on the Cloud project), the **booking still succeeds and the link still works** — only the waiting room stays on. The session appears under Waiting Room with the 403 explained, is retried a couple of times automatically, then flagged as **needing a person**. **Open the door** re-attempts it and re-arms those attempts; on a session with no Meet link yet it is refused with `This session has no Meet link yet — retry the Calendar sync first`, and on a cancelled one with `This session is cancelled — its Meet space is gone`. **Note:** open access removes the **knock**, not the **sign-in**. A meeting organised by a personal Gmail account still requires every participant to be signed in to *some* Google account; only moving the organising account to Google Workspace allows a patient with no Google account at all to join.

`ADM-SESS-004` — Edit, cancel, reopen and restore a session · P1

Action	Expected
Edit a booking's time/therapist	Saves, re-checks conflicts, audits. A session already over: `This session is already over and can't be modified`
Cancel with refund	The session is cancelled, the refund is processed, Money → Summary's Net Revenue drops while Gross is unchanged

Action	Expected
Reopen a completed session	Only a completed session can be reopened (`Only completed sessions can be reopened.`); audited
Restore a cancelled/no-show session	Allowed only from those two states (`Only a cancelled or completed (no-show) session can be restored.`); a second attempt: `This session has already been restored.`
Mark paid by cash	A session already paid: `This session is already marked as paid`

`ADM-ROST-001` — The Roster opens on therapists, not on a date · P0

Feature. The Roster is the clinic's planning record. It opens on a **list of therapists** rather than a calendar date and an eighteen-column grid, and it uses the **same editor** the therapist's own screen uses.

Steps. Open **Sessions → Roster**. Read the landing view. Open `QA Therapist A`. **Expected Result.** A list of therapists with a summary of what each works, plus their leave state. Opening one shows the **period editor**, not an hourly grid. The periods match exactly what the therapist saved in `THR-AVAIL-001`.

`ADM-ROST-002` — An admin saves a therapist's weekly schedule · P1

Steps. Change Therapist A's Friday to `09:00–12:00` and save. **Expected Result.** Saved through the same compare-and-swap function, with the same stale-save 409 and the same double-click no-op success. The therapist's own screen shows the change. **No appointment is moved.**

`ADM-ROST-003` — Set a date exception · P0

Steps. For Therapist A, set `2026-09-15` to `14:00–18:00` with the reason `Clinic audit in the morning`. Save. Then look at 22 September (the next Tuesday) and at the weekly template. **Expected Result.** Only 15 September changes. **Every other Tuesday still shows the weekly template's hours, and the weekly template itself is untouched.** Setting a date exception replaces that **whole day** in one function — it is not a partial merge.

`ADM-ROST-004` — Set leave · P1

Steps. Put Therapist C on leave `2026-09-14` – `2026-09-18`. **Expected Result.** `profiles.on_leave` is set; the roster shows them off; **the weekly schedule is untouched** and is still there when leave is removed.

`ADM-ROST-005` — Roster authorization on every route · P0

Steps. With a **therapist's** cookie, call `POST /api/admin/save-therapist-availability` and `POST /api/admin/set-availability-exception`. With **Admin Finance's** cookie, call both again. **Expected Result.** All four return **403**. The roster is `sessions` scope; Finance cannot open Sessions at all.

`ADM-DELIV-001` — Delivery answers operational questions, not financial ones · P1

Steps. Open **Sessions → Delivery**. **Expected Result.** No-show rate, cancellation rate, repeat-booking rate and sessions-per-therapist. **These live under Sessions, not Money** — a no-show rate is about how the clinic runs, not about its books. All three metric slices (summary, breakdown, delivery) are computed from **one pass** of the same maths, so a figure here can never contradict the same figure on Money.

`ADM-MONEY-GW-001` — Goodwill: take an amount off one session · P0

Feature. The lane for a session cut short, a therapist who ran late, or a patient in genuine hardship. Without it these are settled outside the system, where nobody can see them and the books never learn they happened.

Preconditions. An **unpaid** session on a ₹1,200 category. Signed in as an admin with **Money** scope.

Steps

1. Open the session from **Sessions → All Sessions** and find **Goodwill** in the drawer.
2. Enter `200` and a two-character reason. Submit.
3. Replace the reason with `Their last session was cut short by a connection problem.` Submit.
4. Re-open the drawer.
5. Separately, try `2400` (more than the session costs) on another unpaid session.
6. Separately, try it on a **paid** session.

Expected Result

- Step 1: the panel states the session price and says plainly that this reduces what the patient is asked for, and refunds nothing, because nothing has been paid.
- Step 2: **refused.** Under ten characters is rejected by the route and by a CHECK on the column — a discount nobody can explain a month later is indistinguishable from a mistake.
- Step 3: applied. The row records **all four facts**: list price `120000`, discount `20000`, source `goodwill`, and the reason. A `payment.goodwill_discount` audit row is written.
- Step 4: the drawer shows the amount, what they now pay, and the reason. It is **not** re-editable — the patient has been quoted it.
- Step 5: **refused**, not applied at ₹1. This is a number typed with the price on screen beside it, so more than the price is a typo (2400 for 240), and quietly charging ₹1 is worse than saying no. *(A configured offer behaves differently and is floored — see `ADM-SET-023`.)*
- Step 6: **409** — *"Refund it instead."* Money that has moved comes back through the refund route, with its own Razorpay call and its own audit.
- A **Clinical** or **Operations** admin gets 403: this changes what somebody is charged, which is a money capability whatever the reason for it.

`ADM-MONEY-GW-002` — What discounting cost · P1

Steps. Apply a goodwill discount and let a first-session offer run, then open **Money → Costs**. **Expected Result.** A **Discounts given** figure, split by rule — the offer, goodwill, promo codes and both halves of an invite, each line appearing only when it has something behind it. It is **stated, not deducted** — the note says so — because a discount means less was collected and is already inside gross revenue as a smaller number; subtracting it from profit would count it twice. If Operating profit drops by the discount amount, that is a P0 defect.

`ADM-PROMO-001` — Create a campaign · P1

Steps. As a **Money** admin, open **Money → Costs → Promo codes**. Switch codes on. Create `WELCOME200`, ₹200 off, total 50 uses, 1 per patient. Then try each of: a code with a hyphen in it; a percentage of 150; an end date before the start date; a second code called `WELCOME200`. **Expected Result.** The first saves and appears as **Running**. Each of the four is refused with a sentence naming the problem, not a raw database error. A `promo.create` audit row is written for the one that saved. A **Clinical** or **Operations** admin gets 403 on the route: this decides what every patient who types the code pays, which is a money capability however much it reads like catalog data.

`ADM-PROMO-002` — A used code is paused, never deleted · P1

Steps. Have a patient claim a code at checkout, then try to delete it. Then pause it and have another patient try it. **Expected Result.** Delete is **not offered** on a code that has been claimed, and the route refuses it with *"switch it off instead"* — a paid session pointing at a campaign nobody can name cannot answer which rule gave the money away. Pausing works, and the next patient is told the code is **no longer available** rather than "not recognised".

`ADM-PROMO-003` — The cap means what it says · P0

Steps. Create a code capped at **1** total use. Open checkout for it as patient A and pay. Then try it as patient B. **Expected Result.** Patient B is refused with *"That code has been fully claimed."* Repeat with patient A abandoning checkout instead of paying: after **30 minutes** the claim frees up and patient B can use it. A cap that can be exceeded by opening two checkouts is a P0 defect.

`PAT-PAY-FREE-001` — The payment screen quotes what it charges · P0

Steps. As a patient with a first-session offer running (say 25% off), reach step 3 of the booking wizard. **Expected Result.** The summary shows the session fee **struck through**, the discount named on its own line, and a **Total**. The button reads **Pay ₹<total> Now** with that same total. If the button quotes the list price while Razorpay opens at the discounted figure — or the reverse — that is a P0 defect: quoting one number and charging another is the one thing a payment screen must never do.

`PAT-PAY-FREE-001b` — A brand-new visitor is quoted the offer too · P0

Steps. With a first-session offer running, open `/book` **signed out**, pick a condition and a slot, and reach step 3 without creating an account yet. **Expected Result.** The summary already shows the offer — fee struck through, "First session offer — ₹X off", and the discounted **Total** — before any account exists. This is the path the offer is *for*: a self-signup patient makes their account, their booking and their payment with one tap on this screen. Showing list price here and then opening Razorpay at the offer price is a P0 defect.

`PAT-PAY-FREE-002` — A 100%-off code books without paying · P0

Steps. Create a promo code at **100%** off. As a patient, apply it at step 3. **Expected Result.** Total reads **Free**, the lock line changes to *"Nothing to pay — your discount covers this session in full"*, and the button reads **Confirm booking — free**. Tapping it books the session with **no Razorpay screen at all**. The session appears in the patient's dashboard as confirmed or pending exactly like a paid one. Being charged ₹1 instead is a P0 defect — that was the old behaviour and it charges a figure nobody was quoted.

`PAT-PAY-FREE-003` — Free is decided by the server, never the browser · P0

Steps. With no discount running, POST to `/api/appointments/confirm-free` with a real unpaid appointment id (browser console or curl, signed in as that patient). **Expected Result. 409**, *"This booking still has an amount to pay."*, and the booking stays **unpaid**. If a booking can be confirmed free by asking, every session in the app is free.

`ADM-MONEY-FREE-001` — A free session still shows in the books · P1

Steps. After `PAT-PAY-FREE-002`, open **Money → Costs** and the session's own drawer. **Expected Result.** The booking records the full price as list price, the whole of it as the discount, and `promo_code` as the source — so **What discounting cost** includes it. Amount paid is **₹0** and there is no payment/transaction row, because no money moved. A free session that recorded nothing would make the giveaway invisible, which is the figure that decides whether the campaign continues.

`ADM-INVITE-001` — Invites: the two halves · P1

Steps. As a **Full** admin, open **Settings → Offers & Discounts → Patient invites**. Switch on, set the friend's welcome to ₹300 and the reward to ₹200, and save. Open a patient's dashboard. **Expected Result.** The panel previews the exact sentence the patient will read, and it says the reward arrives **once their friend has had a session** — not on a signup. The patient's dashboard shows their own code, formatted in two halves, with a copy button.

`ADM-INVITE-002` **— What an invite refuses · P0**

Steps. As the code's owner, try to use your own code. As a patient who has already paid for a session, try to use somebody's code. As a patient who has already used one, try a second. **Expected Result.** All three refused, each with its own sentence: *"That's your own invite code."*, *"An invite code can only be used before your first session."*, *"You've already used an invite code."* The entry field is **not shown at all** to a patient who has already paid or already claimed — a field that can only refuse is worse than no field. A patient reaching the API directly gets the same three answers; none of these rules lives only in the browser.

`ADM-INVITE-003` **— A promise already made is kept · P1**

Steps. Have a patient claim an invite while the welcome is ₹300. Lower it to ₹100, then switch invites off entirely. Open that patient's checkout. **Expected Result.** They still get **₹300** off. Amounts are snapshotted at claim, and the switch stops new claims rather than withdrawing one already made. Charging them the new figure — or nothing — is a P0 defect.

`PAT-PAY-DSC-001` **— The patient sees what they were given · P1**

Steps. As a patient who received a discount, open **Payments** and tap the receipt. **Expected Result.** Three lines: the session price struck through, the discount named and negative, and **You paid**. A receipt that silently printed the lower number would tell the patient nothing about having been given something, which is the entire value of an offer.

`ADM-CARE-001` **— Recommendations: see every care plan · P0**

Steps. Open **Sessions → Recommendations**. **Expected Result.** Three bands, in this order: **Waiting for your decision**, **Waiting on a patient**, **Answered and closed**. Every care plan in the clinic is listed with its patient, therapist, package, status and date. A care plan is now the **only** route by which a patient buys a programme, so the clinic must be able to see them all. The first band **renders even when empty**, saying so. A section that disappears when there is nothing in it gives an admin no way to tell "nothing waiting" from "I am on the wrong screen".

`ADM-CARE-004` **— Approve a recommendation · P0**

Feature. A therapist's recommendation is a bill as well as a clinical note, and the clinic that carries it sees one before the patient is asked to pay it.

Preconditions. `care_plan_requires_approval` is **on** (default, at Settings → Programmes & Home Visits). `THR-CARE-001` has been submitted.

Steps

1. Open **Today → Overview** and read the Clinical group of the action inbox.
2. Follow **Recommendations waiting for approval** to the queue.
3. Read the card: patient, therapist, programme, sessions, price, frequency, the clinician's reasoning and their instructions to the patient.
4. Note the order of the cards and what the badge on each one says.
5. Tap **Approve**.
6. Check the patient's Suggested Sessions screen.
7. Attempt the same decision a second time.

Expected Result

- Step 1: the count is there, and it turns **urgent** only once something has been waiting over four hours — never merely because the queue is non-empty. A badge that is always on is a badge nobody reads, and this is the one queue with a patient on the other side of it who has been told nothing at all. The hint names how many are late.

- Step 2: the link lands on `?section=sessions&tab=recommendations` — the figure and the list it opens must agree.
- Step 4: the queue is **oldest first** — every other list on this screen is a record and reads newest-first, but this is work, and the person waiting longest is served next. Each badge reads **how long** it has waited (`Waiting 3 hours`), not the date it arrived; past four hours it turns red. A card that reads `2 September` when the thing arrived nine minutes ago tells an admin nothing they can act on.
- Step 4: where the patient still has unused sessions **or visits** on a live programme, the card says so in amber. The figure comes through the same ledger helper every other balance surface uses, so it cannot disagree with Catalog → Purchases after the ledger switch is flipped. It is **stated, never acted on** — a patient with sessions left may well need a different programme, and the clinician has seen them. This is the commonest reason to turn one down, and an admin previously had to leave the queue to find it out.
- Step 5: **one tap, no reason asked.** Approving is the outcome this queue exists to reach, and demanding a sentence meaning "fine" twenty times a day is how a reason column fills with `ok` and stops being worth reading; a plain approval's evidence is who and when, both already on the row. The plan becomes `active`; `reviewed_by` and `reviewed_at` are stamped; a `care_plan_reviews` row records the decision and the reviewer with a null reason; a `care_plan.approve` audit row is written. **`care_plan_versions.expires_at` is stamped now** — the patient's answering window starts at approval, not at authoring, so a plan that waited in the queue does not arrive with its time already spent.
- Step 6: the recommendation is now visible and purchasable.
- Step 7: **409** — `Someone else decided this one first.` The decision is a compare-and-swap on `pending_review`, so two admins in the queue together cannot both decide.

`ADM-CARE-008` — A stale offer is caught before the patient meets it · P0

Feature. Checkout re-reads the package and refuses on a mismatch rather than charging a different amount. On its own that means the *patient* discovers the clinic's stale data, by having their payment refused at the last step.

Steps. With a recommendation queued, go to **Catalog → Packages** and change that package's price. Come back and tap **Approve**. Then tap **Turn it down**. **Expected Result.** The approval is **refused** with a sentence naming the drift — *"This was written at ₹9,000 and the programme now costs ₹9,500. Approving it would quote one figure and charge another."* The plan is still queued and `reviewed_by` is still null: refused, not half-applied. Deactivating the package or clearing its **recommendable** flag refuses the same way. **Turning it down still works.** Refusing to let an admin close a thread because its package moved would trap exactly the recommendation that most needs closing.

`ADM-CARE-005` — Turn a recommendation down · P0

Steps. On a queued recommendation, tap **Turn it down**. Submit with the reason `ok` (two characters), then with `This patient still has four unused sessions on their current plan.` **Expected Result.** The two-character reason is **refused**, by the route and by a CHECK on `care_plan_reviews` — a rejection is what the therapist acts on, so it owes them a sentence, and one with none reads the same as one nobody got round to. With a real reason the plan becomes `rejected`; a `care_plan_reviews` row and a `care_plan.reject` audit row are written; the patient never sees it. The therapist sees it as something needing them, with the reason (`THR-CARE-007`) — **they rewrite; an admin does not edit their judgement**. The closed thread frees the one-open-plan slot immediately.

`ADM-CARE-006` — Approve with different numbers · P0

Feature. The middle case, and the one whose honesty is in the plumbing rather than the button.

Steps. On a queued recommendation, tap **Approve with changes**. Change the session count chip and the frequency. Submit with the reason `Frequency reduced to match what this patient can attend.`

Expected Result

- The plan becomes `active` and the patient is offered the **new** numbers.
- **The therapist's original version is untouched** and still readable in the thread, marked superseded. A version is append-only; rewriting one under a clinician's name would be a lie about who decided what, and the trigger refuses it regardless.
- The new version carries `authored_by` = the **therapist** and `entered_by` = the **admin**. Check both in the database — this is the assertion the whole feature turns on.
- The `care_plan_reviews` row records `edited_and_approved`, and a `care_plan.edit_and_approve` audit row is written.
- The programmes offered in the change panel are **narrowed to that session's own condition**, exactly as on the therapist's own dialog.

`ADM-CARE-007` — The switch · P1

Steps. At **Settings → Programmes & Home Visits**, turn **Approve recommendations before the patient sees them** off. Have a therapist submit a recommendation. **Expected Result.** It publishes on save and the patient sees it immediately, exactly as before the review step existed. The therapist's panel copy changes to match. Turn it back on afterwards — the rest of the suite assumes the default. The setting **fails closed**: with the column unreadable, a submission is held rather than published. That is the opposite direction from `contact_scan_mode`, and deliberately so.

`ADM-CARE-002` — Withdraw a recommendation · P0

Steps. Withdraw an **active, unpurchased** plan with the reason `Therapist on extended leave; will re-review.` Then attempt to withdraw an **accepted (purchased)** plan. **Expected Result.** The active one closes; the patient's offer disappears; a `care_plan.withdraw` audit row is written; the route required `sessions` scope, a **mandatory reason**, and a compare-and-swap on `status='active'` (a stale attempt returns `Someone else closed this recommendation. Refresh to see it.`). **The purchased plan cannot be withdrawn at all** — the patient has paid and the sessions exist, so the honest lane is a refund or a credit adjustment, each of which has its own screen. Withdrawal also covers a plan **still waiting for approval** — refusing would leave the queue holding a thread nobody intends to approve while the patient's one-plan slot stayed taken. *_There is no admin path to_ edit _a version. Approving with different numbers (`ADM-CARE-006`) writes a new_* one attributed to the clinician; it does not change theirs, and no route anywhere can set a price.

`ADM-CARE-003` — Write a recommendation on a therapist's behalf · P0

Feature. One authoring implementation, three doors. This one exists for when a therapist cannot reach their dashboard — on leave, off sick, gone — and a patient is still waiting to hear.

Steps

1. On **Sessions → Recommendations**, open the authoring panel.
2. Choose the session — it must be a **completed session that the named therapist ran**.
3. Read which programmes are offered.
4. Fill the same four clinical fields as `THR-CARE-001`.
5. Read the text at the submit button.
6. Enter the reason `Therapist on leave; patient waiting since Tuesday.` and submit.
7. Then change the chosen session to a different one and observe the draft.

Expected Result

- Step 3: programmes are **narrowed to that session's own condition**, exactly as on the therapist's dialog.
- Step 5: **whose name it goes out in is stated at the button**, not in a subtitle two screens up.
- Step 6: the write **publishes directly**, without passing through the review queue — the admin writing it is the approver, so their own queue would decide nothing. It succeeds with **split attribution** — `authored_by` is the clinician whose judgement it is, `entered_by` is the admin who typed it. Naming only the therapist would be a quiet lie about who was at the keyboard; naming only the admin a louder one about whose judgement it is. A `care_plan.author_on_behalf` audit row is written. The route required `sessions` scope and a mandatory reason.
- Step 7: **the draft is dropped** when the session changes, so a package for someone else's condition cannot be carried across.
- With **no** eligible session or **no** recommendable package, the panel **still renders** and says which of the two is missing. An admin opens this screen because a patient is waiting; a panel that is simply absent reads as a feature that does not exist.
- The rules are **not weaker than the therapist's door**: the package still comes from the admin whitelist, the source must still be a completed session that therapist ran, and the text is still scanned.

`ADM-NEWB-001` — New Booking · P1

Steps. Open **Sessions → New Booking**. Create a booking for `QA Patient A` with `QA Therapist A` at a chosen slot. **Expected Result.** The booking is created server-side with the same re-derivation as the patient route. **An admin has a lead-time override** (there is somebody on the phone arranging the exception) where the patient route has none. Missing fields are refused with `Missing appointmentId, therapistId, or slotDateTime` / `Choose a patient.` / `Choose a treatment category.` The booking is audited. **The date and time are the patient's own calendar**, not native date/time boxes: a month grid plus hour cells, opened on the earliest eligible slot. Ticking **Book inside the N-hour window anyway** re-opens the grid down to the current hour — the one thing this screen may do that `/book` may not — and the caption under it changes to say the lead-time rule does not apply. It still never offers a past slot.

`ADM-CAL-001` — One calendar, everywhere a session time is chosen · P1

Steps. Open each of these and compare the date grid and the hour cells: `/book` Step 1; the patient's package **bulk scheduler**; the home-visit bulk scheduler; the therapist's **Suggest next session**; admin **Sessions → New Booking**; admin **Reschedule / Reassign** on a session; admin **People → Partners → Patient Referrals → Pick a time**. **Expected Result.** All seven render the **same** control — same month grid, same weekday headers, same cell colours and states (available / selected / struck-through), same hour cells. Nothing on this list is a native `date`, `time` or `datetime-local` box, and nothing opens in a pop-up on the admin screens. The bulk schedulers additionally dot any day already holding a chosen slot; that dot outranks the today marker. **Where the lead time differs, and why:** `/book`, both bulk schedulers, Suggest and Assign-a-referral use the platform's 12-hour rule (home visits use their own, longer, setting). **Reschedule** uses zero — it moves a session that already exists, which is the admin override lane — and **New Booking** uses zero only while its override box is ticked. Zero never means "the past": no screen offers a slot before now. **The hour rule is enforced server-side too.** POST any of these with a slot carrying minutes (e.g. `…T18:52:00+05:30`) and it is refused **400** with `Sessions start on the hour. Pick a time like 6:00 or 7:00.`: `/api/appointments/create`, `/api/appointments/book-package-sessions`, `/api/admin/create-booking` (**including with the lead-time override ticked** — the override is about the lead time, never about landing between hours), `/api/admin/update-appointment`, `/api/admin/assign-referral`, `/api/therapist/suggest-session`, `/api/home-visit/book-visits`, `/api/home-visit/book-cash`, `/api/home-visit/verify`. **Critical check — the timezone:** the minute is read in the **booking's own** timezone, not the server's. 6 PM IST is 12:30 UTC, so a correct IST booking arrives as `…T12:30:00Z`; if that is refused, the check is reading UTC and every booking in the clinic is broken. Conversely `…T13:00:00Z` (6:30 PM IST) must be refused. **Deliberately not enforced at the consuming end:** `/api/patient/respond-suggestion` and `/api/patient/register-via-referral` book a slot somebody already agreed to, so a row created before this rule still books rather than being stranded. **Deliberately unchanged:** dates that are not session slots keep their native inputs — a report's date, a leave range, a schedule exception, a promo campaign's window, and every from/to range filter (Metrics, Costs, Activity Log, All Sessions, Payment History, Earnings, the Calendar tab's day). They have no hours and no lead time, and a grid whose greyed-out cells mean "too soon to book" would be lying on all of them.

14.3 People

`ADM-PEOP-001` — Patients directory · P1

Steps. Open **People → Patients**. Filter, page, and export. **Expected Result.** A paged list ending in the standard pager. **Approvals are not here** — they are under Today. Below the directory sits the **condition requests** area, whose badge counts what is waiting. **Exports:** every admin export offers **CSV and PDF** from one column definition, so the spreadsheet and the printable document cannot describe different tables. Both cover the **whole filtered set**, not the current page. Every export carries a **subtitle naming what the rows are scoped to** — a printed table nobody can date is worthless. **Nothing in the admin dashboard exports JSON.**

`ADM-PEOP-002` — Global search · P2

Steps. Use the admin search to find `QA Patient A` and `QA Sunrise Hospital`. **Expected Result.** Results are grouped by entity type and link to the right detail surface.

`ADM-PEOP-010` — Opening somebody's dashboard · P0

Feature. A Master Admin can sign in as a patient or therapist from their profile page, to see exactly what they see. It is a **real session swap**, not a preview: the browser becomes that account, every control works, and every write is recorded as theirs. That is the cost of being able to reproduce a bug that only shows on submit, and everything below is what fences it.

Steps

1. As **Admin Full**, open **People → Patients → QA Patient A** and tap **Open their dashboard**. (The same control sits on a therapist's profile, and on a hospital's card on **People → Partners** — a hospital has no detail page of its own, and its card is where its record lives.)
2. Try to confirm with the reason `test`. Then enter `Patient says her session link is missing.` and confirm.
3. Read the bar at the top of the patient dashboard. Walk to Sessions, Programmes and Health Profile.
4. Tap **Exit and go back to admin**.
5. Open **Logs → All Activity**.
6. Repeat step 1 as an **Operations**, **Finance** and **Clinical** admin.
7. Open another **admin's** profile, and a **suspended** patient's.
8. Start a swap, then leave the tab for over 30 minutes and reload.
9. **[SQL]** `update admin_impersonation_sessions set reason = 'x'` on the row your swap wrote, and `delete` it.

Expected Result

- `test` is refused — at least ten characters, the same floor an admin credit adjustment uses. A log full of "test" answers nothing six months later.
- The dialog says in advance that this is the real account, that anything tapped happens for real and is recorded as theirs, that it ends after 30 minutes, and how to get back.
- Every dashboard screen carries an **amber bar** naming the patient, saying the actions are real, counting the window down, and offering Exit. It is the **only** difference from what she sees.
- Exit restores the admin's own session and lands on `/admin/dashboard` — no re-login.
- The log carries **`Signed in as a user`** and **`Stopped signing in as a user`**, both naming the patient, the first carrying the reason.
- All three roles work the same way: a therapist lands on `/therapist/dashboard`, a hospital on `/hospital/dashboard`, each with the same bar. A hospital's own dashboard is where its referrals and earnings read from, so "the referral I sent is not showing" is a question about that screen rather than about the Partners card.
- The three scoped admins have **no such button**, and calling `/api/admin/start-impersonation` directly answers **403** — the button's absence is presentation, the route is the rule.
- Another admin is refused (*"You cannot sign in as another admin"*) whether active or suspended; a suspended patient is refused with what to do about it.
- Past 30 minutes the session is **signed out by the proxy**, not merely un-bannered — a marker left to lapse on its own would drop the bar while the swap ran on underneath it. The admin lands on `/admin/login?expired=impersonation`.
- **[SQL]** Both the update and the delete **raise**: the row is append-only apart from being closed once, so the admin it names cannot rewrite the record of it.

Negative: if the row cannot be written, **no session is opened at all** — an impersonation nobody can trace to a person is the one outcome this route must not produce.

`ADM-PEOP-003` — Patient detail · P1

Steps

1. On **People → Patients**, tap the row for `QA Patient A`.
2. Read the section headings down the panel.
3. Under **Admin Notes**, tap the notes box. Enter `Prefers early evening slots. Referred by QA Partner Hospital.`
4. Tap **Save Notes**.
5. Close the overlay, reopen the same row, and confirm the note survived.
6. Copy the id out of the URL and open `/admin/dashboard/patients/<id>` directly in a new tab.
7. Sign in as the **finance** admin and open the same URL.

Expected Result. Tapping opens an **overlay modal**; the direct URL at step 6 renders the **same content as a full page** (a real route, not only an intercepted one). The panel carries **Personal Details**, **Contact Info**, **Admin Notes**, **Therapist Ratings of This Patient**, **Session Performance**, **Booking History**, **Payment History**, **Profit Breakdown** and **Profile Change Request History**, plus contact edit and password reset. The notes box is placeheld `Private notes about this patient — never shown to them.` and reads `No notes saved yet.` when empty; the button reads `Saving...` then `Save Notes`. **The note must never appear on any patient-facing screen** — check the patient's own dashboard and their exported PDF. `ProfileSessionList` and the purchase modals take `canSeeMoney` / `canManageSessions` — **a control an admin's scope cannot call must not render**, or they get a 403 with nothing to explain it. At step 7 the finance admin reads the money sections and has **no** control that changes a session. The same shape holds for a therapist at `/admin/dashboard/therapists/<id>` (**Save Notes** there posts `update-therapist-notes`).

`ADM-PEOP-004` — Condition access grants and change requests · P0

Steps. Approve a therapist's access-grant request. Then approve a patient's condition change request. Then approve a **therapist-submitted** edit for a re-triaged patient. **Expected Result.** The grant lets the therapist edit that patient's record. **Approving a change MERGES; it never replaces** — see `THR-HP-004`. Revoking a grant that is not approved is refused with `Only an approved grant can be revoked`. A stale decision returns `Someone else reviewed this. Refresh to see what they said.`

`ADM-PEOP-005` — Therapists directory · P1

Expected Result. A paged list with approval state, active state, leave state, team visibility, revenue share and rating visibility.

`ADM-PEOP-006` — Therapist detail and revenue share · P0

Steps. Open `QA Therapist A`. Set **Revenue share %** to `60`, and the home-visit share to `65`. Save. Then try `-5` and `150`. **Expected Result.** Valid values save and immediately change the therapist's Earnings and the Money screens' split. Invalid values are refused with `Enter a percentage between 0 and 100.` The change is audited.

`ADM-PEOP-007` — Suspend and restore a therapist · P1

Steps. Toggle Therapist A inactive, then active. **Expected Result.** While inactive: their dashboard redirects to `/account-suspended`, their API routes 403, they disappear from `/team` and from `?therapist=` resolution, and they cannot be assigned. Restoring reverses all of it. **Their existing appointments are unchanged.**

`/team` must update immediately, not eventually. Open `/team` in a second tab *before* suspending, suspend, then **reload that tab** — the therapist is gone on the next load, not after the five-minute cache window lapses. The public view already excludes them (`approved and active and visible_on_team`); what was missing was the route telling the cached page. Check the same for **approving** a pending therapist and for **creating** one from User Access — both put a therapist on `/team` at once, since the visibility column defaults to showing. **Declining** a pending signup deliberately does not invalidate anything: an unapproved account was never on that page.

And the "Hide from /team page" button follows the account. While the therapist is suspended it is **greyed out and does nothing**, with a line saying they are already off `/team` and that the setting returns as you left it. Same while a therapist is still **pending approval**, worded for that reason instead. Restore them and the button is live again, showing the state it had before — set it to hidden, suspend, restore, and it must still read **Show on /team page**, never reset to the default.

`ADM-PEOP-008` — Partners · P1

Covered by `HOS-AUTH-002`, `HOS-MONEY-*`. Additionally: **Copy invite link**, **Update revenue share**, **Set active/inactive**, **Reset password**, **Referral capacity note**, and **Decline referral** (reason mandatory) all work and are audited.

`ADM-REF-001` — Contacting a referred patient before the link goes out · P1

Steps. Open **People → Partners → Patient Referrals** and read the card for the referral created in `HOS-REF-001`. **Expected Result.** Directly under the patient's name: their **phone number as a `tel:` link** and their **preferred language**, before the referring partner's name. That order is deliberate — an admin rings this patient to agree a time *before* the registration link is sent, so the number and the language they want to be spoken to in identify the referral rather than being detail about it. A referral taken before the phone field existed reads `No phone on file`; one with no language reads `Language not stated` — never a blank line. **Critical check:** the phone comes from its **own isolated query** (`patient_phone` is the newest column on `patient_referrals`), so a database missing that migration must lose the number on the card and nothing else — the capacity note beside it and the referral list itself must still render.

`ADM-REF-002` — Assigning a referral uses the patient's own calendar · P0

Steps. On a **Needs triage** referral, pick a therapist and open **Pick a time**. **Expected Result.** An inline compact month grid plus hour chips — **the same control and the same 12-hour lead-time rule as `/book` Step 1**, not a `datetime-local` box and not a pop-up. Today (and any hour inside the next 12) is **not clickable**; the picker opens on the earliest eligible slot with a time already chosen. Changing to a date that does not offer the chosen hour falls back to that day's earliest, never to a time the rule has just ruled out. The line under it reads `Earliest bookable time is 12 hours from now — the same rule the patient's own booking screen follows.` **Negatives:** `POST /api/admin/assign-referral` with a slot inside the window is refused with `The assigned slot must be at least 12 hours from now.` — the rule is re-checked server-side, not trusted from the browser. A card left open past the boundary is refused in the browser first with `That time is no longer far enough ahead. Pick a later slot.` **Unchanged:** therapist conflict detection, the home-visit travel buffer, the concurrent-assignment tiebreak and the registration link are all exactly as before.

`ADM-PEOP-009` — Reset a password · P1

Steps. Reset Patient A's password from the detail page. **Expected Result.** A new password is generated and shown **once**. **It is never written into the audit log's `details`** — the log is readable by every admin, so who reset what and when is the part with audit value. The patient can sign in with the new password and is prompted to change it.

14.4 Catalog

`ADM-CAT-001` — Create a treatment category · P0

Steps

1. Open **Catalog → Conditions**.
2. Tap the create control.
3. Tap **Category Name**. Enter `QA Back & Spine Care`.
4. Tap **Price (₹)**. Enter `1999`.
5. Tap **Session Length (min)**. Enter `60`.
6. Tap **Order**. Enter `1`.
7. Tap **Button Text**. Enter `Book Assessment`.
8. Optionally paste a **Cover Image URL**.
9. Save.

Expected Result. The category is created and appears **immediately** on `/` and `/conditions` — the create route invalidates both ISR-cached pages, so a five-minute wait is now a defect, not expected behaviour — in the `/book` concern dropdown as `QA Back & Spine Care — ₹1,999 / 60 min`, and as an option when creating a package. The cover image is a **plain URL an admin pastes**, not a Storage upload, rendered through a plain `<img>`; a row with no image shows the shared **placeholder panel at the same height**, never a broken-image state. **Negatives:** `Missing title, priceInr, or durationMinutes`; `Price `must be a positive number`; `Session length must be a positive number of minutes`; `Order must be a number`.

`ADM-CAT-002` — Edit, reorder, deactivate, delete a category · P1

Expected Result. Editing the price changes what `/book` charges **for new bookings** and is re-derived server-side at booking time. Reordering changes the display order everywhere.

Reorder, in detail. The up/down arrows rearrange the list **in the browser only** — nothing is written until **Save order** is tapped. That button is **always visible** and is **disabled until something has actually moved**; while there are unsaved moves the screen reads `Not saved yet — the public pages still show the old order.` beside an **Undo changes** link. Saving posts the whole list, renumbers it `1..n`, and shows `Order saved — live on the site.` **Critical check:** reload the tab after saving and confirm the new order **survives**, then open `/` and `/conditions` and confirm they show the same order **immediately** (the pages are ISR-cached, and the save invalidates them). A reorder that reverts on reload, or one the public pages ignore, is the P1 defect this case exists for — it was caused by a pairwise `display_order` swap that did nothing whenever two categories held the same number, which every category created without typing an **Order** did. **Negatives:** with two browser tabs open, add or delete a category in tab B, then Save order in tab A — refused with `The condition list changed while you were reordering it. Refresh and try again.` **[SQL] the same refusal one level down.** The route's completeness check is true only for as long as every caller remembers it, and the function is reachable by the service-role client and by hand in the SQL editor. Against a scratch database with `schema.sql` applied, call it with a subset — `select set_treatment_category_order(array[(select id from treatment_categories limit 1)]);` — and confirm it **raises** `set_treatment_category_order needs every category (1 given, 3 exist)` rather than renumbering one row. Renumbering a subset collides with the rows it never saw, which is how two categories end up sharing an order again — the exact tie the whole change removes. **Also:** creating a category now defaults **Order** to one past the last existing category, so a new condition appends rather than appearing first. Deactivating removes it from public surfaces and refuses new bookings against it (`That concern isn't available any more. Please pick another one.`) while **leaving existing appointments untouched**. Deleting a category referenced by a live purchase must not silently break that purchase.

`ADM-CAT-004` — Deleting a condition says what happened · P0

Feature. Every outcome of Delete is now reported. It previously answered success whether or not a row had gone — supabase-js reports no error when a DELETE matches nothing — so a refusal, an already-deleted row and a real deletion produced the same response, and the screen refreshed on that success and painted the condition still sitting there. **"Delete does nothing, and nothing says why" is the failure this case exists to catch.**

Steps

1. Create `QA Spare Condition` and, without booking anything under it, delete it.
2. On a condition with a booked session (`QA Back & Spine Care`), tap **Delete** and read the dialog.
3. Delete every session under some condition, leave **one home-visit package** filed against it, and delete it again.
4. In a second tab, delete a condition; in the first tab, tap **Delete** on the same row.
5. Tap **Delete**, and with the request in flight, drop the network.
6. Open **Logs → All Activity**.

Expected Result

- Step 1 deletes, the row leaves the list, and `/` and `/conditions` update immediately.
- Step 2 opens a **dialog** — not an 11px line clipped beside the button — naming the counts (`2 sessions and 1 programme purchase still use this condition…`) and saying to **turn it off instead**, with what

that preserves. HTTP **409**.

- Step 3 names the **home-visit package** specifically. A refusal that says "it has bookings" sends an admin to delete sessions and be refused a second time by something they were never told about.
- Step 4 answers **404** with `That condition has already been deleted. Refresh to see the current list.`
- Step 5 says `Could not reach the server. Nothing has changed.` — previously the thrown request left the transition with nothing on screen at all.
- The log's row names the **condition's title**, not "Treatment category": the row is gone by then, so the log is the only thing left that can say which one it was.

Never `{ success: true }` on a delete that removed nothing. If the row is still listed after a success and a refresh, that is a P0 — it is the exact defect this case was written for.

`ADM-CAT-005` — Create a session package · P0

Steps. Create Package P1 exactly as specified in §8.11. **Expected Result.** Saved. The **Category is set once at creation and locked afterwards** — live purchases reference it. The package becomes available to therapists as a recommendable programme for that condition **only**. On the public site it appears as a card with **no Buy button**, showing instead *"Arranged by your therapist after your first session."* **Negatives:** the five validation messages in §8.11.

`ADM-CAT-006` — Editing a package never changes what was already sold · P0

Steps. After Patient A has purchased P1 (6 sessions, ₹9,999), change P1 to 4 sessions and ₹12,000. Then open the patient's package widget and the admin's Purchases row. **Expected Result.** The purchase still reads **6 sessions** at the price paid. `sessions_granted` and `package_snapshot` are **frozen by trigger**. **Never resolve a purchased entitlement by joining the live catalog row** — if the patient's widget now says 4 sessions, that is a P0 defect. **[SQL] confirmation:** attempting `update patient_package_purchases set sessions_granted = 4 where id='<id>'` **raises**.

`ADM-CAT-007` — Package rules reach the booking · P1

Steps. With P1's minimum gap `24h` and max `3`/week, have the patient attempt to book two sessions 12 hours apart, and four in one week. **Expected Result.** Both refused by the batch rules layer, with readable messages. Changing the rule in the catalog changes the refusal for **new** scheduling attempts.

`ADM-CAT-010` — Service areas · P0

Steps. Create Area 1 and Area 2 from §8.13. Then try to create a third area containing `560038`. **Expected Result.** Both save. The third is refused with `Another service area already covers that pincode.` (or `Every pincode in that list is already a service area.`). Other negatives: `Enter at least one pincode.`, `Enter a valid 6-digit pincode.`, `City is required.`, `Travel Fee must be zero or a positive number.` **Dependency:** creating `560038` makes `/book-home-visit` accept it; deleting it makes the same pincode fall through to the waitlist. **Re-checked server-side at every purchase route** — never trust a serviceability answer the browser already has.

`ADM-CAT-011` — The home-visit waitlist · P2

Expected Result. Entries from `PAT-HV-003` appear with status `new`, raising the Service Areas badge. Updating a status clears the badge. `Unknown status` is refused for an invalid value.

`ADM-CAT-014` — Purchases · P1

Steps. Open **Catalog → Purchases**. Open a package purchase's detail modal; then a home-visit purchase's. **Expected Result.** Both list every purchase with balances. The detail modals are **viewer-**

scoped, not role-branched — the route queries with the caller's own RLS-scoped client, so a row coming back **is** the authorization. Money controls inside the modal render only for an admin with `money` scope. Admin actions available here: **extend expiry** (invalid dates refused with `Invalid newExpiresAt`), **reassign the locked therapist** (touches **future sessions only** — completed ones keep whoever actually ran them), **refund**, **restore a session**, and **grant / reverse / revive credits**.

`ADM-CAT-015` — Credit adjustments require a reason · P0

Steps. Grant 2 extra sessions with the reason `Goodwill` (7 characters), then with `Goodwill after a cancelled session.` **Expected Result.** The short reason is refused — `admin_adjust` is the **only** entry type with free-form deltas and the **only** one requiring a reason, **ten characters minimum, enforced by a CHECK** so it holds for any caller. The valid one succeeds and appends a ledger row. **An admin can change any balance and cannot change any history. [SQL]:** `update session_credit_ledger …` and `delete from session_credit_ledger …` both **raise** — append-only by trigger, not by RLS, because every route writes with the service role.

---

15. Admin test plan — Part B (Settings, and configuration → dependent feature)

> **How to test a setting.** Every settings case follows the same four beats: **change it → reload the admin screen (it must persist) → open the dependent feature and prove it changed → change it back**. A setting whose dependent feature you did not check has not been tested.

Every setting below is read through one shared settings module with defaults. **Every dashboard page selects the same column list**, so a new setting cannot silently read as its default on whichever page forgot it.

15.1 Settings → Brand & Contact

`ADM-SET-001` — Brand & Contact Details · P1

What it controls. `site_name`, `site_tagline`, `site_description`, `contact_email`, `whatsapp_number`, `contact_phone`, `footer_copyright_text`. **What depends on it.** The **root layout** is the one place these are read (through a public/anon client, so ISR-cached pages under it are not forced dynamic) and it passes them into `Navbar` and `Footer` **as props**. Those two components must never fetch their own copy or hardcode a string.

Steps

1. Open **Settings → Brand & Contact**.
2. Change **Site name** to `QA Physio Clinic`. Save.
3. Reload the admin page. Confirm the value persisted.
4. Open `/` in a new tab.
5. Change **Footer copyright text** to `QA Physio Clinic. All rights reserved.` Save. Reload `/`.
6. Change **Contact email** to `not-an-email` and save.
7. Restore every original value.

Expected Result. Step 4: the navbar and the page title/description show `QA Physio Clinic`. Step 5: the footer shows the new line. Step 6: refused with a validation message (`Enter a valid email address.`).

Step 3 and every reload: values persist. Existing records are unaffected — this is presentation only. **Interaction worth checking:** the **splash** brand line is **blank by default and falls back to the site name**, so changing the site name also changes the splash greeting until an admin deliberately parts them.

15.2 Settings → Public Site

`ADM-SET-004` — Ratings visibility · P2

Steps. Toggle **ratings visible publicly** off. Open `/team` and `/`. **Expected Result.** The public rating summary disappears from public surfaces. Individual therapist rating visibility is a **separate** per-therapist toggle on their detail page; verify both independently.

`ADM-SET-005` — Home page walkthrough pace · P1

Steps. Set the walkthrough seconds to `2`. Open `/` and watch the "How the process works" widget. Then set it to `0`. **Expected Result.** At `2`, each step holds ~2 seconds before the next takes over. At **\`0\` it does not advance on its own** — the same "0 is off" convention used by the session timeout. Values over 300 are refused (`Keep this to 300 seconds or less.`). **Accessibility interaction:** a visitor with **reduced motion** requested must not be subjected to auto-rotation. **Do not confuse this widget with the care-area showcase**, which **never advances by itself** — a second moving thing while you read the first is worse than either alone. Their `aria-label`s must stay distinct: "How the process works" vs "Areas of practice".

`ADM-SET-006` — The opening splash, all five settings · P1

Feature. A teal sheet painted over the site for a beat. It greets a **cold open** — the first load of a browser tab — and a **return to a tab that has been in the background longer than the away threshold**. It deliberately does **not** show on every navigation, every reload or every tab focus, because a patient paying by UPI leaves the tab for their bank's app and comes back mid-checkout, and **splashing over a payment in progress is the one thing this must never do.**

Setting	Test
On/off	Off → no sheet on a cold open in a fresh tab
Name above the line	Blank → falls back to the site name. Set it → the splash and the navbar deliberately differ. **Blank is a value here, not an error** — it is how the override is undone.
The one line	Changing it changes the greeting text
Hold (seconds)	A longer hold visibly holds longer
Away minutes	Set `1`. Open a tab, switch away for 90 seconds, switch back → greeted again. Set `0` → **first load only**, and returning is never greeted. **There is deliberately no value meaning "greet on every tab focus".**

Expected Result also to check: a **reload** of an already-greeted tab is **not** greeted. Someone who has asked for **reduced motion is skipped outright**. The overlay's markup is present in every page's HTML and visibility is driven by a `data-splash` attribute on `<html>` — not React state — so there must be **no hydration warning** in the console.

`ADM-SET-007` **— Testimonials · P1**

Steps. Create a testimonial with patient name `QA Story` and quote `The exercises made a real difference in six weeks.` Save. Open `/` and `/mission`. **Expected Result.** It appears **immediately** in the same band on **both** pages (the route invalidates `/` and `/mission`, both ISR-cached) — one component serves both, because the two bands make the same claim and a visitor may see both in one session. The avatar is optional; with none, the **patient's initial** is shown, never a generic silhouette. **Critical check:** the five rows the schema seeds are **illustrative copy, not real patients**, and the admin form must **say so at the point of entry**. Never present a seeded testimonial as real. The only place a **real** number is quoted is the public rating summary. **Negatives:** `Missing patientName or quote`; editing requires `Missing id, patientName, or quote`.

`ADM-SET-008` **— FAQ · P2**

Steps. Create, edit, reorder and delete an FAQ. Open `/faq`. **Expected Result.** The public accordion reflects each change **immediately** — `/faq` is ISR-cached and every FAQ route invalidates it. Negatives: `Missing question or answer`, `Missing id, question, or answer`.

15.3 Settings → Booking Rules, Offers & Discounts, Programmes & Home Visits

These were one tab and are now three. "Booking Rules" had grown six unrelated stacks with no heading between them, so an admin opening it to change a refund window scrolled past the discount that decides what every new patient pays. What lives where now:

Screen	What it covers
Booking Rules	One video session, start to finish: when it may be booked, when it may be cancelled with a refund, when the Join button works, and the Google Meet / Calendar switches. Plus the two platform-wide odds and ends (idle timeout, sign-out message).
Offers & Discounts	Money off, to win a patient: the first-session offer and patient invites. A note on the screen points at Money → Costs for promo codes and at the session itself for a goodwill discount.
Programmes & Home Visits	More than one appointment, arranged in advance: the recommendation settings, the package settings, and the nine home-visit settings.

Every Settings screen also states what it is and gives one example, under its heading — check that line renders and matches the screen you are on. The cases below keep their original IDs; the **Where** line on each says which of the three screens it is now on.

Where a case below still says "Settings → Booking Rules", that is correct — it did not move.

`ADM-SET-009` **— Every Settings screen says what it is · P2**

Steps. Open each of the nine Settings screens in turn: Brand & Contact, Public Site, Booking Rules, Offers & Discounts, Programmes & Home Visits, Clinical Questions, User Access, System Health, Account Security. Then open both Logs screens: All Activity and Archive & Clear.

Expected Result. Under the page heading, each one shows **two lines**: one plain sentence saying what the screen is, and a second beginning **"For example:"** with one concrete thing you would come there to do. The sentences differ per screen — none of them says "How the product behaves", which is the section's line and is what every one of these screens used to show. No jargon, no database column names, no feature names.

Spot checks. Offers & Discounts ends with a **"Looking for promo codes?"** note pointing at **Money → Costs**, and saying a goodwill discount is applied to a session rather than set up here. Booking Rules holds **only** the single-session rules and the Google Meet block — no discount, no package and no home-visit settings on it any more.

Critical check: these are the same ten screens the sidebar lists and the same ten `?tab=` values. A screen reachable from the sidebar with no sentence under its heading, or a sentence on a screen that is not in the sidebar, means `adminNav.ts` and the shell have drifted.

`ADM-SET-010` — Online Booking Lead Time → the booking wizard · P0

Configuration. `online_booking_lead_time_hours`, default **12**. **Dependent features.** The `/book` Step 1 calendar and hour list **and** `/api/appointments/create`'s own validator — deliberately the same setting, so the picker can never offer a slot the server rejects.

Steps

1. Note the current earliest bookable slot on `/book` under simulated time `2026-09-10T10:00`.
2. In **Settings → Booking Rules → Online Booking Lead Time**, change `12` to `48`. Save.
3. Reload the admin page and confirm it persisted.
4. Reload `/book` under the same simulated time.
5. Attempt a booking at the old boundary (10 September 22:00) by any means, including a direct API call.
6. Change it back to `12`.

Expected Result. Step 4: the earliest offered slot is now **12 September at 10:00**, and 10–11 September are greyed. Step 5: the API refuses with **409** and `Please pick a slot at least 48 hours from now.` Step 6: the picker returns to the 12-hour boundary. **Existing bookings are untouched** — this rule applies to new bookings only. **Negatives:** `value must be a non-negative whole number`.

`ADM-SET-011` — Online Cancellation Refund Window → the cancel dialog and the refund · P0

Configuration. `online_cancellation_refund_hours`, default **24**. **Steps.** Change it to `72`. Reload `/book` Step 3 and read the cancellation notice. Then cancel a paid session whose slot is 48 hours away. **Expected Result.** Step 3's notice now reads *"Free cancellation up to 72 hours before your slot…"*. The 48-hour-away cancellation now falls **inside** the window: the dialog says it will not be refunded, and **no refund is processed**. Restore `24`. **Independence check:** this must **not** change the **home-visit** refund dialog, which reads its own setting.

`ADM-SET-012` — Booking Languages → the Step 1 chips · P1

Steps. Add `Hindi` and `Kannada`. Save. Reload `/book`. Then remove every language and save. **Expected Result.** Three chips appear in Step 1 in the configured order; the first is auto-selected. A language not on the list is **dropped server-side** rather than stored as a preference nobody is matched on. Removing all is refused with `Keep at least one language — booking needs something to offer.` — booking must never present an empty language picker. Duplicates that differ only by case are de-duplicated.

`ADM-SET-013` — Home Visit master switch → six surfaces at once · P0

Configuration. `home_visit_enabled`, default **off**. **Dependent features.** `/home-visit` (404s when off), the header nav, the footer Explore column, the home page connector grid, every "Where to go next" strip, the hospital's **Session Type** option, `/book-home-visit`, and `/api/care-plan/create-order`'s re-check.

Steps. With it **off**, check all seven surfaces. Switch it **on**, check all seven again. Then, with a **live unpurchased home-visit recommendation** outstanding, switch it off and have the patient try to buy. **Expected Result.** Off: `/home-visit` is a 404 and the entry is **dropped from every list rather than linking into a dead end**; the hospital cannot choose a home-visit referral. On: everything appears. The care-plan purchase with the switch off is refused with `Home visits aren't available right now. Please talk to your therapist.` — **an admin who switches home visits off has stopped the service, and a recommendation written before that must not stay purchasable.**

`ADM-SET-014` — The remaining home-visit settings · P1

Setting	Default	Dependent feature to verify
Allow cash on visit	on	The cash option on `/book-home-visit` Step 4; `/api/home-visit/book-cash` refuses when off
Booking lead time	24 h	The wizard's Step 2 copy and its picker; **must stay independent of the online 12 h**
Travel buffer	45 min	A locked therapist's conflict check is padded by this on **both** sides of a new visit; online passes **0**
Full-refund window	24 h	The home-visit cancel dialog and the refund actually paid
Default package validity	90 d	A new home-visit purchase's expiry when the package leaves it blank
Bulk scheduling limit	8	`Too many slots in one request.` above this
Page heading / subheading	(defaults)	The `/home-visit` page's own copy

`ADM-SET-015` — Session Timeout of Inactivity · P2

Steps. Set it to `1` minute. Sign in as a patient and leave the tab idle. **Expected Result.** The idle dialog appears and signs the patient out to **their own login page** (`/patient/login`, `/therapist/login`, `/hospital/login` — the door they came in through). **`0` means off. Admins are exempt from the inactivity timeout entirely.**

`ADM-SET-016` — Sign-out message duration · P3

Steps. Set `farewell_banner_seconds` to `2`, then `0`. Sign out each time. **Expected Result.** At `2`, the banner clears after two seconds. At **`0` it stays until dismissed**.

`ADM-SET-017` — Google Meet toggle, join window, and the Session Completed cutoff · P0

Setting	Default	Verify
Auto-Create Meet Links	on	Off → a newly confirmed **online** session gets no Meet link. **A home visit still gets a calendar event** — this toggle gates the Meet conferencing only, not event creation.
Join Button Window (before)	15 min	Set `5`; the join control goes live 5 minutes before the slot on **every** surface
Join Button Window (after)	15 min	The short grace period for a late arrival
Session Completed Cutoff	60 min	Set `30`; 31 minutes after the slot **every** join control on **every** surface — patient, therapist and **admin** — reads **Session Completed**

Note: the cutoff is **not** the same thing as the after-window. See `XR-CUTOFF-001` for the cross-role check.

`ADM-SET-018` — Package settings · P1

Setting	Default	Dependent feature
Assign a Therapist Automatically	**off**	See `ADM-SET-021`
Therapist Lock (site-wide)	on	Off → later sessions on a purchase are not auto-assigned to the first therapist
Session Balances From The Ledger	**off**	See `ADM-SET-019`
Therapist-Suggested Sessions	**on for a fresh database**	Off → `/api/therapist/suggest-session` returns `Suggesting sessions is switched off.` and the control is absent. The column default is now true, but that only applies to a new `site_settings` row — **an existing database keeps its current value until an admin toggles it, or a reset restores defaults.** Check the toggle before running `THR-SUGG-*` rather than assuming
Default Validity	90 d	A new purchase's expiry when the package leaves it blank
Bulk Scheduler Limit	8	`Too many slots in one request.` above this
Expiry Reminder Lead Time	14 d	When the expiry nudge appears on the patient's dashboard

`ADM-SET-022` — Recommendation settings · P0

At **Settings → Programmes & Home Visits**, above the package settings.

Setting	Default	Dependent feature
Approve recommendations before the patient sees them	**on**	Off → a therapist's submission publishes on save and the patient sees it immediately, as it did before the review step. On → it lands in Sessions → Recommendations and the patient sees nothing. See `ADM-CARE-004..008`. **Fails closed:** an unreadable column holds the recommendation rather than publishing it. Read in its own query, not through `SITE_SETTINGS_SELECT` — the same treatment **Therapist-Suggested Sessions** gets, so a database one apply behind loses this one control instead of resetting every setting on the page
How long a recommendation holds	30 d	The patient's answering window, counted **from approval**, not from when the therapist wrote it

Setting	Default	Dependent feature
Most sessions a week a clinician may ask for	5	A ceiling over the programme's own `max_sessions_per_week`, whichever is lower. Above it: `This programme allows at most N sessions a week.`

Note there is no longer a **Show programme prices publicly** switch. Programmes are not advertised on the public site at all — see `PUB-CAT-002`.

`ADM-SET-023` — First session offer · P0

At **Settings → Offers & Discounts**, above Patient invites.

Setting	Default	Dependent feature
First session offer	**off**	On → a patient who has never paid for a session is charged the offer price for a video consultation. Off → everyone pays list price
Offer type	`A set price`	`A set price` names what they pay ("₹499"); `A percentage off` adapts across categories priced differently
Offer value	0	Rupees or whole percent depending on the type. The panel previews what a real category's session would cost a new patient

Steps. Turn it on, set a set price of ₹499. Read the preview. Book and pay as a **brand-new** patient, then as one who has paid before. **Expected Result**

- The preview quotes a real category's price — *"A ₹1,200 session would cost a new patient ₹499."*
- The new patient is charged **₹499**; the returning patient is charged **₹1,200**. Eligibility is decided server-side from payment history, so it cannot be requested, repeated, or sent from the browser.
- Programmes and home visits are **not** discounted by it.
- Values the route must refuse: a type other than `fixed` / `percent`, a value of 0 or negative, a non-boolean for the switch.
- Set 100% off: the patient is charged the ₹1 minimum rather than zero — Razorpay refuses a zero-amount order, so an unguarded 100% would be a 500 at the last step of checkout.

`ADM-SET-021` — Automatic therapist assignment · P0

Feature. When a session is paid for and **exactly one** therapist is unambiguously free for it, assign them and confirm the booking immediately instead of leaving it in the admin queue. It reads the roster (weekly template + that date's exceptions + leave) and the same conflict check the admin's assign form uses. **It does not change what times a patient is offered** — the roster still does not filter the picker.

#	Set up	Expected
1	Switch **off**. Book and pay a session.	Session is `requested`, **unassigned**, in the queue — the pre-existing behaviour.
2	Switch **on**. Roster **only** Therapist A for the slot's hour. Book and pay.	Assigned to Therapist A, `confirmed`, Meet link created, and the therapist sees it immediately. The unassigned badge does **not** rise.

#	Set up	Expected
3	Roster **both** Therapist A and B for that hour, neither busy. Book and pay.	**Nothing is assigned.** The session waits in the queue. Two free clinicians is a choice for a person.
4	Both rostered, but Therapist B already has a clashing session.	Assigned to **A** — one free candidate.
5	Both rostered and free, and the patient booked via `/book?therapist=<B>`.	Assigned to **B**, not A. A stated preference beats the count.
6	Patient requested B, but B is busy; A is free.	Assigned to **A**. The preference is dropped rather than the session waiting.
7	Patient requested a therapist who is **not rostered** for that hour.	Falls back to the count. A stale `?therapist=` link never overrides the roster.
8	Nobody rostered for that hour.	Nothing assigned; queue as before.
9	Therapist A rostered but **on leave**.	Not a candidate.
10	Therapist A rostered but **unapproved** or **inactive**.	Not a candidate.
11	A **home visit**, with two therapists free but one finishing a visit within the travel buffer.	The buffered one is treated as busy — the conflict check is padded by `home_visit_travel_buffer_minutes` on both sides.
12	Pay, then close the tab before the callback lands (webhook configured).	The **webhook** applies the same assignment. Both paths use one decision, so they cannot disagree about who is free.

Expected Result throughout. No session is ever assigned to a therapist who is unavailable, on leave, unapproved, inactive or already booked. When it declines to choose, the outcome is **identical to the switch being off**. A failure inside this logic must never fail the payment — the appointment is still marked paid either way.

Cross-check `XCFG-ROSTER-001` afterwards: rostering changes must still leave `/book`'s picker byte-identical.

`ADM-SET-019` — The ledger authority switch · P0

Feature. Whether a balance shown and offered is read from the **credit ledger** or from the older `sessions_used` / `visits_used` counters is **one admin switch**, off by default and **reversible in a second** — both are written either way.

Steps

1. Open **Settings → System Health** and confirm **Books & Sessions Agree** is `Healthy`, and that **Google Connection** is not red.
2. Turn **Session Balances From The Ledger on**.
3. Check every surface that shows a balance: the patient's package widget, the therapist's programme list, both purchase detail modals, the admin Purchases table, and the bulk scheduler.
4. Turn it back off and check them all again.

Expected Result. Every surface reads the same shape and follows the switch **together** — the substitution happens once, where the row is loaded. A refunded package still reads its original `6 sessions` with none pending rather than becoming a 1-session package. A purchase with **no** entitlement behind it is untouched, so a database without the backfill behaves exactly as before. **Important:** the switch **does not change how a session is claimed.** The counter's compare-and-swap still wins the booking race, with the ledger's row lock beside it. Flipping the switch must not change any booking outcome. The screen warns you to turn it on only once System Health has been clean.

`ADM-SET-020` — Clinical Questions · P0

Feature. The Health Profile question sets and the Pain Map templates. **Editing these changes what is asked from here on; answers already submitted are untouched.**

Steps

1. Open **Settings → Clinical Questions**.
2. Confirm the question bank is presented as **one tab per specialty**, not three stacked sections.
3. On the **Orthopaedic** tab, change the wording of `severity` and save.
4. Open a patient's intake wizard.
5. Open an existing patient's already-answered profile.
6. On **Enabled condition types**, switch **Paediatric** off.
7. As a therapist, open triage for a new patient, and then re-triage an existing paediatric patient.
8. Attempt to switch **Orthopaedic** off.

Expected Result

- Step 2: tabs, because twenty-odd textareas stacked is the wall-of-fields shape this product keeps correcting.
- Step 4: the new wording is shown. A new question requires **`helpText`** (why this answer matters, in the patient's words) and a **`shortLabel`** alongside its label — a question added without them is a defect.
- Step 5: **the already-submitted answer is unchanged.**
- Step 7: Paediatric is **removed from the triage picker** — and **an existing profile carrying it still renders**, and a therapist **re-triaging such a patient is still offered it**. If a live paediatric chart blanks, that is a P0 defect.
- Step 8: **Orthopaedic can never be switched off.**
- `schema_version` is **per specialty**, so changing a neuro question must **not** fire the "we've changed some of these questions" banner at orthopaedic patients.
- Pain Map templates edit per region and question; unknown values are refused with `Unknown region` / `Unknown questionKey for this region`.

`ADM-SET-025` — User Access: scopes · P0

Steps

1. Open **Settings → User Access**.
2. Read the admin list.
3. Attempt to change **your own** scope.
4. Narrow every other `full` admin, then attempt to narrow the last one.

Expected Result. Step 3: refused with `You can't change your own access. Ask another Master Admin.` Step 4: the last `full` admin (**Master Admin** in the picker) **cannot be narrowed** — otherwise a single mis-click locks everyone out permanently. Only a `full` admin can change scopes or mint another admin.

`ADM-SET-025a` — User Access: what each level can do · P1

Steps. On **Settings → User Access**, switch the toggle from **People** to **What each level can do**.

Expected Result. A table: rows are jobs in plain words, grouped by section (Today, Sessions, People, Money, Catalog, Settings); columns are **Master Admin · Operations · Finance · Clinical**. A legend names the three levels — **View and edit**, **View only**, **No access**. Each group heading also states the level each desk holds for that whole section.

Spot checks.

- Under **Sessions**, Finance shows a **View only** mark on *"See every session and who is running it"* and **No access** on *"Assign or change a session's therapist"*.
- Under **Money**, Operations and Clinical show **No access** on every row.
- Under **Settings**, only Master Admin shows anything.

Critical checks.

- **There are no checkboxes here.** Every cell is a read-only mark. If any cell can be clicked, that is the bug — a tick that does not also change what the server allows is worse than no tick.
- **It agrees with reality.** Pick any row showing access for a scope, sign in as that scope, and confirm the control is there. Pick any row showing none, and confirm both that the control is absent *and* that calling the route directly returns 403 (see `ADM-SET-027`).

`ADM-SET-025b` — Suspend and restore a back-office account · P0

Steps

1. On **Settings → User Access → People**, note each admin's status chip (**Active** / **Suspended**).
2. Suspend `qa.admin.ops@example.test`. Sign in as them.
3. Restore them. Sign in again.
4. Attempt to suspend **yourself**.
5. Suspend every other Master Admin, then attempt to suspend the last one who can still sign in.

Expected Result. Step 2: the row reads **Suspended**, and that account **cannot get in** — `getAdminUser` and the proxy both refuse an inactive admin, so a still-valid session cookie cannot POST an admin route either. Step 3: access returns. Step 4: refused with `You can't suspend your own access. Ask another Master Admin.` Step 5: refused — `This is the only Master Admin who can still sign in. Make someone else a Master Admin first.`

Critical checks.

- **The account is suspended, never deleted.** Their name still appears on every activity-log row they wrote. An account that could be deleted would take the record of what it did with it.
- A suspended admin **stays listed**, with a note saying how many are suspended.
- Only a **Master Admin** sees the Suspend button at all, and `POST /api/admin/set-admin-active` answers **403** to any other scope.

`ADM-PEOP-011` — The detail page behind the overlay is still the back office · P1

Feature. A patient's, therapist's or condition's detail is normally an overlay over the dashboard. Interception only covers client-side navigation, so a reload, a shared link, a new tab and any `router.refresh()` from inside the overlay land on the real page — which used to be a bare panel with a small "Back to Dashboard" link and none of the dashboard around it.

Steps

1. **People → Patients → a patient → Profile.** Press **Mark Done** on a session and confirm.
2. Look at what you are on afterwards: is there a sidebar?
3. Copy the URL, open it in a **new tab**, and reload it.

4. Use the sidebar from that page — tap **Sessions**, then come back and tap **Back to the dashboard**.
5. Repeat 1–4 for a **therapist** and for a **patient condition**.
6. Sign in as **Operations**, then **Finance**, then **Clinical**, and open a patient detail URL directly.
7. Watch the screen while it loads on a slow connection.

Expected Result

- Steps 1–3: the dark **sidebar**, the **Master Admin** brand and the section list are all present. It reads as the back office, not as a different, plainer site.
- Step 4: every entry lands on a **real dashboard screen**, and Back to the dashboard returns to Today.
- Step 5: identical on all three.
- Step 6: the sidebar shows **only the sections that scope can open** — it is built from the same list and the same scope grid the dashboard's own sidebar uses, so the two can never disagree. Logs is absent for all three.
- Step 7: the skeleton **keeps the sidebar rail** rather than blanking the chrome and putting it back.

`ADM-SET-025d` — Delete an account, and be refused when it has history · P0

Feature. Delete account sits on all four roles' screens — the Back office rows, and a patient's, therapist's and partner hospital's own page — for a **Master Admin only**. It can only ever succeed on an account with no history at all. That is the database's rule: thirty-five tables reference `profiles(id)` with no delete behaviour, so removing an account that has done anything would mean removing the books and the audit trail with it.

Steps

1. Create a throwaway patient with a typo'd email and, without doing anything else with it, delete it from their profile.
2. Delete a patient who has at least one session, one payment and a programme.
3. Delete a therapist who has run sessions and written notes.
4. Delete an admin who has performed any action at all.
5. Try to delete **your own** row in Back office.
6. Leave exactly one active Master Admin and try to delete it.
7. Sign in as **Operations**, then **Finance**, then **Clinical**, and look for the button. Then POST `/api/admin/delete-account` directly as each.
8. Open **Logs → All Activity** after step 1.
9. After step 1, try to sign in as that deleted account.

Expected Result

- Step 1: a **dialog** first, naming the account and saying this only works with no history, with Suspend named as the alternative. Confirming deletes it; the page returns to the directory it came from.
- Steps 2–4: **nothing is deleted**, and a dialog names what is on file — *"3 sessions, 2 money records, 1 programme and 12 back-office actions"* — and says to suspend instead. Each group is counted separately and pluralised on its own count. A refusal is never an 11px line beside the button: it is a paragraph and it belongs in a dialog.
- Step 5: refused — *"You can't delete your own account."* The button does not render on your own row either.
- Step 6: refused — the last Master Admin who can still sign in cannot be deleted, the same guard suspension carries, and this one has no undo at all.

- Step 7: **no button** for any of the three, and the direct POST is **403** for all three — deleting is Master Admin's alone even though every one of those desks can manage People.
- Step 8: an **`Deleted an account`** entry naming who did it, the account's name and role, and the email. It is written **before** the delete runs, because afterwards there is no row left to name.
- Step 9: the login no longer exists.

`ADM-SET-026` — Create the three scoped admins · P0

Steps. Create `qa.admin.ops@example.test` (Operations), `qa.admin.finance@example.test` (Finance), `qa.admin.clinical@example.test` (Clinical). **The Account type picker is one control, not two.** It lists six entries in two groups — **Clinic**: Patient, Therapist · **Back office**: Master Admin, Operations, Finance, Clinical — using the same four names the dashboards call themselves. There is no separate **Access level** dropdown; picking a back-office desk shows that desk's one-line description under the picker. As a **non-`full`** admin, the whole Back office group is **absent** (only a Master Admin may mint an admin, and `create-account` enforces that with a full-only check, not a section gate — see §2). **Expected Result.** Each is created with a generated password that is shown on the panel **and kept on that admin's own row in Back office** until they set their own, and is **never logged** (`ADM-SET-026b`). Signing in as each shows only their allowed sections in the sidebar — Operations: Today, Sessions, People, Catalog. Finance: Today, **Sessions (read-only)**, People, Money. Clinical: Today, Sessions, People.

`ADM-SET-026b` — The issued password survives, and the chosen one is never shown · P0

Feature. Every route in this app that generates a password now stores the plaintext on a service-role-only table, so a credential cannot be lost to a re-render. Create-account was the last one that did not: it held the password in React state alone, and the `profiles` row it had just inserted fired a realtime refresh that took it off the screen mid-sentence.

Steps

1. Create a back-office account. **Without touching anything**, wait for the dashboard to refresh (or have a second admin approve something so a realtime event fires).
2. Reload the page entirely and open **Settings → User Access**.
3. Press **Copy** on that row.
4. Create a **patient** and a **therapist** the same way, then open each of their profiles under **People**.
5. Sign in as the new admin and change the password through the forgot-password flow. Reopen User Access.
6. Look for anywhere in the product that displays the password they just chose.
7. **[SQL]** `select * from admin_account_notes;` as an authenticated non-service-role session.
8. **[SQL]** Search `admin_activity_log` for any generated password.

Expected Result

- Steps 1–2: the password is **still readable** on that admin's row — the panel going away does not lose it. The row reads *Still on the password we issued*, with the date it was issued.
- Step 3 puts it on the clipboard.
- Step 4: the same password appears on the patient's and the therapist's profile, in the existing **Current admin-set password** panel beside Reset Password.
- Step 5: the row now reads **Signing in with their own password** and the password is gone — cleared by `/api/clear-temp-password`, so the screen never offers a credential that no longer works.
- Step 6: **nowhere, by design.** A password somebody chose is stored by Supabase as a bcrypt hash and cannot be read back by this app or anyone else. The lane for a locked-out account is **Reset Password**, which issues a new one and puts the row back into the first state.

- Step 7: **no rows** — the table carries no RLS policies at all, so only the service role reads it. A plain column on `profiles` would be handed straight back to the account owner by `profiles_select_own`, which is why these four tables exist.
- Step 8: **no password anywhere in the log**, which every admin can read.

`ADM-SET-025c` — Suspend and Create release the button, not the page · P1

Feature. Both controls used to run their request and the dashboard refresh inside one transition, so the button stayed disabled and spinning until the whole Server Component had re-run — every screen, ~40 queries — which reads as a hang rather than as work. The button now owns its request only; the teal bar at the top of the page owns the refresh.

Steps

1. On **Settings → User Access**, tap **Suspend access** on an admin and watch the button and the top of the page.
2. Do the same for **Restore access**, and for the **Access level** dropdown.
3. Create an account and watch the **Create account** button.
4. Double-tap Suspend as fast as you can.
5. Put the browser offline (devtools) and tap Suspend.
6. With a second admin signed in on another machine, have them approve something and watch how many times your dashboard rebuilds.

Expected Result

- Steps 1–3: the button returns to its normal label **as soon as the request lands** — a beat, not seconds — and the **teal progress bar** carries the remaining wait. The row's new state appears when the refresh finishes. Neither control sits disabled through the rebuild.
- Step 4: **one** request. The second tap is refused by a synchronous guard, not by the disabled attribute, which lands a render too late.
- Step 5: an error under the control saying the server could not be reached. Never a button that silently did nothing — an unhandled throw inside the old transition put nothing on screen at all.
- Step 6: **one** rebuild, not two. `admin_activity_log` sits on the 30-second realtime channel with the other append-only records; while it was on the 2-second one, every admin action anywhere rebuilt every admin's dashboard a second time for the log entry describing it.

`ADM-SET-026c` — "Forbidden" means forbidden, and nothing else · P1

Feature. Create account intermittently answered **Forbidden** to a Master Admin. The cause was not permissions: this dashboard fires many requests at once, Supabase rotates refresh tokens, and a request carrying one that another request has just rotated comes back with no user — which the guard reported the same way it reports a stranger. A failed profile read did the same.

Steps

1. Create several accounts in a row, quickly, while the dashboard is busy (have a second admin acting at the same time). Watch for any refusal.
2. As an **Operations** admin, try to create a **Master Admin** from the Account type picker, and POST `/api/admin/create-account` with `{"role":"admin"}` directly.
3. Signed out entirely, POST the same route.
4. As an Operations admin, POST it with `{"role":"patient"}`.

Expected Result

- Step 1: **no spurious refusal.** A session blip is answered 401 (*"Your session has expired. Sign in again and retry."*) or 503 (*"Could not check your access just now."*), and the form **retries once by itself** before showing either — both are answered before anything is written, so there is no account to duplicate. What must never appear is *Forbidden* for an admin who is allowed.
- Step 2: the Back office group is **absent from the picker**, and the direct POST is **403 Forbidden** — flat, with no detail. Only a Master Admin may mint an admin, and the refusal stays opaque so a limited admin cannot map what exists beyond their access.
- Step 3: **401**, not 403 — being signed out is not a statement about what you may do.
- Step 4: **succeeds.** Creating a patient is an operations capability.

`ADM-SET-026a` — Each scope opens on its own dashboard · P1

There is one admin login (`/admin/login`) and one dashboard route; the scope decides what it opens on. Sign in as each of the four in turn and read the Today screen without tapping anything.

Admin	Sidebar brand and header read	Today leads with	Quick actions
Master Admin	**Master Admin**	Sessions today	Approvals · All sessions · Money summary
Operations	**Operations**	Unassigned sessions	Assign a session · Approvals · Roster · New booking
Finance	**Finance**	Owed to therapists	Money summary · Payouts · Transactions · Costs
Clinical	**Clinical**	Recommendations	Recommendations · Patients · All sessions · Roster

Expected Result. The dashboard names itself in two places on every screen — the sidebar brand (above **Admin Panel**) and the eyebrow above the section heading — so an admin never has to work out which dashboard they are on from which sidebar entries are missing.

Three further checks, each one a bug this replaced:

- **Every quick action lands somewhere.** Tap all of them for each scope. None may bounce to a different screen — a link into a section the scope cannot open silently falls back to the first allowed one, which looks like it worked.
- **"Needs you" agrees with the list below it.** The figure counts only the queues in the **Queues** card beside it. Add up the badges; they must match. It must not count queues this scope cannot open **or can only read** — a Finance admin's figure must exclude the session queues under Sessions, since nothing they can do would ever bring that number down.
- **A limited scope gets a "Your access" card** under Queues, naming the sections it opens, the ones it can only **read** (Finance: Sessions — *nothing to change here*), and the ones it does not open at all. **Master Admin gets no such card** — nothing is withheld, so there is nothing to account for.

`ADM-SET-027` — Scope is enforced at the route, not the sidebar · P0

For each scoped admin, do **both**: navigate to a forbidden section by URL, **and** call a route in that section directly.

Admin	Forbidden URL	Forbidden route	Expected
Operations	`?section=money&tab=payouts`	`POST /api/admin/settle-therapist-payout`	Page falls back to an allowed screen; route **403**
Operations	`?section=settings&tab=booking`	`POST /api/admin/update-setting`	Same
Finance	`?section=sessions&tab=all` — **reachable, read-only**	`POST /api/admin/assign-appointment`	Page **opens** and shows the list with no action buttons; route **403**
Finance	`?section=sessions&tab=new`	`POST /api/admin/create-booking`	Tab is **not in the sidebar** and the URL falls back to an allowed screen; route **403**
Finance	`?section=catalog&tab=packages`	`POST /api/admin/create-package`	Same
Clinical	`?section=money&tab=summary`	`POST /api/admin/refund-package`	Same
Clinical	`?section=settings&tab=team`	`POST /api/admin/set-admin-scope`	Same
All three	`?section=today&tab=risk`	—	The tab opens with **only the signals that desk can act on**, and the flagged-message and reveal-log trails and the threshold editor are absent (`ADM-RISK-004`)
All three	`?section=logs&tab=all`	`POST /api/admin/clear-activity-log`	Logs is **not in the sidebar** and the URL falls back to an allowed screen; route **403** (`ADM-LOG-003`)

Expected Result. Every one returns 403 at the route. The sidebar hiding a section is presentation only — a session cookie can call any route directly. **Do not report the three full-only routes as violations.** `set-admin-scope`, `debug-reset` and `create-account` guard with an explicit `scope !== "full"` check rather than `requireAdminScope`, deliberately: a section gate would let a scoped admin widen its own access or mint a full admin. They are stricter than the rule, not exceptions to it.

`ADM-SET-028` — The section is chosen by the capability, not the button's location · P1

Purpose. A refund is `money` scope **even though its button lives on a Catalog screen**. **Steps.** As **Admin Ops** (who *can* open Catalog), open **Catalog → Purchases** and look for the refund control. Then call `POST /api/admin/refund-package` directly. **Expected Result. The refund control does not render** — a control an admin's scope cannot call must not be shown, or they get a 403 with nothing to explain it. The route returns **403**.

`ADM-SET-029` — Contact controls · P1

Steps. On **Settings → User Access**, change `contact_scan_mode` through `flag_and_block` → `flag_only` → `off`, and toggle `contact_masking_enabled`. **Expected Result.** As per `THR-LEAK-006` and `THR-SESS-

`003`. Note the deliberate asymmetry: **`contact_scan_mode` fails open, `contact_masking_enabled` fails closed** — the safe answer to "I don't know" is opposite for the two, on purpose. This tab also surfaces the `communication_flags` and `contact_reveal_log` evidence, **read-only**.

`ADM-SET-030` — System Health · P0

Steps. Open **Settings → System Health**. Read the verdict strip at the top, then tap the **i** button on each check. **Expected Result.** A verdict strip naming how many checks need a person (`All 5 checks healthy`, `2 checks need you`, or `Nothing is broken` when a check is switched off or could not be run), with a jump chip per failing check, followed by five cards in a fixed shape: **Payment Confirmations**, **Google Connection**, **Session Links**, **Waiting Room**, **Books & Sessions Agree**. Every card carries a status **word** as well as a colour — `Healthy`, `Needs a look`, `Needs you now`, `Not set up`, `Not checked` — a one-line plain-words headline, and, whenever it is not healthy, a numbered **How to fix it yourself**. The **i** button expands *What this watches* and *For example* inside the same card and nothing else moves. **Books & Sessions Agree** reports where the entitlement **cache**, the **ledger** and the **legacy counter** disagree, plus captured payments attached to nothing and delivered sessions with nothing behind them. **It reports and never repairs** — a silent auto-fix on a money record is how a discrepancy becomes permanent, and that card deliberately has no fix button. The badge on this tab is the number of **checks** needing a person, and it matches the strip's own count and its chips exactly. **Negative:** a missing `RAZORPAY_WEBHOOK_SECRET` and a dead Google credential both badged **0** before this, because the badge counted rows and those two failures have no rows.

`ADM-SET-030a` — System Health reaches the admin who never opens it · P1

Steps. Unset `RAZORPAY_WEBHOOK_SECRET` (or revoke the Google refresh token) and open **Today**. Then fix it and reload. Then, on **Settings → System Health**, read the line at the foot of each card and press **Copy for my developer** on a failing one. Finally sign in as an **Operations**, **Finance** and **Clinical** admin and open Today. **Expected Result.** One red line at the top of Today naming the failing check and carrying its own headline, linking straight to Settings → System Health; it is gone once the check is green. **Only a red check does this** — an amber one (a Google account connected without Meet permission, sessions still being retried) puts nothing on Today. Each card's foot reads `Checked just now`, except **Google Connection**, which reports the age of its own cached probe (up to ten minutes after a success, one minute after a failure) — so an owner who has just re-run the token script can tell a stale answer from a disagreement. **Copy for my developer** puts the check name, its status, the headline and the numbered steps on the clipboard as plain text; a healthy card offers no such button. The three scoped admins see **no banner at all** — they cannot open Settings, and a banner linking somewhere they cannot follow would land them on a different screen through `findTab`'s fallback.

`ADM-SET-031` — Session Links retry · P1

Covered by `ADM-SESS-003`.

`ADM-SET-033` — Activity Log · P0

Steps

1. Open **Logs → All Activity**.
2. Confirm each of these earlier actions appears with actor, action, target and timestamp: account approval, therapist revenue-share change, care-plan withdrawal, care-plan authored on behalf, payout settlement, cash amount correction, credit adjustment, hospital onboarding, password reset.

2a. Confirm the same for the actions that used to leave no trace at all: `Changed a patient's contact details` (`ADM-PEOP-003`'s contact edit), `Edited notes on a patient`, `Changed whether a therapist appears on Team`, `Excluded a session rating from the average`, `Changed a home visit's address`, `Decided a record-access request` and `Decided a health-profile change` (`ADM-PEOP-004`), `Reworded an `intake question` (`ADM-SET-020`), `Reviewed a risk signal` (`ADM-RISK-002`) and `Retried a Meet sync` (`ADM-SET-031`).

1. Search the log for any of the generated passwords from `ADM-SET-026` or `ADM-PEOP-009`.
2. **[SQL]** Attempt `insert into admin_activity_log …` as an authenticated (non-service-role) session, and attempt `update` / `delete`.

Expected Result. Step 2: **every one is present.** `payout.settle` is the largest money move in the application and must be attributed — if it is missing, that is a P0 defect. Step 3: **no password appears anywhere in the log.** Step 4: the insert is refused (there is a select policy and deliberately **no insert policy**), and the log is append-only from any session. **Ordering guarantee:** each log row is written **after** the route's compare-and-swap, so the log can never record a settlement or cancellation that lost its race.

`ADM-SET-034` — Reading one entry: what changed, from what · P1

Feature. Tapping a row opens the whole entry. It used to expand the route's raw JSON into the table cell (`{"fromPercent":40,"toPercent":55}`) — a developer's view of a record whose purpose is to be read months later by somebody asking what a colleague altered and what it was before.

Steps

1. Change a therapist's revenue share from one percentage to another, then open **Logs → All Activity** and tap that row.
2. Tap a row for **Changed a patient's contact details**.
3. Tap a row for a **plain care-plan approval** (no reason required).
4. Tap **Show the exact record** on any entry that has one.
5. Tap a row for **Edited notes on a patient**.

Expected Result

- A **dialog**, not an inline expander: admin, when, subject and amount at the top, then **What changed** as `40% → 55%` — the old value struck through in red, the new one in green — then **Also recorded** for everything else in the entry.
- Step 2 names **both old and new email and phone**. It previously recorded `emailChanged: true` — an entry saying a patient's sign-in address was altered without saying what it had been is unusable for the only question it gets asked.
- Step 3 says the action **recorded no further detail**, and that who/what/when is the whole entry — never an empty table that reads as missing data.
- Money reads as **₹2,499**, not `249900`; a date reads as a date in IST; `true` / `false` read as **Yes**/**No**; an absent value is a dash. A field the screen does not recognise is **still listed** — in an audit record the unfamiliar key is the one somebody is looking for — and the raw record stays available behind the toggle.
- Every dialog ends with the line saying the entry **cannot be edited by anyone**, including the admin who wrote it, and that after 30 days a Master Admin can clear it and that clearing is logged too. The first half is enforced by the table having no update policy; the second half is there because the screen must not promise something the product stopped keeping when Archive & Clear shipped.
- Step 5 shows a **note length**, never the note's text: this log is readable by every admin, and a note written about one patient must not be reproduced across the whole back office.

The one action logged after the fact on purpose: `Reset all data` (`SETUP-RESET-001`). The wipe truncates `admin_activity_log`, so the row is written **after** the reset returns — open the log on a freshly reset database and it holds exactly one row, naming who emptied it. A log that is completely empty after a reset means that row is missing, and the most destructive action in the product is unattributed.

11.x Logs

The Logs section is **Master Admin only**. Operations, Finance and Clinical have no such section in their sidebar and read their own desk's work on **Today → Activity**, filtered to their own domain and their own desk.

`ADM-LOG-001` — Finding one entry in a long log · P1

Feature. The log grows for ever, so All Activity is built around finding one row rather than reading the list. The dashboard's own render carries the newest 200 entries; older ones are fetched on demand.

Steps

1. Open **Logs → All Activity** as a Master Admin.
2. Type a patient's name into the search box. Then type an admin's first name and a word from an action together, e.g. `asha refund`.
3. Set **Any type** to **Money**, then to **Sessions**.
4. Set a **From**/**To** range covering yesterday only, then press **Clear filters**.
5. Press **Load older entries** twice, watching the `x of n loaded` count.
6. Press it repeatedly until the button is replaced by a line saying there is nothing older.
7. Export CSV and PDF with a filter applied.

Expected Result

- Search matches the **action's label as well as its stored key** — typing `refund` finds `refund.issue`, and typing `settled` finds `Settled payout` — and also the subject and the admin's name. Two terms **narrow**, they do not widen.
- The type dropdown offers **only categories that have rows behind them**; a category with nothing in it is not listed. Each row carries its type as a chip, and the categories are the dashboard's own section names plus **Master Admin only** for the four capabilities no desk holds.
- **Clear filters** appears only while something is set, and returns the count to `n of n loaded`.
- Each **Load older entries** press adds up to 200 more and the count rises. No entry is ever listed twice, including one written while the page was open.
- When the log is exhausted the button is replaced by *"That is the whole log"*.
- Both exports carry **exactly the filtered rows**, with a Category column, and the note under the button says filters and exports run over what is loaded.

`ADM-LOG-003a` — One subject's whole history · P1

Feature. The log answers *"what did this admin do"* well and *"what happened to this patient"* badly — and the second is the question asked when somebody complains. Tapping an entry's subject fetches every entry recorded against that record.

Steps

1. Do three different things to one patient as an admin (edit their contact details, refund a session, grant a credit).
2. Open **Logs → All Activity**, tap the refund row, and press **See everything done to this record**.
3. Tap another entry inside the timeline.
4. **Rename the patient**, then do one more action on them, and open the timeline again.
5. Tap a row whose Subject is a dash, or one whose action recorded no record (`Reset all data`).

Expected Result

- Step 2: a dialog listing all three, newest first, each with the actor, the time, the amount where there is one and its type chip. The entry you came from is marked **This entry**.
- **The entry dialog closes when the timeline opens** — one dialog at a time, never two stacked over the table.
- Step 3 opens that entry's own detail, including one **older than anything the screen had loaded** — the timeline is fetched by record, not filtered from the loaded page.
- Step 4: **all four entries are still one history.** The timeline is keyed on the record's id, not on the name snapshotted into each entry, so a rename does not split one patient into two.
- Step 5: **no button is offered** — an action that named no record has no history to trace, and the footer says the list is entries recorded against that exact record rather than claiming to be everything.

`ADM-LOG-002` — Archive & Clear, and the floor under it · P0

Feature. The only way a row ever leaves `admin_activity_log`. The whole point of the design is that an admin cannot act and then remove the record of having acted.

Steps

1. Open **Logs → Archive & Clear**. Try to find any control offering a cutoff inside 30 days.
2. Choose **Older than 365 days** and press **Count them**.
3. Try to press **Clear** before downloading anything.
4. Press **Prepare the archive**, then download the CSV.
5. Type `clear logs` in lower case, then `CLEAR LOGS`, and press Clear.
6. Change the cutoff to a different choice after step 4 and look at the screen.
7. Open **All Activity** afterwards.
8. **[API]** POST `/api/admin/clear-activity-log` with `{"olderThanDays": 1, "confirm": "CLEAR LOGS"}`, and again with `{"olderThanDays": 365}` and no `confirm`.
9. **[API]** POST the same route as an **Operations**, **Finance** and **Clinical** admin.
10. **[SQL]** `select public.purge_admin_activity_log(1);` against a scratch database.

Expected Result

- Step 1: **no such control exists.** The choices are 30, 90, 180 and 365 days, and nothing shorter is offered.
- Step 2: a count from the **server**, not from the rows on screen. Zero says plainly there is nothing to clear.
- Step 3: the Clear button is **disabled** until a download has actually been produced — a checkbox saying a copy was taken is deliberately not what unlocks it.
- Step 5: the lower-case phrase is refused; only the exact phrase works.
- Step 6: changing the cutoff **resets the count, the archive, the download and the typed phrase** — a copy taken for one year must not unlock a clear at three months.
- Step 7: entries older than the cutoff are gone, everything newer is intact, and there is a new **`Cleared older log entries`** row naming the cutoff, the protected window and the number removed. **That row cannot be cleared by any later clear**, because it is inside the 30-day floor.
- Step 8: both are refused with 400 and nothing is removed.
- Step 9: **403** for all three, the same answer a stranger gets.
- Step 10: the function **raises** — the floor holds where no route check runs, which is the case the database half exists for.

`ADM-LOG-003` — Who can open Logs at all · P0

Steps

1. Sign in as **Operations**, then **Finance**, then **Clinical**. Look for a Logs entry in the sidebar.
2. For each, navigate directly to `/admin/dashboard?section=logs&tab=all`, and to `&tab=retention`.
3. Open **Settings** as a Master Admin and look for an Activity Log screen.
4. Open **Settings → User Access** as a Master Admin and read the matrix.

Expected Result

- Step 1: **no Logs section** for any of the three.
- Step 2: each lands on a screen their scope can open — never a heading over nothing, and never the log.
- Step 3: Settings has **nine** screens and Activity Log is **not** among them; it moved to the Logs section, which is the whole of it in one place.
- Step 4: the matrix carries a **Logs** group with a read row and a clear row, ticked for Master Admin and blank for the other three — derived from the same module the routes enforce with, so it cannot claim access nobody has.

`ADM-SET-035` — Account Security · P2

Steps. Open **Settings → Account Security** and change the admin's own password. **Expected Result.** The change succeeds and the new password works. Admins are exempt from the idle timeout, so no timeout dialog appears while working here.

15.4 Configuration → dependent feature: the full matrix

Every row here is a required test. The **Verify** column is what proves the change actually landed.

#	Configuration	Where it is changed	Dependent feature	Verify	Test
1	Online booking lead time	Settings → Booking Rules	`/book` picker **and** the create-appointment validator	Earliest offered slot moves; a boundary API call returns 409	`ADM-SET-010`
2	Online cancellation refund window	Settings → Booking Rules	`/book` Step 3 notice; the cancel dialog; the refund paid	Notice text changes; a 48h-away cancel stops refunding at 72h	`ADM-SET-011`
3	Booking languages	Settings → Booking Rules	Step 1 chips; the stored `preferred_language`	New chips appear; an off-list language is dropped	`ADM-SET-012`
4	Treatment category price	Catalog → Conditions	`/book` header, Step 3 fee, the Razorpay amount	All three show the new price; an old cached page still charges the new one	`ADM-CAT-002`
5	Treatment category duration	Catalog → Conditions	The appointment's `duration_minutes`; overlap checks	A new booking stores the new duration	`ADM-CAT-002`
6	Category active/inactive	Catalog → Conditions	The concern dropdown; the create route	Absent from the dropdown; API returns 409	`PAT-BOOK-013`

#	Configuration	Where it is changed	Dependent feature	Verify	Test
7	Package definition	Catalog → Packages	The therapist's recommendation dropdown	The new package is offered for its condition only	`THR-CARE-001`
8	Package price	Catalog → Packages	The patient's offer card and what is charged	Card and charge both move — **for new plans only**	`PAT-CARE-002`
9	Package edited after purchase	Catalog → Packages	An existing purchase	**Nothing changes** — snapshot frozen	`ADM-CAT-006`
10	Package min gap / max per week	Catalog → Packages	The bulk scheduler	Violating slots are refused	`ADM-CAT-007`
11	Package default validity	Settings → Programmes & Home Visits	A new purchase's expiry	Expiry date matches	`ADM-SET-018`
12	Bulk scheduler limit	Settings → Programmes & Home Visits	`/api/appointments/book-package-sessions`	`Too many slots in one request.`	`ADM-SET-018`
13	Therapist lock switch	Settings → Programmes & Home Visits	Auto-assignment of later package sessions	Off → later sessions are not auto-assigned	`ADM-SET-018`
14	Therapist suggestions switch	Settings → Programmes & Home Visits	The suggest control and its route	Off → control absent, route 403	`THR-SUGG-001`
15	Ledger authority	Settings → Programmes & Home Visits	Six balance surfaces	All six follow together	`ADM-SET-019`
16	Service area created/deleted	Catalog → Service Areas	`/book-home-visit` check; every purchase route	Serviceable ↔ waitlist	`ADM-CAT-010`
17	Travel fee per area	Catalog → Service Areas	The quoted total and the therapist's payout	Total = programme + fee × visits	`PAT-CARE-003`
18	Home visit master switch	Settings → Programmes & Home Visits	Seven surfaces + care-plan purchase	404 / entries dropped / purchase refused	`ADM-SET-013`
19	Cash on visit	Settings → Programmes & Home Visits	Step 4 option; `book-cash`	Option absent; route refuses	`PAT-HV-007`
20	Home visit lead time	Settings → Programmes & Home Visits	The home-visit picker only	Online picker unchanged	`ADM-SET-014`
21	Travel buffer minutes	Settings → Programmes & Home Visits	The locked therapist's conflict check	Padded both sides for visits, 0 for online	`ADM-SET-014`

#	Configuration	Where it is changed	Dependent feature	Verify	Test
22	Home visit refund window	Settings → Programmes & Home Visits	The home-visit cancel dialog only	Online dialog unchanged	`PAT-CANCEL-003`
23	Join window before/after	Settings → Booking Rules	Every join control	Goes live earlier/later everywhere	`ADM-SET-017`
24	Session Completed cutoff	Settings → Booking Rules	Every join control on every role	All three read **Session Completed** together	`XR-CUTOFF-001`
25	Google Meet toggle	Settings → Booking Rules	New online sessions' Meet link	No link; **home visit still gets an event**	`ADM-SET-017`
26	Session timeout	Settings → Booking Rules	The idle dialog on all three non-admin dashboards	Signs out to the right login; admins exempt	`ADM-SET-015`
27	Farewell banner seconds	Settings → Booking Rules	The post-logout banner	Duration changes; 0 = until dismissed	`ADM-SET-016`
28	Clinical question wording	Settings → Clinical Questions	The intake wizard	New wording; old answers untouched	`ADM-SET-020`
29	Enabled condition types	Settings → Clinical Questions	The triage picker only	Removed from triage; existing charts render	`ADM-SET-020`
30	Pain Map templates	Settings → Clinical Questions	The exam dialog	New questions per region	`ADM-SET-020`
31	Admin scope	Settings → User Access	Every admin route and the sidebar	Route 403 + control hidden	`ADM-SET-027`
32	Contact scan mode	Settings → User Access	Every cross-role free-text write	block → flag → none	`THR-LEAK-006`
33	Contact masking	Settings → User Access	Therapist session cards	Masked ↔ plain; **fails closed**	`THR-SESS-003`
34	Risk signals on/off + thresholds	Today → Risk	The detector sweep	Sweep stops; thresholds change what fires	`ADM-RISK-003`
35	Brand & contact details	Settings → Brand & Contact	Navbar, Footer, page metadata, splash fallback	All update	`ADM-SET-001`
36	Walkthrough seconds	Settings → Public Site	The home page walkthrough	Pace changes; 0 = static	`ADM-SET-005`
37	Splash (5 settings)	Settings → Public Site	The opening splash	Each behaves as documented	`ADM-SET-006`
38	Testimonials	Settings → Public Site	`/` and `/mission` bands	Both update from one component	`ADM-SET-007`
39	FAQ	Settings → Public Site	`/faq`	Accordion updates	`ADM-SET-008`

#	Configuration	Where it is changed	Dependent feature	Verify	Test
40	Public ratings visibility	Settings → Public Site	`/team` , `/`	Summary hidden	`ADM-SET-004`
41	Therapist revenue share	People → Therapists	Earnings, Payouts, Money split	All three move together	`ADM-PEOP-006`
42	Therapist home-visit share	People → Therapists	Home-visit payout maths	Falls back to online share when unset	`THR-EARN-002`
43	Hospital revenue share	People → Partners	Partner Earnings, Money breakdown	Both move; unset ⇒ **excluded, not guessed**	`HOS-MONEY-002`
44	Therapist team visibility	People → Therapists	`/team` , `?therapist=` resolution	Hidden ⇒ link resolves to nothing, silently	`PAT-BOOK-008`
45	Payment gateway fee %	Settings (Costs context)	Operating profit on Money → Costs	The automatic fee line moves	`FIN-COST-002`
46	**Therapist roster (any change)**	Sessions → Roster	**`/book` picker**	**Nothing changes — this is the guard**	`XCFG-ROSTER-001`

16. Finance test plan (Money) and payment integrity

16.0 Feature guide — the money model

One word, one money figure. If a new figure needs a word that is already taken, the figure gets renamed — the word is never overloaded. If two figures end up with the same meaning, one is deleted rather than explained. `MoneyGlossary` renders on **every** Money screen, not only Summary, because an admin reading "Net payable" on Payouts is the one who needs it.

Word	Means
Gross Revenue	Every paid session in range, before refunds
Refunds	Refunds that actually processed
Net Revenue	`Gross − Refunds`
Splittable Net	The part of Net whose split is **knowable**. Always ≤ Net.
Therapist share	Earned by **delivering** — completed **and** paid only. Includes a home visit's travel fee in full.
Partner share	A commission on **net** revenue
Clinic share	`Splittable Net − Therapist share − Partner share` . A **gross** figure.
Operating profit	Clinic share **less** the gateway fee and hand-entered business expenses. The only figure that may be called profit — and only because costs exist.

Word	Means
Package cash collected	What came into the bank up front
Recognised revenue	The same money, recognised one session at a time
Owed to therapists	An all-time **balance**, net of cash held. Never date-filtered.

The two identities that must always hold:

```
net           = gross − refunds
clinic share  = splittable net − therapist share − partner share
```

Three split rules, each a correction of a real misstatement:

1. **A therapist's share is earned by delivering, not by being booked.** Only a `completed` **and** paid session adds to it. Counting every paid session deducted a share nobody would ever be paid, which understated the clinic's take on every forfeited late cancellation.
2. **A home visit's travel fee is part of the therapist's share and is never revenue.** The Money screens must be passed the payout-enriched appointments (visit mode, travel fee, the cash columns) and the per-therapist home-visit rate. Passing the plain array silently moved the whole travel bill into the clinic's share.
3. **Refunds reverse the partner's commission, not the therapist's.** A refunded session was cancelled, so it never earned a therapist share; a hospital's cut is taken on net.

Different eligibility for revenue and for the split. Gross, refunds and net count **every** paid session. A session whose split is **unknowable** — no therapist share set, or a hospital-referred patient whose hospital has no share configured — is excluded **from the split alone** and surfaced as a **named count**. **Never guess a percentage to make the numbers tie.**

Flows are range-scoped; balances are not. "Owed to therapists" is all-time and net of cash held, matching what the Pay button actually transfers. Scoping it to the range in view once let an admin read "nothing owed" off a quiet week while a real debt sat outside the window. The label has to say which it is.

16.1 The reference dataset

Build this exact dataset before running §16.2 onward. It is small enough to compute by hand and exercises every rule.

#	Patient	Therapist	Mode	Paid	Status	Refund	Notes
S1	A	A (60%)	online	₹1,999	completed	—	Ordinary delivered session
S2	A	A (60%)	online	₹1,799	confirmed (not completed)	—	Paid, **not delivered**
S3	A	A (60%)	online	₹1,999	cancelled	₹1,999 processed	Refunded outside the window
S4	C (Hospital A, 10%)	A (60%)	online	₹2,499	completed	—	Partner attribution

#	Patient	Therapist	Mode	Paid	Status	Refund	Notes
S5	C (Hospital A, 10%)	B (**no share set**)	online	₹1,999	completed	—	**Excluded from the split**
S6	A	A (home 65%)	home visit	₹2,499	completed	—	Travel fee ₹150, paid online
S7	B	A (home 65%)	home visit	₹2,499	completed	—	Travel ₹150, **cash at the door**, not remitted

Costs: the three expenses from §8.15. Gateway fee: **2%.**

Hand-computed expectations for the whole range (all figures in rupees):

- **Gross** = 1999 + 1799 + 1999 + 2499 + 1999 + 2499 + 2499 = **₹15,293** *(S7 is a cash home visit: include it only once the cash is recorded as collected — see `FIN-SUM-002`.)*
- **Refunds** = **₹1,999** (S3)
- **Net** = 15,293 − 1,999 = **₹13,294**
- **Excluded from the split**: S5 → count `1`, excluded revenue **₹1,999**
- **Splittable Net** = 13,294 − 1,999 = **₹11,295**
- **Therapist share** (completed **and** paid only):
- S1 `1999 × 60% = 1,199.40`
- S4 `2499 × 60% = 1,499.40`
- S6 `2499 × 65% + 150 = 1,774.35`
- S7 `2499 × 65% + 150 = 1,774.35`
- S2 (not completed) → **0**; S3 (cancelled) → **0**; S5 (excluded) → **0**
- **Total ≈ ₹6,247.50**
- **Partner share** (10% of **net**, hospital-referred, non-excluded): S4 only → `2499 × 10% = ₹249.90`. S5 is excluded entirely, so it contributes nothing.
- **Clinic share** = 11,295 − 6,247.50 − 249.90 = **₹4,797.60**
- **Gateway fee** (2% of **online** collections, charged on **gross**, **skipped for cash-on-visit**): online gross = 15,293 − 2,499 (S7 cash) = 12,794 → `≈ ₹255.88`
- **Expenses in a September range** = 25,000 + 4,000 = **₹29,000** (the ₹6,000 August row is **outside** and must not be counted)
- **Operating profit** = 4,797.60 − 255.88 − 29,000 = **−₹24,458.28** (a loss — which is the honest answer for this dataset)

> Rounding: each per-appointment share is rounded to the nearest paise **individually** before summing. Expect ±1 paise against a spreadsheet that rounds at the end.

16.2 Money screens

`FIN-NAV-001` — Each Money screen says what it is, and holds what it claims · P1

Steps. Open each of the five Money screens in turn: Summary, Transactions, Payouts, Costs, Breakdown. Read the line under each heading. Then open **Settings → Offers & Discounts**, read its "Looking for promo codes?" note, and follow it. **Expected Result.** Every screen prints its own one-line description plus a **For example:** line under the heading, in place of the section's own blurb — the same treatment the Settings screens get, and for a sharper reason: five screens named with abstract nouns ("Summary", "Breakdown") make an owner open three to find the one answering their question. **Promo codes are on Costs**, beside the *Discounts given* figure they produce — which is where the Offers note, the README and `ADM-PROMO-001` have always sent people. **Negative:** they used to render on **Summary**, so following that note landed on a screen with no promo codes anywhere on it and no way to tell whether the feature existed at all.

`FIN-SUM-001` — Summary and the two identities · P0

Steps. Open **Money → Summary**. Set the date range to cover the dataset. Read every figure. Compute the two identities by hand. **Expected Result.** `net = gross − refunds` and `clinic share = splittable net − therapist share − partner share` **hold exactly**. The figures match §16.1. The excluded count reads `1` with `₹1,999` named. Every screen ends with the **MoneyGlossary**. No figure is labelled "approximate" — the split is exact over a stated subset. **No figure appears twice on the screen.** The strip carries Net revenue, Operating profit, Owed to therapists and Package cash collected; the two blocks below it are the subtraction chain, where Clinic share is deliberately carried down from *Where the money went* into *What it cost to run*. Before this, Net revenue was printed twice, Clinic share three times and Operating profit twice, because the strip repeated the chain under it.

`FIN-SUM-004` — Every figure says what it means and when it is measured · P1

Steps. On **Money → Summary**, tap the **i** beside Net revenue, Clinic share and Owed to therapists. Read the chip on each. Then open the glossary at the foot of the screen and compare the sentences. Then check the **Gateway fee %** tile on Costs. **Expected Result.** The **i** expands one sentence beside the figure, and it is **word-for-word** what the glossary prints — both read `src/lib/moneyTerms.ts`. Clinic share's says in place that it is **not** profit. Chips: `These dates` on the flows, `Right now` on Owed to therapists (amber) and on the Payouts heading, `A setting` on Gateway fee %. **That tile used to be called "Payment fees"** — the same name Summary gives the rupee amount derived from it, which is the one-word-two-figures collision the vocabulary exists to prevent.

`FIN-SUM-002` — Paid vs unpaid, completed vs not · P0

Steps. Confirm S2's treatment. **Expected Result.** S2 (paid, **not** completed) **is** in Gross/Net, and contributes **nothing** to the therapist share. If the therapist share includes S2, the "earned by delivering" rule has regressed — that is a P0. **Cash check:** S7's cash home visit sits at `payment_status='unpaid'` for its whole life. **Read `payment_mode` first.** It must not be presented anywhere as a failed or outstanding online payment.

`FIN-SUM-003` — Date filtering, and what is never filtered · P0

Steps. Narrow the range to one day containing only S1. Read every figure on Summary. Then read **Owed to therapists**. **Expected Result.** Gross, refunds, net, and the split all narrow to S1. **"Owed to therapists" does not change** — it is an all-time balance, net of cash held, matching what the Pay button transfers. Its label says so. If it moves with the range, that is a P0: an admin could read "nothing owed" off a quiet week while a real debt sat outside the window.

`FIN-SUM-005` — Opening a figure, and what needs you · P1

Steps. On **Money → Summary**, tap **See the sessions** on Net revenue, then on Therapists' share, Partners' share and Clinic share. Add up the last column by hand in each. Then leave S7's cash un-remitted and a payout request pending, and open each of the five Money screens. **Expected Result.** The modal lists exactly the sessions behind that figure and its footer **equals the card**, to the rupee — both come from `moneyLineFor`, which the totals themselves accumulate. A session paid for but **not delivered** appears under Therapists' share with **nothing** against it rather than being hidden. A session left out of the split is not counted in the two share modals. Partners' share offers no link when nothing was referred in range. Every Money screen opens with a **needs-you strip**: `N things need you` over one row per item — payout requests waiting, cash a therapist is holding, refunds to hand back by hand, payments attached to nothing — each linking to the rows it counted. With nothing outstanding it reads `Nothing in Money needs you`. A **Finance** admin (no `settings`) sees the first three and **not** payments-attached-to-nothing, whose fix is on a screen they cannot open.

`FIN-SUM-006` — Exporting a figure's sessions, and last period's comparison · P1

Steps. Open **See the sessions** on Clinic share and export both CSV and PDF. Then set the range to September, read the line under **Net revenue**, and re-run with an August that has no paid sessions in it. **Expected Result.** The export covers the **same rows the modal listed** and both formats come from one column definition, so they describe the same table; the PDF's subtitle names the date range. The strip's Net revenue carries a comparison against **the same number of days immediately before** the range in view — never a calendar month against a 30-day window, which would move the figure by the number of days rather than by the business. Up is green, down is red, and a move under half a percent reads **Level with the N days before** rather than drawing an arrow over noise. With nothing in the previous period it reads **Nothing in the N days before** — it must **never** print `+100%` or `∞` from a zero baseline.

`FIN-BRK-001` — Breakdown agrees with Summary · P0

Steps. Open **Money → Breakdown** for the same range. **Expected Result.** Every figure and every chart segment matches Summary **exactly** — both come from **one pass** of the same maths. A discrepancy of any size is a P0.

`FIN-TXN-001` — Transactions · P1

Steps. Open **Money → Transactions**. Reconcile every row against the dataset. Export CSV and PDF. **Expected Result.** One row per payment, with the order id and payment id. **No payment id appears on two rows.** The CSV and the PDF describe the **same table** — they are generated from one column definition, and the PDF route is sent the exact filtered rows the browser rendered. Both cover the **whole filtered set**. The PDF carries a subtitle naming the scope and the date. **No JSON export exists.**

`FIN-PAY-001` — Payouts · P0

Steps. Open **Money → Payouts**. Read Therapist A's row. **Expected Result.** `Owed` matches §16.1's therapist share for A, **less anything already settled**. **The Pay button shows the net figure** — owed minus cash held — not the gross owed. A therapist with **no revenue share set** shows the "not set" state rather than `₹0`; paying them is refused with `Set this therapist's revenue share % before paying out.`

`FIN-PAY-002` — Settling a payout · P0

Steps. Settle Therapist A's payout. Then attempt to settle again immediately. Then double-click Settle on Therapist B. **Expected Result.** The transfer amount is recorded, `therapist_payout_paid_at` is stamped on exactly the sessions settled, and a **`payout.settle` audit row** is written **after** the compare-and-swap claim. A second attempt is a no-op or is refused — **no double payout**. The Owed figure drops to zero for the settled sessions.

`FIN-PAY-003` — Netting cash off a payout is a remittance · P0

Purpose. Without this, the same rupees are deducted again on the next payout and the Cash Ledger goes on asking someone to chase money already recovered. **Preconditions.** Therapist A is holding S7's cash (₹2,499 + ₹150 travel, per the reconstructed total), un-remitted. **Steps.** Read the Pay button's figure. Settle. Then open the **Cash Ledger** panel on the same screen. Then run a second payout cycle. **Expected Result.** The transfer is reduced by the cash held. **The same run marks exactly those visits `cash_remitted_at`.** The Cash Ledger stops listing them. The second payout cycle does **not** deduct that cash again.

`FIN-PAY-004` **— A therapist holding more cash than they are owed · P0**

Preconditions. Arrange cash held > amount owed. **Expected Result.** The transfer **floors at zero** (never negative). The difference is shown as **still owed to the business**, and **those collections deliberately stay open on the Cash Ledger** for a person to chase. They must not be silently marked remitted.

`FIN-PAY-005` **— Payout requests · P1**

Steps. With a therapist request outstanding, tap **Start review**, then **Complete**. Then try to complete a request that was never reviewed. **Expected Result.** The states move `pending → reviewing → completed`. Completing without review is refused with `Start review on this request before marking it completed.` A completed request re-submitted returns `This request is already completed.` A stale action returns `This request is no longer pending — please refresh.` All audited.

`FIN-PAY-006` **— Correcting a cash amount is the admin's job, not the therapist's · P0**

Steps. As Admin Full, correct S7's cash amount to `₹2,000` with the reason `Patient short ₹649 at the door; agreed balance next visit.` Then try with a blank reason, then on an **already remitted** visit. **Expected Result.** Valid: succeeds with a CAS on the figure being replaced, and writes a **`cash.correct_amount`** audit row. Blank reason: refused. Already remitted: **refused** — that transfer has gone out, so the fix is an adjustment against the next payout, not a silent edit of a settled one (`This session isn't a cash-on-visit home visit.` / the remitted refusal). A stale figure returns `Someone else changed this figure. Refresh and try again.` As Admin Ops (no `money` scope): **403**, and the control does not render.

`FIN-COST-001` **— Costs · P1**

Steps. Open **Money → Costs**. Add the three expenses from §8.15. Set the range to September 2026. **Expected Result.** Only the two September rows count; the 28 August row is excluded — expenses are dated by **when they were incurred**, not when they were typed in. Negatives: `Enter an amount greater than zero.`, `Pick the date this cost was incurred.` Deleting an expense removes it from the total.

`FIN-COST-002` **— Operating profit and the gateway fee · P0**

Steps. Read Operating profit. Then change the gateway fee percentage and re-read. **Expected Result.** The gateway fee is **derived automatically** from what was collected **online**, charged on **gross** (a processor keeps its fee through a refund), and **skipped for cash-on-visit**, which never touches a gateway. Operating profit = clinic share − gateway fee − expenses. Changing the percentage moves it. **With no costs recorded for a range, Operating profit is a ceiling and the screen must say so** rather than implying a number it cannot know. **Nothing here is post-tax — the label must never read "net profit".**

`FIN-REF-001` **— Refunds across the screens · P0**

Steps. Refund S3 in full. Then partially refund another session by `₹500` with the reason `Session cut short by a connection failure.` Then attempt a partial refund of `₹0`, and one with no reason. **Expected Result.** Full: Gross unchanged, Refunds +₹1,999, Net −₹1,999, **therapist share unchanged**, **partner share reduced**. Partial: the same shape at ₹500. `₹0`: `Enter a refund amount greater than zero.` No reason: `Say why this refund is being made.` A second full refund: `This session has already been refunded in full.`

`FIN-REF-002` **— Refund failure at the gateway · P1**

Steps. Force a Razorpay refund failure (use a payment that cannot be refunded in test mode). **Expected Result.** `Razorpay refused the refund. Nothing was refunded — check Razorpay and retry.` **Nothing is marked refunded locally** — the local state must never claim a refund the gateway did not make.

`FIN-REF-003` — A cash refund becomes a manual pending item · P1

Steps. Refund a cash-on-visit home purchase. **Expected Result.** With no Razorpay payment behind it, the refund becomes `refund_status='manual_pending'` and is **surfaced on the admin Cash Ledger** until an admin confirms the cash was handed back. Marking it returned clears it and is audited. Attempting to refund a cash purchase as if it were online: `Cash-on-visit packages have no single payment to refund — cancel visits individually instead.`

`FIN-REF-004` — A refund voids available credits, never delivered ones · P0

Preconditions. A 6-session purchase with 2 sessions **completed** and 4 available. **Steps.** Refund the package. **Expected Result.** The **4 available** credits are voided; the **2 delivered stay delivered**. The ledger records a `void` for exactly 4. The patient's widget shows the programme as refunded with nothing available. **A delivered session is never un-delivered.**

`FIN-REF-005` — A refunded session says so wherever it is listed · P0

Preconditions. One session refunded in full, one refunded partially, one cash home visit at `manual_pending`, one refund that failed at the gateway, and one cancelled inside the window with no refund due. **Steps.** Open **People → Patients → the patient's profile** and read the session rows. Then **Sessions → All Sessions**, then open each session's detail drawer. Export All Sessions as CSV **and** as PDF. **Expected Result.** Every one of the five carries a refund chip beside its payment chip, reading `Refunded ₹1,200` / `Refunded ₹500` / `Hand back ₹500` / `Refund failed` / `No refund due` respectively, in that wording and nothing else. A session that was never refunded carries **no chip at all** — an empty refund column reading "—" on every ordinary session is noise. A partial refund states the amount refunded, not the amount paid. The drawer adds a **Refunded** panel above the partial-refund form giving when, the reason, and the gateway reference where there is one. Both exports carry a `Refund` column and a `Refunded on` column agreeing with the chips. **A refund is as visible as a payment on every surface that lists a session** — a refund that happened and left no trace on the session is the failure this case exists to catch.

`FIN-REF-008` — A refund the clinic owes is counted, and the count opens it · P0

Preconditions. One cash home visit at `manual_pending`, one **session** at `manual_pending`, and one session whose refund `failed` at the gateway. **Steps.** As **Master Admin**, read the alerts strip at the top of any Money screen and the Today inbox. Tap each refund row. Then repeat as **Finance**, **Operations** and **Clinical**. **Expected Result.** *Refunds to hand back* reads **2** — cash visits and sessions together, because both are money a patient is owed with no card payment to reverse; counting only the visits is the bug this case exists to catch. *Refunds that failed* reads **1** and is its own row, because the work is different. Tapping it lands on **Sessions → All Sessions** filtered to exactly that one session, with every other filter cleared. Finance see the hand-back row and **not** the failed one: they read Sessions without being able to change one, so a figure nothing they could do would bring down does not belong on their screen. Operations and Clinical open no Money screen at all. Every row is **urgent** — this is money the clinic has agreed to return and has not returned.

`FIN-REF-009` — The All Sessions refund filters · P1

Steps. On **Sessions → All Sessions**, take the payment filter through **Refunded**, **Refund to hand back** and **Refund failed**. **Expected Result.** Each returns exactly the sessions in that state. **Refunded** in particular must return rows: `payment_status` is CHECKed to `unpaid` / `paid` / `failed` and can never hold `refunded`, so this option previously matched nothing and quietly returned an empty table — a filter that always looks like "no refunds have ever happened". A refund lives on `refund_status`.

`FIN-REF-006` — A refund issued before the columns existed · P1

Steps. Against a database whose `appointments` rows predate `refunded_at` / `refunded_by`, open a session refunded before the migration. **Expected Result.** The chip still reads the refund from `refund_status` and `refund_amount_paise`; **the date reads `—`** rather than guessing one, and the drawer's panel omits the "when" line. The columns are deliberately **not** backfilled — a stamped date nobody recorded is worse than an absent one. Nothing on the screen errors, and `Refunded on` is blank in both exports for that row.

`FIN-REF-007` — Finance and the desks that cannot see money · P1

Steps. As **Finance**, open a refunded patient's profile and All Sessions. Then repeat as **Operations** and as **Clinical**. **Expected Result.** Finance reads the refund chips everywhere (Money is theirs at `manage`, Sessions at `view`). Operations and Clinical see **no refund chip** on either the patient profile or All Sessions — `canSeeMoney` gates it exactly as it gates the amount paid, so a desk that cannot read what was paid cannot read what was given back either. The `paid` / `unpaid` word itself is unchanged for them: whether a session is paid for is operational, and how much is not.

16.3 Payment integrity (duplicates, concurrency, webhooks)

`PAY-DUP-001` — One payment, one appointment · P0

Steps. Complete `PAT-BOOK-003`, then reload and inspect Sessions → All Sessions and the `payments` table. **Expected Result.** Exactly one appointment, exactly one `payments` row. *_The appointment is created_ before _the order, and the order is minted against it_*, so there is no path by which one payment creates two appointments.

`PAY-DUP-002` — One order, one purchase · P0 [SQL]

Steps. Attempt to insert a second `payments` row with an existing `razorpay_order_id`. **Expected Result.** The unique index rejects it. **Do not drop that index to make an import succeed** — a collision means a duplicate already exists and wants investigating. Repeat for `razorpay_payment_id`.

`PAY-DUP-003` — One purchase, one entitlement · P0

Steps. After a care-plan purchase, trigger the verify route twice (retry the callback). **Expected Result.** Exactly one entitlement, `sessions_granted = 6`. The grant function is idempotent.

`PAY-DUP-004` — Duplicate webhook · P0

Preconditions. `RAZORPAY_WEBHOOK_SECRET` is set. **Steps.** Capture a real webhook body and its `x-razorpay-signature` header from the Razorpay dashboard's webhook log. POST it to `/api/razorpay/webhook`. POST **the identical body** again. **Expected Result.** The first is processed. The second collides on `razorpay_event_id` in `payment_webhook_events` and is treated as already-seen — **that insert is the deduplication, and it happens before any work is done**, so a retry arriving mid-flight cannot do the work twice. Money → Transactions shows the payment **once**. Gross Revenue does not double.

`PAY-WH-001` — Signature verification · P0

Steps. POST a webhook body with (a) no signature header, (b) a wrong signature, (c) the correct signature but a **re-serialised** body (`JSON.parse` then `JSON.stringify`). **Expected Result.** (a) `400 Missing signature`. (b) `400 Invalid signature`. (c) **`400 Invalid signature`** — and this is correct: the signature is checked against the **raw** body, because a re-serialised body does not round-trip byte-for-byte. **The fix for a legitimate webhook failing here is never to skip the check.**

`PAY-WH-002` — Webhook races the browser callback · P0

Steps. Complete a payment and, as close to simultaneously as you can manage, POST the webhook body and let the browser callback fire. **Expected Result.** Whichever arrives first applies the capture; the second finds it captured and **changes nothing**. The appointment is paid once, `payments` has one row, and the Meet event is created **once** (the webhook only creates one if the appointment does not already carry an event id).

`PAY-WH-003` — Non-capture events are recorded and ignored · P2

Steps. POST a `payment.failed` webhook with a valid signature. **Expected Result.** The event row is recorded and marked processed. No capture is applied.

`PAY-WH-004` — Without the webhook secret, a closed tab loses the confirmation · P0

Steps. Unset `RAZORPAY_WEBHOOK_SECRET`, restart, and POST any webhook. **Expected Result.** `503 {"error":"Webhook not configured"}`. **Consequence to state in your report if you see it in the wild:** a patient who pays and closes the tab before the callback lands leaves a **paid Razorpay order against an unpaid booking**. Restore the secret.

`PAY-DUP-005` — Double settle · P0

Covered by `FIN-PAY-002`. **No double payout.**

`PAY-DUP-006` — Partner commission is not accumulated twice · P0

Steps. Re-run the webhook and the callback for S4, then read Money → Breakdown and the hospital's Earnings. **Expected Result.** `₹249.90` appears **once** on both.

`PAY-DUP-007` — Credit idempotency keys are derived, never random · P0 [SQL]

Steps. Inspect `session_credit_ledger` for a booked package session. **Expected Result.** Keys read `reserve:<appointment_id>` and `consume:<appointment_id>`. **A random key would make every retry look like a new event, which is the exact bug the key exists to prevent.** Availability is checked **after** idempotency in the reserve function, deliberately — checking availability first would answer "no credits available" for a booking that in fact succeeded.

`PAY-CONC-001` — Twelve concurrent reserves against one credit · P0 [script]

Steps. With a purchase holding exactly **one** remaining credit, fire twelve concurrent booking requests for distinct slots. **Expected Result. Exactly one succeeds.** The rest are refused. The row lock in the reserve function is real, and the CHECK constraint on the cached counts makes an overdrawn balance impossible rather than merely unwritten.

`PAY-CONC-002` — Two admins settle the same payout at once · P0

Expected Result. One wins; the other is refused. The audit log records **only the winner** — the log write happens after the CAS claim.

`PAY-AMT-001` — The amount is server-derived · P0

Steps. Intercept the `create-order` request and change any client-supplied amount or package id. Also attempt `create-order` with another patient's `appointmentId`. **Expected Result.** The amount charged is **re-derived server-side** from the category or the catalog row. A tampered amount has no effect. Another patient's appointment is **not found** under the caller's own scoped client → `Appointment not found` (404). Attempting to pay an already-paid booking: `This booking is already paid`.

`PAY-AMT-002` **— A care-plan price cannot be tampered with · P0**

Steps. Accept a recommendation, intercepting the request to change the package id to a cheaper one. **Expected Result.** The route **re-derives the price from the plan's own recommended package** and refuses a catalog mismatch (`That recommendation is incomplete.` / a mismatch 409). The programme granted is the one recommended, at the price recommended.

---

17. Public marketing site test plan

17.0 Feature guide

The eight public pages are **one template, not eight layouts**. Every page assembles from the same design system: a hero (photo right, one headline, one sentence, up to two calls to action), a trust bar, some section bands, an "explore" strip, and a closing call to action. **Every page ends the same way on purpose** — wherever a visitor stops reading, the next step is in the same place.

The site's own index lives in **one array**, which the header nav, the footer's Explore column, the home page's connector grid and every "Where to go next" strip all read. So a page cannot exist in the header and be missing from the index, and a renamed page cannot leave a stale description behind.

Every Explore band ends on Book a session, on all eight pages, in the same full-width photo-beside-text tile — booking was on the home page's grid alone, so the six inner pages ended their index on another page to read. And **the page tiles above it square up**: the count varies (the page you are on is always missing, and Home Visit drops out when the clinic switches it off), so a row that would end short stretches its leftover tiles across it rather than leaving dead cells on the right.

`PUB-EXP-001` **— The Explore band, on every public page · P1**

Steps. Open each of `/`, `/conditions`, `/how-it-works`, `/home-visit`, `/team`, `/mission`, `/faq`, `/hospitals` and read the Explore band at the foot. **Expected Result.** Every one ends with **Book a session** → `/book`, as the last tile, full width, photo beside the text. Above it: every other page, never the one you are on, and never Home Visit while the master switch is off. **Alignment.** With the usual **seven** page tiles: two rows of three, then the seventh **stretched across the whole row** in the same wide layout — no empty cells to the right of it. Switch Home Visit off and reload: **six** tiles, two clean rows of three, nothing stretched. Narrow the window to the two-column breakpoint and repeat both: the last row must still be full. **Critical check:** this is arithmetic, not a hand-placed exception (`src/lib/exploreGridSpans.ts`, unit-tested). A tile that is full width on a tablet and half width on a desktop must **not** switch to the photo-beside-text layout — that would read as two designs rather than one stretched tile.

Word budgets are numbers, not a vibe — the rewrite exists because visitors could not tell what the site was, and the second round of feedback was that there was still too much to read:

Slot	Budget
Hero subtitle	12 words
Section lede	9 words — and dropped entirely when the heading already says it
Icon card / step / split-feature body	10 words
Bullet / check	5 words
Care-area blurb 8 words · detail 14 words	
Page blurb	8 words

Slot	Budget
Closing call-to-action body	12 words
Mission / vision sentence	15 words

Photography is load-bearing, not decoration. A visitor should be able to tell what a page is about with the text blurred out. Three rules hold:

1. **Every photograph shows a screen** — a laptop, tablet or phone in frame — **except the two home-visit images**, which show hands-on treatment. This clinic sells video consultations; a site of clinic photography reads as a walk-in practice.
2. **Every photograph shows a face, and the face is glad to be there.** The one exception is the clinician reading a scan, who is concentrating — a physiotherapist grinning at an X-ray is the opposite of reassuring.
3. Photos are **static imports**, never `/photos/x.jpg` strings and never remote URLs, so a missing file is a compile error.

`PUB-CTA-001` — The closing band asks for the sale · P1

Steps. Open each of `/`, `/conditions`, `/how-it-works`, `/home-visit`, `/team`, `/mission`, `/faq` and read the very last band. (`/hospitals` deliberately ends on its referral form instead.) **Expected Result.** A teal band with the copy on the left — headline, one line, one or two buttons — and a **photograph on the right**. The band's shape is identical on all seven, but **each page shows a different photograph**, and it is one this band alone uses — never a photograph that appears anywhere else on the site: home a patient booking on her phone with a laptop on her knees, Conditions Treated a patient laughing as she books, How It Works a patient waving as his video session opens, Home Visit an older couple booking on a tablet in their front room, Our Team a physiotherapist smiling at her laptop, Mission two people on a sofa with a session on a laptop, FAQ a patient reading her phone by a window. Over the photo, a white chip reading **Session confirmed / Tuesday, 6:00 PM / Calendar invite on its way.** and, underneath it, **Example of what you get** — the chip must always say it is an example, on every page. **Assurances.** Three lines under the buttons: *One-to-one, never a group*, *Reports read before your session*, *Secure UPI payment*. **Critical check:** none of those three states a **number**. A session's length is set per treatment category and the cancellation window is an admin setting, so a fixed "60 minutes" or "free cancellation up to 24 hours" printed here would be a promise the settings can contradict. If you see a number in this band, that is a bug. **Layout.** Narrow to a phone width: the copy and buttons come **first**, the photograph below them — nobody should have to scroll past a picture to reach the button. The image keeps the same landscape crop at every width; it must never go tall and leave the band mostly empty beside the text.

`PUB-HOME-001` — The home page · P1

Steps. Open `/`. Scroll to the bottom. **Expected Result.** Hero → trust bar → care-area showcase → walkthrough → programmes → testimonials → mission band → connector grid. The **mission band gives the mission and vision in full** (they are two sentences; paraphrasing would make the home page a weaker version of the same claim) while the **four promises appear as titles only**, each linking to the mission page's promises anchor. The connector grid shows the other seven pages **plus booking** — the index of the site always ends on the one action the site exists for.

`PUB-COND-001` — Care areas show one photograph at a time · P1

Steps. On `/` and `/conditions`, use the care-area showcase: swipe, the arrow buttons, and the picker. **Expected Result. One panel at a time** — photograph left, the answer right, the other five one tap away. All three controls go through one selection path so they cannot disagree about what is showing. The picker is a real tablist with **roving focus and arrow keys**. **It never advances by itself** — the home page already carries the auto-rotating walkthrough, and two moving things is worse than either alone. Its accessible name is **"Areas of practice"**, distinct from the walkthrough's **"How the process works"**.

`PUB-NAV-001` — The section rail and the scroll arrow · P1

Steps. On each public page, use the section rail's entries and then the bottom-right scroll arrow repeatedly. **Expected Result.** Every rail entry corresponds to a section that **actually rendered** (several bands are conditional on admin-controlled catalog data), and the entries are in **DOM order**. The arrow walks the list **top to bottom** — if it ever sends you backwards, an entry is out of order.

`PUB-CAT-001` — A catalog card opens a dialog; booking is its own button · P1

Steps. On `/` and `/conditions`, tap a programme (treatment category) card's **body**. Then tap its **Book …** link. **Expected Result.** The card body is **one tap target that opens a detail dialog** carrying the long description and what the programme covers. The **Book …** link sits **below the card** and again at the foot of the dialog, **outside** the tap-target button (a link nested inside a button is invalid markup and behaves differently per browser). It goes to `/book?category=<id>` — a **first session**, never a course of them. **Cover images:** a card with no photo shows the **shared placeholder panel at the same height** as one with a photo. It must never look like an image that failed to load.

`PUB-CAT-002` — No course of treatment is advertised anywhere public · P0

Feature. A course of treatment is a clinical recommendation, so the public site does not carry a price list of them. Removed outright rather than hidden behind a setting: a toggle somebody can flip back on is not the rule being gone.

Preconditions. Packages P1–P3 exist, are `active`, and have `visible_on_home` and `visible_on_conditions` on. That matters — this is an absence tested against rows that genuinely could have rendered.

Steps. Read `/` and `/conditions` end to end, including inside every programme detail dialog. Then read `/home-visit`. **Expected Result**

- **No session package appears anywhere** — no card, no title, no price, and no "Where this usually leads" list inside a programme's dialog.
- **No link anywhere matches `/book?package=`.** A card still linking to package checkout would take money for something the server now refuses; this is the assertion that catches a partial revert.
- `/home-visit` shows **single**-visit packages only. `HV1` (1 visit) is there and books directly; `HV2` (4 visits) is **absent** — it is a recommendation, not a product.
- There is **no admin switch** for any of this. "Show programme prices publicly" no longer exists on Settings; if you find it, that is a defect.

`PUB-TEAM-001` — Team · P2

Expected Result. Only approved, active, team-visible therapists appear. Tapping one opens a popup with their profile and a **Book with …** action carrying `?therapist=` into the wizard.

`PUB-FAQ-001` — FAQ · P2

Expected Result. The accordion renders the admin-managed FAQs in order. With none configured, the page shows a sensible empty state rather than a broken band.

`PUB-HV-001` — Home visit page · P1

Covered by `PAT-HV-001` and `ADM-SET-013`.

---

18. Security and authorization test plan

> **The rule for this whole section: do not only verify that the UI hides something.** Every case has a route-level twin. The application enforces its gates in two places — the proxy for navigation, and `requireActiveProfile` / `requireAdmin` / `requireAdminScope` inside the routes — because a valid session cookie can call the API around the UI.

How to call a route as a given user. Sign in as that user in a browser, copy the session cookie from DevTools → Application → Cookies, and use it with curl:

```
curl -i -X POST http://localhost:3000/api/<route> \
  -H 'Content-Type: application/json' \
  -b '<paste the cookie header>' \
  -d '{ ...body... }'
```

18.1 Signed-out access

`SEC-ROUTE-001` — Every protected route redirects a signed-out visitor · P0

Steps. In a private window, navigate to `/patient/dashboard`, `/patient/dashboard/health-profile`, `/therapist/dashboard`, `/therapist/dashboard/earnings`, `/hospital/dashboard`, `/hospital/dashboard/revenue`, `/admin/dashboard`, `/admin/dashboard/patients/<id>`. **Expected Result.** Each redirects to that role's own login page. **No protected content is rendered even for a frame.**

`SEC-ROUTE-002` — Every mutating route refuses an anonymous caller · P0

Steps. Call a representative route from each family with **no cookie**: `/api/appointments/create`, `/api/appointments/cancel`, `/api/patient/condition-profile/submit`, `/api/therapist/save-availability`, `/api/therapist/care-plan/submit`, `/api/hospital/withdraw-referral`, `/api/admin/approve-account`, `/api/admin/settle-therapist-payout`, `/api/medical-documents/view`, `/api/razorpay/create-order`. **Expected Result. 401 \`Not signed in\`** or **403 \`Forbidden\`** on every one. **None returns 200, and none leaks data in the error body.**

18.2 Cross-role access

`SEC-ROUTE-003` — A signed-in user of the wrong role is sent to Get Started · P0

Steps. As each of patient / therapist / hospital, navigate to each of the other three dashboards. **Expected Result.** Every one redirects to **\`/get-started\`**.

`SEC-ROUTE-004` — The back office is never named to an outsider · P0

Steps. As a signed-in **patient**, navigate to `/admin/dashboard`. Then view the page source of `/`, `/patient/dashboard` and `/team`, and search for `/admin/login` and `/admin/dashboard`. **Expected Result.** The navigation redirects to **\`/get-started\`**, never to `/admin/login` — which would confirm the back office exists and name its door. **No admin path appears in any public client bundle**: the role→dashboard mapping is resolved **server-side** at `/dashboard`, and `Navbar` and the wrong-account panel link to `/dashboard`. **The Debug Bar is the deliberate exception** — it still lists the admin routes, and is deleted before release. If you find an admin path in a public bundle **outside** the debug bar, that is a P0.

`SEC-ROUTE-005` — `/admin/login` is not indexed · P2

Expected Result. The page carries `robots: noindex`.

`SEC-ROUTE-006` — `/dashboard` routes by role, server-side · P1

Steps. As each role, open `/dashboard`. Then open `/dashboard?hash=<something>` and `/dashboard?hash=//evil.example.com`. **Expected Result.** Each role lands on their own dashboard. A legitimate `hash` becomes a real fragment (the anchor-based shells need it) and is **pattern-checked**; a value that could smuggle a host is rejected. **No open redirect.**

`SEC-ROUTE-008` — The booking wizard's exit follows the account · P1

Feature. `/book` and `/book-home-visit` hide the site nav so a stray link cannot lose somebody's progress mid-payment, which makes the one link at the foot the only way out. It always read *Back to Home* — right for a visitor from the marketing site, wrong for the commonest case: a patient who came from their own dashboard to book, and was sent to the public home page instead of back where they started.

Steps

1. Signed out, open `/book` and read the link at the foot. Then `/book-home-visit`.
2. Sign in as an **approved patient**, go to the dashboard, start a booking, and read it again on both.
3. Tap it.
4. Sign in as a patient who is **not yet approved** (a fresh self-signup, before paying) and read it.
5. Tap it.
6. Open `/book` on a slow connection and watch the link before the session resolves.
7. Sign in as an admin or a therapist and open `/book`.

Expected Result

- Step 1: **Back to Home** → `/`, and **Back to Home Visit** → `/home-visit`. Each wizard keeps its own signed-out destination.
- Step 2: **Back to Dashboard**, on both.
- Step 3: lands on their dashboard, not the marketing site.
- Step 4: **Approval pending** — not *Back to Dashboard*. An unapproved patient bounces off their dashboard to the waiting screen, and a label naming a destination they cannot reach is the failure this resolution exists to avoid.
- Step 5: lands on `/pending-approval` directly, in one navigation.
- Step 6: **Back to Home** until the session resolves, then the signed-in wording. Fail-closed — never a dashboard link offered to somebody who might not have one.
- Step 7: the wizard shows the wrong-account card (one account carries one role), and the exit still reads **Back to Dashboard** — it is about the account, not about who may book.
- **Critical check:** both pages must still build as **prerendered** (`○`, 5-minute ISR). Resolving this server-side would force every booking page dynamic to answer a question about one link.

`SEC-ROUTE-007` — The public nav always offers a signed-in person somewhere · P1

Feature. The Navbar hides **Sign In** and **Get Started** once somebody is signed in, so whatever replaces them is the only route back into the app from the marketing site. An account waiting on approval used to get **neither** — both CTAs gone because they are signed in, and no button in their place — so an unapproved patient who tapped Home had no way forward at all.

Steps

1. Register a patient and leave them **unapproved**. From `/pending-approval`, tap the logo to reach Home and read the top right.
2. Tap the button.

3. Approve them from the back office, reload Home, and read it again.
4. Suspend an account (`active = false`), open Home as them, and read it again.
5. Look at the top right **on `/pending-approval` itself**.
6. Open Home signed out.
7. Open Home on a slow connection and watch the top right before the role lookup resolves.
8. Repeat steps 1–4 on a phone width, in the mobile drawer.

Expected Result

- Step 1: a button reading **Approval pending** — not an empty space, and not *Go to Dashboard*, which would name a destination they cannot reach.
- Step 2: lands on **`/pending-approval`** directly — one navigation, not a bounce through `/dashboard` and their role's dashboard.
- Step 3: the same button now reads **Go to Dashboard** and opens `/dashboard`.
- Step 4: **Account suspended**, opening `/account-suspended`. An account that is both suspended and unapproved reads *suspended* — that is the one that decides where they land.
- Step 5: **no button at all.** Those two pages are in `AUTH_CTA_HIDDEN_ROUTES`, so the nav never offers a link to the page it is already on.
- Step 6: **Sign In** and **Get Started**, as before.
- Step 7: **nothing** until the lookup resolves — fail-closed, so a slow or failed role read never briefly offers the wrong destination.
- Step 8: identical in the drawer.

18.3 Horizontal privilege (one user reaching another's data)

`SEC-DATA-001` — Patient A cannot read Patient B · P0

Steps. As Patient A: (a) call `/api/packages/purchase-detail` with Patient B's purchase id; (b) call `/api/home-visit/purchase-detail` likewise; (c) call `/api/appointments/cancel` with Patient B's appointment id; (d) call `/api/medical-documents/view` with Patient B's document id; (e) call `/api/patient/condition-profile/export` and check whose record comes back. **Expected Result.** (a)–(d) all refused. The purchase-detail routes query with **the caller's own RLS-scoped client**, so **a row coming back at all is the authorization check** — there is deliberately no manual ownership branch duplicating what the policies guarantee. (e) returns **only Patient A's** record.

`SEC-DATA-002` — Therapist A cannot reach Therapist B's data · P0

Covered by `THR-SEC-001`, `THR-SEC-002`. Additionally: `/api/therapist/record-cash-collection` with Therapist B's appointment id → refused; `/api/therapist/session-notes/submit` for Therapist B's session → refused; `/api/therapist/suggest-session` on a programme locked to Therapist B → `That isn't your programme.`

`SEC-DATA-003` — Hospital A cannot reach Hospital B · P0

Covered by `HOS-SEC-001`.

`SEC-DATA-004` — Private documents · P0

Steps. Upload a report as Patient A. Copy the signed URL. (a) Open it in a private window **within** 120 seconds. (b) Open it **after** 120 seconds. (c) Try to guess/construct a direct Storage object URL for the `medical-reports` bucket. (d) As Therapist A (assigned), view it. (e) As Therapist B (not assigned), request it. **Expected Result.** (a) opens — a signed URL is a bearer token by design, which is why it is short-lived.

(b) **fails**. (c) **fails — the bucket is private**, unlike `avatars`. A scan report is the most sensitive thing this application holds, and a public bucket would make the object URL itself the only secret. (d) allowed. (e) refused. **Also confirm:** `patient_medical_documents` has **no bytea/base64 column** — a handful of MRI PDFs stored inline would dominate the database's size and ride along on every read of a patient's chart.

`SEC-DATA-005` — Contact information exposure · P0

Covered by `THR-SESS-003` (masking, and the plaintext number absent from the page source) and `THR-SESS-004` (the reveal log). Additionally: check the **admin export** of patients — a masked field must mask the same way there, and no export anywhere emits JSON.

18.4 Vertical privilege (scope)

`SEC-ADMIN-001` — Scoped admins · P0

Covered by `ADM-SET-027` and `ADM-SET-028`. Run the whole table.

`SEC-ADMIN-002` — A non-admin cannot call an admin route · P0

Steps. With **each** of a patient's, a therapist's and a hospital's cookie, call ten admin routes across different sections. **Expected Result. 403 on every one.**

`SEC-ADMIN-003` — Scope changes are self-protecting · P0

Covered by `ADM-SET-025`: no self-change, and the last `full` admin cannot be narrowed.

`SEC-AUTH-006` — A suspended account is locked out everywhere · P0

Steps. Suspend Patient A, Therapist A and Hospital A in turn. With each still-valid cookie: load the dashboard, then call a self-service route. **Expected Result.** Dashboard → **`/account-suspended`**. Route → **403** (`Your account has been suspended.` / `Your account is not active.`). **A live cookie must not outlive suspension. Contrast:** an **admin** is refused on `active` but **deliberately not on `approved`** — an admin is promoted by hand rather than through the signup queue, so gating on approval would lock out the people it protects. Verify an admin with `approved=false` **can still sign in**.

18.5 Input tampering

`SEC-TAMPER-001` — Manipulated ids · P0

Steps. For ten routes, substitute another user's id, a random UUID, an empty string, `null`, and a non-UUID string. **Expected Result.** Each returns a **readable 400/403/404**. **No 500 with a database message.** No response body contains a table name, a column name, a row id belonging to somebody else, or a stack trace.

`SEC-TAMPER-002` — Manipulated prices · P0

Covered by `PAY-AMT-001` and `PAY-AMT-002`.

`SEC-TAMPER-003` — Manipulated session counts · P0

Steps. Attempt to book more package sessions than remain by sending extra slots; attempt to set `sessions_granted` through any route. **Expected Result.** Refused. **There is no route that accepts a session count from the client.** The count comes from the frozen snapshot.

`SEC-TAMPER-004` **— Manipulated permissions · P0**

Steps. As a patient, send `{"role":"admin"}` and `{"admin_scope":"full"}` in the body of a profile-update route. As a scoped admin, call `set-admin-scope` on yourself. **Expected Result.** All refused or ignored. **Never trust a role, an id, or an amount sent from the client — it is re-derived server-side.**

`SEC-TAMPER-005` **— Malformed bodies · P1**

Steps. POST invalid JSON, an empty body, a deeply nested object, and a 5 MB string field to ten routes. **Expected Result.** `Malformed payload` or a specific field message, always **4xx**, never a 500 and never a crash.

`SEC-TAMPER-006` **— Duplicate submissions · P0**

Covered by `PAT-BOOK-017`, `PAT-SUGG-004`, `THR-AVAIL-004`, `FIN-PAY-002`, `PAY-DUP-004`.

18.6 Authentication behaviour

`SEC-AUTH-004` **— No email-confirmation step exists · P0**

Steps. Register through **all four** sign-up call sites: `/patient/register`, the `/book` wizard, the `/book-home-visit` wizard, and `/therapist/login` → Apply to Join. **Expected Result.** Every one returns a session immediately. **None shows a "check your email" instruction.** If a sign-up returns no session, the app reports it as a **failure** (`Your account was created but we couldn't sign you in to finish this booking. Please sign in and try again.`) — because that state means the Supabase project has *Confirm email* switched on, which is a misconfiguration, not a step.

`SEC-AUTH-005` **— Session expiry · P2**

Steps. Invalidate the session server-side (sign out elsewhere / delete the cookie) and then submit a form mid-flow. **Expected Result.** A readable message such as `Your session expired. Please refresh the page and try again.` — not a raw 401 body, and not a silent no-op.

`SEC-AUTH-007` **— A display code outlives the role that generated it · P2 [SQL]**

Purpose. Every self-signup is inserted as a patient, so an account later promoted to admin or hospital **keeps its `PT####`**. **Steps.** Promote a patient to hospital, then register a new patient. **Expected Result.** The new signup succeeds. The uniqueness of codes is scoped to the **column**, not the role, and the sequence resync takes its max over **every** non-null code regardless of role. **A signup failing intermittently with a 500 from `auth.signUp` and an empty body is the symptom of this being broken.**

---

19. UX, responsive and accessibility test plan

19.1 Viewports to test

Device class	Viewport	Coverage
Desktop	1440 × 900	Everything
Tablet	820 × 1180	Booking, patient dashboard, admin All Sessions
Mobile	**390 × 844**	**The full patient booking + payment flow, mandatory**

19.2 Mobile booking flow

`UX-MOB-001` — The critical path at 390 × 844 · P0

Steps. At 390 × 844, execute `PAT-BOOK-002` → `PAT-BOOK-003` end to end, including Razorpay checkout.

Expected Result — check every one of these:

- **No horizontal scrolling on any screen.** The page body never scrolls sideways. Wide content (tables, the calendar) scrolls **inside its own container**, not by moving the page.
- The Step 1 **calendar is legible and its day cells are tappable** — this is the control most at risk, because the calendar is a 7-column grid squeezed directly by the card's padding. Padding is deliberately reduced on phones for exactly this reason; each day cell must remain a comfortable tap target (≈44 px).
- Hour chips and language chips wrap rather than overflow.
- All text is readable without zooming; nothing is clipped mid-word.
- **The on-screen keyboard does not hide the field being typed into, nor the primary button.** Tap each Step 2 field in turn and confirm you can still see what you are typing and can reach **Review Booking →**.
- The Step 3 summary rows do not collide; the fee is fully visible.
- The **Razorpay checkout renders and completes** at this width.
- The **Payment Confirmed** panel and its **Go to Dashboard** button are fully visible.

`UX-MOB-002` — Dashboards on mobile · P1

Steps. At 390 × 844, open each of the four dashboards.

`UX-BUSY-001` — The app says it is working · P1

Steps. Tap anything that saves, creates or navigates — a Save on a settings form, **Done** on a session, a payout, a sidebar entry — and watch the very top of the viewport. **Expected Result.** A thin teal bar appears across the top for the whole time between the tap and the page answering, then completes and fades. It covers the part a button cannot: after `router.refresh()` the button is idle (and often unmounted with its row), while the server is still re-rendering — the gap that reads as a freeze. **Details that are the design, not decoration:**

- **No percentage.** The bar eases toward a ceiling and only completes when the work lands. A bar sitting at 90% is a defect, not a slow server.
- **It waits ~220ms before drawing.** An action that finishes faster must show **nothing at all** — a flash on every tap is the failure this delay prevents.
- **Reduced motion** (`prefers-reduced-motion: reduce`) keeps the bar and drops the travel: a static full-width band that fades. It must not disappear entirely — that setting means less movement, not less information.
- **Two overlapping actions**: start a second before the first finishes; the bar must stay up until **both** are done, not vanish with the first.
- The five money buttons (**Done**, **Collect ₹…**, **Request Payout**, **Confirm … Payment**, **Assign & Confirm**) show a spinning ring beside their busy label rather than only swapping the text.

`UX-TIME-001` — Every time on screen is clinic time · P0

Feature. A date formatted without an explicit timezone renders in whatever zone the *runtime* is in — the host's on the server (UTC), the viewer's in the browser. A 6 PM IST session therefore printed as **12:30 PM** on the patient's own Overview. Every date the app renders is now pinned to `Asia/Kolkata`.

Steps

1. Book a session for **6:00 PM IST**. As that patient, open the dashboard **Overview** and read the **Next session** figure and the line under it.
2. Read the same session on **Your Sessions**, on the therapist's dashboard, on the admin's All Sessions and in the session drawer.
3. Change your **device** timezone to something far from IST (London, New York) and reload every one of those screens.
4. Book a session at **00:30 IST** — the instant that is the *previous* day in UTC — and check the date shown everywhere.
5. Open a **purchase detail** modal and a **payment receipt**, and read the purchased/expiry dates.
6. On the admin **Calendar**, read the month title and the selected-day title at a non-IST device timezone.
7. On the patient's **Book a Session** calendar, page through months at a non-IST device timezone.
8. Open the health-profile intake wizard, type, and read the **Saved …** line.

Expected Result

- Steps 1–2: **6:00 pm**, everywhere, and the same date on every surface. If any screen says 12:30, that is this defect.
- Step 3: **unchanged.** The zone is the clinic's, not the reader's — two people looking at one session must not disagree about when it is.
- Step 4: the **IST date**, on every surface. This is the case a naive formatter gets a full day wrong.
- Step 5: clinic time, and anything unreadable renders as **—**, never `Invalid Date`.
- Steps 6–7: the month and day titles are **correct at every device timezone** and must not shift by a day. These are wall-clock dates with no instant behind them, and are deliberately *not* pinned — pinning them is the opposite bug.
- Step 8: **"Saved 3:42 pm" follows your own clock**, deliberately: it is your own draft, set in your browser, gone on reload — not a stamp on a record anyone else reads.
- A session slot keeps being shown in the zone the patient **booked** it in, where the booking recorded one.

`UX-SAID-001` — Every change says what it was · P0

Feature. A mutating control ends with `router.refresh()`, which re-renders the screen into a state that looks identical to the one before it. Turning home visits on left the admin looking at the same page with no idea whether it had worked. Every control now raises a confirmation naming the thing and its new state.

Steps

1. **Settings → Programmes & Home Visits**: toggle **Home Visit enabled** off, then on. Read the message each time.
2. Change **online booking lead time** to 12, then to 1. Read both.
3. Change an **invite reward** amount and read the message.
4. Set the **home page walkthrough** rotation to 0, and the **splash revisit** minutes to 0.
5. Disconnect the network and toggle any setting.
6. Watch the message while the page re-renders behind it — do not click anything.
7. Toggle the same switch three times quickly.
8. Wait without touching it. Then raise another and press its **×**.
9. As an **admin**: suspend a colleague, then change their access level.
10. As a **patient**: save an address, upload a report, delete it.

11. As a **therapist**: write a session note, mark a session complete, record a cash collection, request a payout.
12. As a **hospital**: withdraw a referral.
13. On a phone width, raise any of them.

Expected Result

- Step 1: **"Home visits are off. The public page and the booking wizard are hidden."** then **"Home visits are on…"**. Never *Saved* or *Updated successfully* — a confirmation that does not name the thing says only that a request finished, which was already visible.
- Step 2: **"12 hours"** and **"1 hour"** — the unit is spelled out and pluralised on its own count.
- Step 3: reads **₹** and a rupee figure. The word *paise* must never appear: it is a storage unit.
- Step 4: both zeroes read as words — *"no longer rotates by itself"* and *"shows on a first load only"* — never "every 0 seconds".
- Step 5: a **red** message saying nothing was saved, and the toggle **snaps back** to where it was. A switch left showing a value the server refused is the worst outcome here.
- Step 6: the message **survives the refresh**. It is mounted above every route, so the tree re-renders underneath it.
- Step 7: **one** message on screen, not three — the same sentence replaces itself rather than stacking. Flipping a switch twice is one fact.
- Step 8: it clears itself after a few seconds (errors stay longer, since they need acting on), and × dismisses immediately.
- Steps 9–12: each names what happened and to whom — *"Asha Rao can no longer sign in."*, *"Asha Rao now has Finance access."*, *"Report uploaded."*, *"Session note saved."*, *"Payout requested. The clinic will review it."*, *"Referral withdrawn."* **All four dashboards, not only the admin's.**
- Step 13: full width at the bottom, with the dismiss in thumb reach, and never covering the control that was just used.

`UX-BUSY-003` — Nothing waits in silence, on any dashboard · P0

Feature. The teal bar used to appear on the admin dashboard and nowhere else, which read as three dashboards with no loading state at all. The cause was not the bar: `useRouter` reports every navigation the app starts *in code*, and the patient, therapist and hospital dashboards move between sections with **plain anchors** — a hard browser navigation, which never touches the router hook. Nothing in React learned a navigation had started.

Steps

1. As a **patient**, click each sidebar entry in turn — Sessions, Packages, Payments, Health Profile, Book, Edit Profile — and watch the **top edge of the window** the moment you click, before the new page arrives.
2. Repeat as a **therapist** (Availability, Earnings, My Patients, Sessions, Edit Profile) and a **hospital** (Refer, Your Referrals, Earnings, Edit Profile).
3. Watch what the **new** page paints first, before its data arrives.
4. Click the entry for the screen you are **already on**.
5. Click an in-page anchor (Edit Profile's own sub-sections).
6. Click **Back to Home** from any of the four dashboards.
7. Go somewhere, then press the browser's **Back** button, and look at the top edge of the restored page.
8. On the public site, click between Home, Conditions, How It Works and FAQ.
9. Throttle the network to Slow 3G and repeat steps 1 and 3.

Expected Result

- Steps 1–2: the **teal bar appears at the top of the page you are leaving** and stays up until the new screen arrives. No sidebar entry on any dashboard may leave the screen silent.
- Step 3: a **skeleton in the shape of the page** — sidebar rail in place, heading, figure tiles, cards — not a white gap and not a blank content area. Every dashboard sub-route has its own boundary now; before, only the four dashboard roots did.
- Step 4: **no bar.** Re-navigating to the screen you are on finishes instantly, and a bar for it is the flicker the 220ms delay exists to prevent.
- Step 5: **no bar** — an in-page scroll is not a navigation.
- Step 6: the bar shows on the way out to the public site, from all four.
- Step 7: **no stuck bar** on the restored page. A back/forward-cache restore brings the page back exactly as it was, bar included, so it is cleared on `pageshow`.
- Step 8: the bar runs for public navigations too.
- Step 9: both are obvious and neither screen is ever static and unexplained — this is the case the whole mechanism exists for.

`UX-REFRESH-001` — One Refresh button, on all four dashboards · P1

Feature. Every dashboard carries the same Refresh control in its header. These are Server Components, and the only automatic updates come from the realtime channels — which cover subscribed tables only, and hold a 30-second cooldown on the catalog one. Before this there was no way to ask for the page again except a browser reload, which throws away the state the shells keep on purpose and re-downloads the bundle.

Steps

1. Open the patient, therapist, hospital and admin dashboards in turn and find the button in the header.
2. On the admin dashboard, collapse the sidebar, open a screen with a half-typed filter, then tap **Refresh**.
3. Watch the button and the top of the page while it runs.
4. Tap it repeatedly, as fast as you can.
5. Have a second person change something (assign a session, add a cost) and tap Refresh without reloading.
6. Compare against a browser reload (F5) on the same screen.
7. Check it at a phone width on all four.

Expected Result

- Step 1: present on **all four**, same place, same label.
- Step 2: the data is current and **the collapsed sidebar and the typed filter survive** — this re-runs the server render, it does not reload the page.
- Step 3: the icon spins and the label reads **Refreshing…**, and the **teal progress bar** runs across the top. The button stays busy for the whole refresh: this is the one control whose own work *is* the refresh, so releasing it early would leave it looking idle while the page was still thinking.
- Step 4: **one refresh at a time.** The button is disabled while pending — stacking them on the admin dashboard stacks around forty queries a tap for no new answer.
- Step 5: the other person's change appears.
- Step 6: the reload loses the sidebar state and the filter; Refresh does not. That difference is the point of the control.
- Step 7: visible and tappable on a phone, in the header rather than the sidebar — the sidebar is a closed drawer at that width.

`UX-BUSY-002` — The admin dashboard's second batch · P2

Steps. Load **/admin/dashboard**, then tap any button that saves. **Expected Result.** Noticeably quicker than before: eleven migration-dependent reads that used to run one after another (accounting health, the suggestion and recommendation switches, discounts, the first-session offer, promo and invite settings, category covers and condition types, testimonial avatars, hospital notes) now run as one parallel batch, and both ledger-balance passes run together. **Critical check — the isolation is unchanged:** drop one of those columns (see `admin-degraded-schema.spec.ts`) and only that panel degrades. If the whole dashboard blanks, the batch has lost a `guard()` and that is a P0.

Expected Result. Each shell offers a **mobile drawer** for the sidebar. **Back to Home is present in all three renders** — expanded sidebar, collapsed rail, and mobile drawer. On all four shells it sits at the **foot of the nav, directly above Collapse** (above the profile/Log Out footer in the mobile drawer, which has no Collapse). Without it the only exit from a dashboard is Log Out, which also ends the session. It is a plain link, not a client-side transition, because transitions into a differently-chromed route were silently not completing.

`UX-MODAL-002` — A dialog opened inside a modal covers the screen · P1

Feature. `position: fixed` is relative to the viewport until an ancestor carries `backdrop-filter` — and every modal here sets `backdrop-blur-sm`, so every modal is one. A confirmation opened inside one was therefore measured against that modal's scrolling panel instead of the screen.

Steps

1. **People → Patients → a patient → Profile.** In their sessions list, press **Mark Done** on a session.
2. Look at the dark sheet: how much of the screen does it cover?
3. Scroll the page while the prompt is open.
4. Press **No**, scroll the profile **half way down**, and press Mark Done again.
5. Repeat from the **session drawer** on All Sessions, and from a **purchase detail** modal.
6. Do the same on a phone width.
7. Press **No**, then **Yes**, and confirm the action still works and the confirmation toast appears.
8. On the patient's own dashboard, open a **programme detail** and the **bulk scheduler**, and check their open/close animation still plays.

Expected Result

- Steps 1–2: the prompt is **centred in the viewport** and the dark sheet covers the **whole screen**, not a band of it.
- Step 3: it **stays put** as the page scrolls behind it — it follows the reader rather than the document.
- Step 4: identical wherever the page is scrolled to. Previously it appeared at the top of the scrolled content, often off-screen.
- Steps 5–6: identical from every modal and at every width.
- Step 7: **unchanged behaviour.** The dialog moved in the DOM but not in the React tree, so clicks, cancel and confirm all work exactly as before.
- Step 8: the animated overlays still fade in **and out**. They are deliberately not portalled — a portal between them and their animation wrapper would kill the exit animation, and they are always outermost anyway.

`UX-MOB-003` — Modals and drawers fit · P1

Steps. At 390 × 844 open: a catalog detail dialog, the admin session drawer, the intake wizard, the pain-exam dialog, the confirm dialog. **Expected Result.** Each fits the viewport, scrolls internally if it must, and its close control is reachable without scrolling.

`UX-MOB-004` **— The body map on a phone · P1**

Expected Result. Front and back views **stack** rather than shrinking each tap target below a fingertip. Every region is tappable.

`UX-MOB-005` **— Admin tables on mobile · P2**

Expected Result. Wide tables scroll inside an `overflow-x` container. The page itself never scrolls horizontally. The pager remains usable.

19.3 Accessibility pass

`UX-A11Y-001` **— Keyboard navigation of the booking flow · P1**

Steps. With the mouse untouched, complete Step 1 and Step 2 using only Tab, Shift+Tab, arrows, Space and Enter. **Expected Result.** Every interactive control is reachable in a sensible order. **A visible focus ring is present on every focused control.** The calendar and the chip groups are operable from the keyboard. No control is reachable only by pointer.

`UX-A11Y-002` **— Accessible names · P1**

Steps. With a screen reader or the accessibility inspector, walk the booking wizard, the roster editor and the admin sidebar. **Expected Result.** Every button has a name that says what it does (not "button" or an icon name). The roster's controls are labelled per day and per period (e.g. "Remove Monday period 1", "Set Monday to …"). Photo `alt` text **describes the picture**, not the page — the blurb already says what the page is for, and a screen reader announcing the same sentence twice tells someone nothing about the image.

`UX-A11Y-003` **— Dialog focus · P1**

Expected Result. Opening a dialog moves focus into it; Tab is trapped inside; Escape closes it; focus returns to the control that opened it.

`UX-A11Y-004` **— Form errors are announced and visible · P1**

Expected Result. A validation failure moves focus to or announces the message, and the message is visible without scrolling on a 390-wide screen.

`UX-A11Y-005` **— Two tablists must not share a name · P2**

Expected Result. The care-area picker is "Areas of practice"; the walkthrough is "How the process works". **Two tablists sharing a name makes both unfindable.**

`UX-A11Y-006` **— Reduced motion · P1**

Steps. Enable "reduce motion" in the OS, then load `/`. **Expected Result.** The splash is **skipped outright** — it is decoration over content that is already rendered, so the honest answer to "don't animate" is not to show it. The walkthrough does not auto-rotate distractingly.

`UX-THEME-001` **— No hydration warnings · P1**

Steps. With the console open, load `/`, `/book`, and each dashboard. **Expected Result. No hydration mismatch warnings.** The splash's visibility is an attribute on `<html>` read by CSS, not React state, precisely to avoid this. `suppressHydrationWarning` covers that one element only — if you see it on a descendant, raise it.

20. Error, loading and empty-state testing

The rule that applies to every case in this section: an internal database error, a column name, a row id or a stack trace **must never reach the screen**. Every route tree has an error boundary, and only the framework's opaque `digest` is shown — because an error message can carry a column name or a row id, and **patients see these screens**.

`ERR-BOUNDARY-001` — Route error boundaries · P0

Steps. Force an error in each dashboard (block the Supabase host in DevTools, then navigate). **Expected Result.** A friendly error screen with a retry affordance and **only** an opaque digest. **No message, no column name, no id, no stack.** Repeat for a root-layout failure — the global error page **inlines its styles and supplies its own document shell**, because at that point nothing else has rendered.

`ERR-LOAD-001` — Loading states keep the chrome · P1

Steps. Throttle the network to Slow 3G and navigate between dashboard sections. **Expected Result.** A skeleton appears. **The sidebar is not blanked** — the patient, therapist and hospital dashboards render their sidebar per page rather than in a layout, so a loading screen that forgot the sidebar would blank the chrome on every navigation.

`ERR-NET-001` — Network failure mid-action · P0

Steps. Set the network to Offline, then: submit the booking; accept a suggestion; save a roster; settle a payout. **Expected Result.** Each shows a readable connection message (e.g. `Could not reach the server. Please check your connection and try again.`). **Nothing clears optimistically** — the person is left exactly where they were and can retry. When the network returns, a retry succeeds and **does not duplicate** the action.

`ERR-API-001` — API failure is readable · P1

Steps. Force a 500 from a route (stop Supabase mid-request). **Expected Result.** A human message. Never the raw error text.

`ERR-STALE-001` — Stale URLs · P1

Steps. Open `/admin/dashboard?section=today&tab=requests` and `?section=today&tab=sync` (tab keys that never existed). Open `/admin/dashboard/patients/<deleted id>`. Open `/book?category=<deleted id>`. **Expected Result.** The admin links **fall back to that section's first screen** — so they *look* like they work. **That is exactly why links must be built with the typed helper rather than hand-written.** A deleted patient id shows a not-found state, not a crash. A deleted category id simply leaves the concern unselected.

`ERR-404-001` — Not found · P2

Expected Result. `/no-such-page` renders the 404 page. `/home-visit` with the master switch off renders 404. Neither leaks internals.

`ERR-EMPTY-001` — Empty states everywhere · P2

Steps. Immediately after `SETUP-RESET-001`, open every admin screen, then each role's dashboard for a brand-new account. **Expected Result.** Every screen shows a **purposeful empty state**, never a blank panel, a zero-filled table, or a spinner that never resolves. **Filter chips are hidden unless at least two**

of them would have rows behind them — a filter nobody can act on is noise. A **screen that can only ever be empty is not in the sidebar at all.**

`ERR-DUP-001` — Duplicate submission of every form · P1

Steps. Double-tap the primary button on: the booking wizard, the register form, the roster save, the care-plan submit, the suggestion send, the payout settle, the refund, the expense create. **Expected Result.** Exactly one record in every case. Buttons disable synchronously, not a render later.

---

21. Cross-role consistency tests

Why these exist. Most defects in a multi-role application are not "the feature is broken" but "two roles disagree about the same fact". Each case below drives **one business event** and then checks **every** role's view of it.

`XR-BOOK-001` — One booking, five views · P0

Event. Patient A books and pays for a session (`PAT-BOOK-003`), and an admin assigns Therapist A.

Role	Where	Must show
Patient	`/patient/dashboard/sessions` → Upcoming	The session, `Confirmed`, with Therapist A's name and the Meet link
Therapist	`/therapist/dashboard/sessions`	The same session, same slot, patient's name, **masked phone**, no email
Admin	Sessions → All Sessions **and** Sessions → Schedule	The same session, in both, opening the **same drawer**
Finance	Money → Transactions	One payment row of ₹1,999 with its order and payment ids
Hospital	Your Referrals	**Nothing** — Patient A was not referred

Expected Result. The slot time is identical everywhere (allowing for the timezone label each screen states). The status is identical. **No screen shows a therapist the others do not.**

`XR-BOOK-002` — A referred patient's booking adds a sixth view · P0

Repeat with Patient C (Hospital A). **Additional expectation:** the hospital sees the referral as **Registered**, and once the session is completed, `₹249.90` appears on their Earnings and in Money → Breakdown's partner share — **the same number in both places**.

`XR-PAY-001` — One payment, one figure everywhere · P0

Event. The payment from `XR-BOOK-001`. **Expected Result.** Patient's Payments screen, Money → Transactions, Money → Summary's Gross, and the appointment's own paid state all describe **one** payment of ₹1,999. **No screen shows it twice and no screen shows a different amount.**

`XR-REFUND-001` — One refund, four views · P0

Event. Patient A cancels S3 outside the window. **Expected Result.** Patient: cancelled, refund shown. Therapist: the session leaves Upcoming; **their earnings do not change** (it never earned a share). Admin Money: Gross unchanged, Refunds +₹1,999, Net −₹1,999. Hospital (if referred): partner share **reduced**. Activity Log: the cancellation, attributed.

`XR-COMPLETE-001` **— One completed session, five views · P0**

Event. Therapist A completes S1. **Expected Result.** Therapist: Earnings +₹1,199.40, and a "Notes to write" nudge. Patient: the session moves to Past and a rating control appears. Admin Sessions: status `completed`, `completed_at` stamped. Money: the therapist share now **includes** this session (it did not while it was merely paid). Delivery: the completed count and the no-show rate update.

`XR-CUTOFF-001` **— The Session Completed cutoff reads the same on every surface · P0**

Event. A confirmed session passes the cutoff (default 60 minutes after the slot). **Steps.** Simulate 90 minutes after the slot in **each** role's browser and look at every surface that lists that session: the patient's card and calendar, the therapist's card, the admin's All Sessions row, the admin's Schedule day panel, and the admin's session drawer. **Expected Result. Every one reads "Session Completed"** — including the admin's own. A session an hour past its start must read the same way on every screen it appears on. Changing the cutoff setting must move **all** of them together.

`XR-CARE-001` **— One recommendation, three views · P0**

Event. `THR-CARE-001`. **Expected Result.** Therapist's chart, patient's Suggested Sessions **and** patient's Health Profile all render **the same rows** — one record, two readers, branching only on whose voice the copy is written in. Admin → Sessions → Recommendations lists it. **All four agree on the package, the session count and the price.**

`XR-CARE-002` **— One purchase, five views · P0**

Event. `PAT-CARE-002`. **Expected Result.** Patient: Your Packages, `6 sessions · 0 used · 6 remaining`. Therapist: the programme appears in the Programmes view of My Patients with the same balance. Admin: Catalog → Purchases with the same balance and a frozen snapshot. Money: the purchase's cash appears as **Package cash collected** and is recognised session by session as **Recognised revenue** — these are **two different words for two different things and must not be conflated**. Activity Log: nothing (a patient purchase is not an admin action).

`XR-CREDIT-001` **— One credit consumed, four views · P0**

Event. The patient books one package session. **Expected Result.** Patient widget: `1 used · 5 remaining`. Therapist: the session appears, auto-assigned and auto-confirmed with its own Meet link. Admin Purchases: the same balance. **[SQL]** the ledger holds exactly one `reserve:<appointment_id>` row. Flipping the ledger-authority switch must not change any of these four figures while the ledger and the counters agree.

`XR-PAYOUT-001` **— One settlement, four views · P0**

Event. `FIN-PAY-002`. **Expected Result.** Therapist Earnings: paid-out increases, owed decreases by the same amount. Admin Payouts: the row settles. Money → Summary: the therapist share is unchanged (settling moves money, it does not create a new share). Activity Log: a **`payout.settle`** row with the actor, target and amount.

`XR-CASH-001` **— One cash collection, four views · P0**

Event. `THR-SESS-007`. **Expected Result.** Therapist: the visit shows cash recorded, and their next payout figure **drops by the cash held**. Admin Cash Ledger: the collection appears as un-remitted. Money → Payouts: the Pay button shows the netted figure. After settling: the Cash Ledger stops listing it, and a second payout does **not** deduct it again.

`XR-HP-001` **— One health-profile update, three views · P0**

Event. `THR-HP-002` then `PAT-HP-004`. **Expected Result.** Patient: their own answers, correctly attributed — **the counter must not claim the patient answered questions a clinician wrote**, and the banner must not say "Your therapist has your answers" about a record the patient never sent. Therapist: the same answers on the chart. Admin: the change request in the review queue and, after approval, in Review History. **Nobody is told they did something they did not do.**

`XR-SUGG-001` — One suggestion, two views · P1

Event. `THR-SUGG-001` → `PAT-SUGG-002`. **Expected Result.** While pending: therapist sees **Waiting on the patient**, patient sees the card. After acceptance: therapist sees the booked session, patient sees it under Upcoming, and exactly one credit has moved.

`XCFG-ROSTER-001` — A roster change moves nothing · P0

Purpose. This is the guard on the deliberate separation between the roster and the booking picker. It is the single most likely place for a well-intentioned "fix" to break the design.

Steps

1. Note exactly which dates and hours `/book` offers under simulated time TIME-A. Screenshot it.
2. As an admin, blank Therapist A's **entire** weekly schedule (mark every day unavailable) and save.
3. Put Therapist A on leave for the next fortnight.
4. Add a date exception that closes a day entirely.
5. Reload `/book` under the same simulated time and compare with the screenshot.
6. Check the patient's existing confirmed session with Therapist A.

Expected Result. Step 5: **the picker is byte-for-byte identical.** The roster is the clinic's planning record — who can be *offered* — and it deliberately does **not** filter the patient's picker, which applies the lead-time rule alone. Step 6: **the appointment is unchanged** — not cancelled, not moved, not flagged. **If the picker changed, do not "fix" the roster — raise it.** Connecting the two is a product decision with a deploy-sized blast radius, not a refactor.

---

22. Full regression journeys

Each journey is an end-to-end run through the product, executed in one sitting. Run all four before a release.

`REG-J1` — The core money journey · P0

```
SETUP-RESET-001
  → SETUP-CAT-001 (three conditions)  → SETUP-PKG-001 (P1, P2, P3)
  → ADM-SET-026 (scoped admins)       → ADM-PEOP-006 (revenue shares)
  → THR-AUTH-001 → ADM-APPR-002 → THR-AVAIL-001
  → PAT-BOOK-002 → PAT-BOOK-003 (book + pay)
  → ADM-SESS-002 (assign)             → XR-BOOK-001 (five views agree)
  → THR-SESS-005 (complete)           → XR-COMPLETE-001
  → THR-CARE-001 (recommend)          → ADM-CARE-004 (clinic approves) → XR-CARE-001
  → PAT-CARE-002 (accept + pay)       → XR-CARE-002
  → PAT-PKG-001 (spend 3 credits)     → XR-CREDIT-001
  → THR-SESS-005 ×3 (deliver them)
  → FIN-SUM-001 (identities hold)     → FIN-PAY-002 (settle) → XR-PAYOUT-001
  → ADM-SET-033 (every action is in the log)
```

Pass criterion. Both money identities hold at the end, every cross-role check agrees, and the log contains every mutating action including `payout.settle`.

`REG-J2` — The home-visit and cash journey · P0

```
ADM-SET-013 (home visits on) → SETUP-AREA-001 → SETUP-HVPKG-001
  → PAT-HV-002 (prepaid, serviceable)   → PAT-HV-003 (unserviceable → waitlist)
  → PAT-HV-006 (cash on visit)
  → ADM-SESS-002 (assign)               → THR-SESS-007 (record cash)
  → THR-SESS-005 (complete)             → XR-CASH-001
  → FIN-PAY-003 (cash nets off the payout, and is marked remitted)
  → FIN-PAY-004 (cash > owed → floors at zero, stays on the ledger)
  → FIN-REF-003 (cash refund → manual pending)
  → PAT-CARE-003 (a recommended home programme quotes travel per visit)
  → ADM-SET-013 again (switch off → the recommendation stops being purchasable)
```

`REG-J3` — The partner journey · P0

```
HOS-LEAD-001 → HOS-AUTH-002 (provision) → HOS-AUTH-001
  → HOS-REF-001 (refer) → HOS-REF-004 (the whole status pipeline)
  → HOS-REF-006 (patient registers with the code)
  → PAT-BOOK-003 as Patient C → ADM-SESS-002 → THR-SESS-005
  → HOS-MONEY-001 (commission on net) → XR-BOOK-002
  → PAT-CANCEL-001 on a second referred session (refund reverses the commission)
  → HOS-MONEY-002 (a therapist with no share ⇒ excluded, not guessed)
  → HOS-SEC-001..004 (isolation)
```

`REG-J4` — The clinical journey · P0

```
THR-HP-001 (triage, ortho) → THR-HP-002 (first fill, live)
  → PAT-HP-001 then PAT-HP-002 (locked → unlocked)
  → PAT-HP-003 (one question at a time) → PAT-HP-004 (change → review)
  → PAT-DOC-001..003 (upload, limits, delete)
  → THR-HP-005 (Pain Map on ortho; refused on neuro)
  → THR-HP-004 (re-triage to neuro MERGES, never replaces)
  → ADM-SET-020 (edit questions; disable paediatrics; ortho cannot be disabled)
  → THR-SESS-008 (session notes; invisible to the patient and to the PDF)
  → PAT-HP-005 (export as PDF, including a non-Latin name)
  → THR-LEAK-001..007 (the scanner, both tiers, and the evidence tables)
  → XR-HP-001
```

`REG-J5` — Security sweep · P0

Run **all** of §18 in one pass, then `ADM-SET-027`'s full table.

`REG-J6` — Payment integrity sweep · P0

Run **all** of §16.3 in one pass.

---

23. Test dependency map and recommended execution order

23.1 The main dependency chain

```
SETUP-RESET-001
   └─ SETUP-CAT-001 ─┬─ SETUP-PKG-001 ─┐
                     └─ PAT-BOOK-002 ──┤
THR-AUTH-001 → ADM-APPR-002 → THR-AVAIL-001
                                       │
                     PAT-BOOK-003 ◄────┘
                          │
                          ├─ PAT-PAY-001..005      (payment outcomes)
                          └─ ADM-SESS-002          (assign)
                                  │
                                  ├─ THR-SESS-003/004  (masking, reveal)
                                  └─ THR-SESS-005      (complete)
                                          │
                            ┌─────────────┴──────────────┐
                    THR-HP-001/002                 THR-CARE-001
                            │                            │
                      PAT-HP-002                   PAT-CARE-002
                            │                            │
                      PAT-HP-003/004               PAT-PKG-001
                                                         │
                                                   THR-SUGG-001
                                                         │
                                                   PAT-SUGG-002
                                                         │
                                            FIN-SUM-001 → FIN-PAY-002
                                                         │
                                                   ADM-SET-033
```

23.2 Recommended execution order

#	Phase	Tests	Notes
1	**Reset**	`SETUP-RESET-001`, `SETUP-RESET-003`	Must be first. Confirm an admin survives.
1b	**Reset, scope gate**	`SETUP-RESET-002`	**Runs after `ADM-SET-026` in phase 2**, not here: it signs in as the Operations admin, and a freshly reset database has only the one admin made by hand in Supabase. Run it as soon as that account exists — the wipe it attempts must be refused, so it costs nothing to run late.
2	**Admin & catalog setup**	`ADM-CAT-001`, `ADM-CAT-005`, `ADM-CAT-010`, `SETUP-HVPKG-001`, `ADM-SET-026` → then `SETUP-RESET-002`	Nothing downstream works without a catalog. `ADM-SET-026` shows each new admin's password **once** — copy all three before leaving the screen.
3	**Create users**	`PAT-AUTH-002`, `THR-AUTH-001`, `HOS-LEAD-001` → `HOS-AUTH-002`	Patient A is created *inside* `PAT-BOOK-003`, deliberately — that is the guest path. **Run these through the UI the first time**: they are the sign-up, application and onboarding flows, and `npm run seed:qa` is not a substitute for testing them. Use the seeder afterwards, to put the same accounts back after every later reset (§6.2).
4	**Approve users**	`ADM-APPR-001..004`	

#	Phase	Tests	Notes
5	**Configure availability**	`THR-AVAIL-001..007`, `ADM-ROST-001..005`	
6	**Booking**	`PAT-BOOK-001..017`, `PAT-HV-001..007`	Time-simulation scenarios TIME-A…D.
7	**Payment**	`PAT-PAY-001..005`, §16.3	Needs Razorpay test keys **and** the webhook secret.
8	**Therapist session**	`ADM-SESS-002..004`, `THR-SESS-001..008`	
9	**Care plan**	`THR-CARE-001..008`, `ADM-CARE-001..008`	
10	**Purchase**	`PAT-CARE-001..004`	
11	**Credits**	`PAT-PKG-001..004`, `THR-SUGG-*`, `PAT-SUGG-*`	
12	**Clinical**	`THR-HP-*`, `PAT-HP-*`, `PAT-DOC-*`, `THR-LEAK-*`	
13	**Finance**	§16.1–16.2	Build the reference dataset first.
14	**Configuration dependencies**	§15.4, all 46 rows	Restore every setting afterwards.
15	**Security**	§18 in full	
16	**UX / mobile / a11y**	§19	
17	**Error / loading / empty**	§20	Empty states are easiest right after a reset — consider running `ERR-EMPTY-001` in phase 1.
18	**Cross-role**	§21	
19	**Final regression**	`REG-J1..J6`	

23.3 Tests that must be run on a fresh database

`SETUP-RESET-001`, `ERR-EMPTY-001`, `PAT-EMPTY-001`, and any test that books a **fixed** future slot. A rerun on a dirty database will refuse the booking as a clash — **that refusal is correct behaviour**.

23.4 Tests that change global state — restore afterwards

Test	Restore
Every `ADM-SET-*`	Put the setting back to its documented default
`DBG-TIME-*`, and every TIME scenario	**Reset to Real Time**
`SETUP-RESET-003`	Re-arm `ALLOW_DEBUG_DATA_RESET=true`
`PAY-WH-004`	Restore `RAZORPAY_WEBHOOK_SECRET`
`ADM-PEOP-007`, `SEC-AUTH-006`	Re-activate the suspended accounts
`ADM-CAT-002` (deactivate)	Re-activate the category
`ADM-SET-020` (disable paediatrics)	Re-enable it

---

24. Coverage audit

24.1 Area × dimension matrix

Area	Patient	Therapist	Hospital	Admin	Finance	Security	Mobile
Registration / login	`PAT-AUTH-001..006`	`THR-AUTH-001..003`	`HOS-AUTH-001..003`	`SETUP-RESET-001`	—	`SEC-AUTH-004..007`	`UX-MOB-002`
Approval gates	`PAT-AUTH-003`	`THR-AUTH-002`	`HOS-AUTH-003`	`ADM-APPR-001..004`	—	`SEC-AUTH-006`	—
Booking (video)	`PAT-BOOK-001..017`	`THR-SESS-001`	—	`ADM-NEWB-001`, `ADM-SESS-002`	`FIN-SUM-002`	`PAT-BOOK-012`, `SEC-TAMPER-001`	`UX-MOB-001`
Booking (home visit)	`PAT-HV-001..007`	`THR-SESS-007`	`HOS-REF-002`	`ADM-CAT-010`, `ADM-SET-014`	`FIN-PAY-003`	`PAT-HV-005`	`UX-MOB-001`
Payment	`PAT-PAY-001..005`	—	—	`FIN-TXN-001`	§16.3 in full	`PAY-AMT-001/002`	`UX-MOB-001`
Refunds	`PAT-CANCEL-001..003`	—	`HOS-MONEY-001`	`ADM-SESS-004`	`FIN-REF-001..004`	—	—
Sessions lifecycle	`PAT-SESS-001..006`	`THR-SESS-001..008`	—	`ADM-SESS-001..004`, `ADM-SCHED-001`	`FIN-SUM-002`	`THR-SEC-002`	`UX-MOB-002`

Area	Patient	Therapist	Hospital	Admin	Finance	Security	Mobile
Availability / roster	—	`THR-AVAIL-001..008`	—	`ADM-ROST-001..005`	—	`THR-SEC-001`, `ADM-ROST-005`	—
Health profile	`PAT-HP-001..005`	`THR-HP-001..006`	`HOS-SEC-002`	`ADM-PEOP-004`, `ADM-SET-020`	—	`SEC-DATA-001`	`UX-MOB-004`
Documents	`PAT-DOC-001..003`	(read)	`HOS-SEC-002`	(read)	—	`SEC-DATA-004`	`UX-MOB-003`
Care plans	`PAT-CARE-001..004`, `PAT-SCHED-001..003`	`THR-CARE-001..008`	—	`ADM-CARE-001..008`	`PAY-AMT-002`	`THR-CARE-002`	—
Suggested sessions	`PAT-SUGG-001..005`	`THR-SUGG-001..002`	—	`ADM-SET-018`	—	`PAT-SUGG-004`	—
Session credits	`PAT-PKG-001..004`	(view)	—	`ADM-CAT-014/015`, `ADM-SET-019`	`FIN-REF-004`	`SEC-TAMPER-003`, `PAY-CONC-001`	—
Referrals	`HOS-REF-006`	—	`HOS-REF-001..007`	`ADM-PEOP-008`	`HOS-MONEY-001..003`	`HOS-SEC-001`	—
Earnings / payouts	—	`THR-EARN-001..004`	`HOS-MONEY-001..003`	`FIN-PAY-001..006`	`FIN-PAY-001..006`	`SEC-ADMIN-001`	—
Catalog	(reads)	(reads)	—	`ADM-CAT-001..015`	`ADM-CAT-006`	`ADM-SET-028`	`UX-MOB-005`
Settings	—	—	—	`ADM-SET-001..035`	`FIN-COST-002`	`ADM-SET-025..028`	—
Contact controls	`THR-LEAK-005`	`THR-LEAK-001..007`, `THR-SESS-003/004`	—	`ADM-SET-029`	—	`SEC-DATA-005`	—
Risk	—	—	—	`ADM-RISK-001..004`	—	`ADM-RISK-003`, `ADM-RISK-004`	—
Audit log	—	—	—	`ADM-SET-033`	`XR-PAYOUT-001`	`ADM-SET-033`	—
Public site	`PUB-*`	—	`HOS-LEAD-001`	`ADM-SET-004..008`	—	`SEC-ROUTE-004`	`UX-MOB-001`

Area	Patient	Therapist	Hospital	Admin	Finance	Security	Mobile
Debug bar	`DBG-TIME-001`	—	—	`SETUP-RESET-001..003`, `DBG-NAV-001`	—	`SETUP-RESET-002/003`	—

24.2 Route coverage

Every route in §3 is mapped to at least one test in its own table's rightmost column. **All 9 public pages, 2 booking routes, 10 patient routes, 8 therapist routes, 6 hospital routes, 2 admin routes + 31 admin screens + 3 admin detail routes, and 4 system routes are covered.** API routes are covered by the tests that drive them plus §18's direct calls.

24.3 Admin screen coverage

All **31** screens have at least one dedicated test — see the §3.6 table. Every screen with a mutating control also has a negative and an authorization test.

24.4 Configuration coverage

All **46** configuration→dependent-feature pairs in §15.4 have a verification test. Every one names the screen where the change must be *proved*, not merely saved.

24.5 Payment coverage

Success (`PAT-BOOK-003`), failure (`PAT-PAY-001`), cancellation/dismissal (`PAT-PAY-002`), abandonment (`PAT-PAY-004`), retry (`PAT-PAY-001`, `PAT-PAY-005`), refund (`FIN-REF-001..004`), refund failure (`FIN-REF-002`), duplicate click (`PAT-BOOK-017`), duplicate callback (`PAY-WH-002`), duplicate webhook (`PAY-DUP-004`), signature tampering (`PAY-WH-001`), amount tampering (`PAY-AMT-001/002`), concurrency (`PAY-CONC-001/002`), and the seven "must never happen" duplications (`PAY-DUP-001..007`).

24.6 Role coverage

Every protected action has both an authorized case and an unauthorized one. The unauthorized case is always tested **at the route**, not only in the UI.

24.7 State coverage

Entity	Transitions covered
Appointment	requested → confirmed → completed; → cancelled (+refund / no refund); completed → reopened; cancelled/no-show → restored; unpaid → paid
Package purchase	active → expired; active → refunded; credits reserved → consumed → released/voided
Care plan	active → accepted (purchased, closed); active → withdrawn; active → declined; superseded
Suggestion	pending → accepted; pending → declined; pending → lapsed (computed, never written)
Referral	pending_review → therapist_assigned → invite_sent → converted; → declined; → withdrawn
Payout request	pending → reviewing → completed

Entity	Transitions covered
Risk signal	open → reviewing → dismissed / actioned; reopened as a fresh signal
Account	unapproved → approved; active → suspended → active; patient auto-approved by payment attempt
Cash	collected → (corrected) → remitted; refund → manual_pending → returned

24.8 What is not testable in this environment, and why

Item	Why	What to do instead
Real email delivery	All fixtures use the reserved `.test` TLD, and the product sends no transactional email of its own (the Google Calendar invite is the only outbound notification)	Verify the request succeeded and the reset landing page renders
Google Calendar/Meet against a real calendar	Needs live Google credentials and a real calendar	Test the **failure and retry** path instead (`ADM-SESS-003`) — that is the behaviour that matters
Live Razorpay settlement, chargebacks, real bank refunds	Test mode does not settle	Verify the local state transitions and the gateway's own test dashboard
Server-clock-dependent gates under simulation	The simulated clock is client-side only, by design	Use real near-future slots (§7.2)
`next start` ISR behaviour matching fixtures	Public pages cache for 300 s	Run against `next dev`; if you must use `next start`, wait out or trigger revalidation
Browser-side Supabase reads in a network-isolated sandbox	Chromium needs egress for `?therapist=` resolution	Check egress first (`PAT-BOOK-007`'s note) before filing a defect
Multi-region / timezone-shifted testing	The app records the browser's detected timezone	Change the OS timezone and re-run `PAT-BOOK-002`; the calendar and the stored `timezone` must agree
Load and performance	Out of scope for a manual plan	Note any screen that feels slow, especially the admin dashboard's ~40 queries

---

25. Questions and clarifications before execution

These are **product decisions**, not gaps in the inspection. In each case the code's current behaviour is established and stated, and the test asserts exactly that behaviour — but whether it is the *intended* behaviour is a call only the product owner can make. Confirm each before executing the affected tests; if a decision changes the answer, the named test changes with it. **Nothing here is invented behaviour.**

1. **Duplicate referral prevention (affects `HOS-REF-003`).** Established from the schema: `patient_referrals` carries **no uniqueness constraint** on hospital + patient name, so a hospital that genuinely re-types the same referral creates a **second row**. Only the double-tap case is defended, by the form's own submit guard. **Decision needed:** is a genuine re-submission meant to be blocked at the database, de-duplicated on the admin's side, or simply left visible in the queue for a human to decline? The test currently asserts the last of these, because that is what the code does today.

1. **Therapist timezone (affects `THR-AVAIL-001`).** Established from the code: the roster editor reads **`profiles.timezone`** and prints it as a label, falling back to `Asia/Kolkata`; there is **no field anywhere in the therapist's Edit Profile that sets it**, and the admin's Roster only displays it. So today it is per-therapist in the schema and un-editable in the product. **Decision needed:** should a therapist (or an admin) be able to set it, or is the fallback the intended behaviour for a clinic operating in one timezone? Until that is decided, `THR-AVAIL-001` asserts only that the header states a timezone.

1. **Tablet support (affects §19).** The plan tests 820 × 1180 as a courtesy. **Question:** is tablet a supported breakpoint with its own expectations, or simply "desktop layout, narrower"?

1. **Paediatric fixture depth (affects §8.6).** Patient D's paediatric data is supplied so the third specialty can be exercised, but no journey in this plan requires a paediatric patient end to end. **Question:** should paediatrics get its own full journey, or is `ADM-SET-020`'s enable/disable coverage sufficient for this release?

1. **`plan_conversion_low` and `post_consultation_dropout` (affects `ADM-RISK-001`).** Both ship **disabled** because a threshold invented before the clinic has a baseline fires on everyone or on nobody. **Question:** should this release enable them with a provisional threshold, or leave them off? The tests currently assert they are off.

1. **Refund of a partially delivered home-visit package (affects `FIN-REF-004`).** The session-package rule ("void what is available, never what is consumed") is explicit. The equivalent for a **cash** home-visit package, where no online payment exists to reverse, resolves to `manual_pending`. **Question:** confirm the intended split when a cash home-visit package is partly delivered — how much is expected back at the door?

1. **Reassigning a programme's therapist mid-course (affects `ADM-CAT-014`).** Reassignment touches **future** sessions only. **Question:** should the patient be notified, and if so through which surface? The plan currently asserts only the data outcome.

1. **Expected UI copy for a handful of validation messages.** The API messages quoted throughout this document are taken verbatim from the route handlers. Where a component *re-words* a route's error before showing it, the test asserts the API string. **Question:** for any case where the on-screen wording differs from the quoted API string, confirm which is authoritative so the test can assert the right one.

---

26. Defect reporting template

Paste this, filled in, when reporting a failure. The **Test ID plus the actual result** is what makes a report diagnosable.

```
Test ID:
Role:
Feature:
Environment:                 (local dev / staging; Supabase project ref; app URL)
Server:                      (next dev / next start)
Simulated Date/Time:         (the value set in the Debug bar, or "real time")
Browser / viewport:
Step that failed:            (the numbered step)
Expected Result:             (quote it from the test case)
Actual Result:               (exactly what happened, including the exact on-screen text)
Test Data:                   (the values entered)
Screenshot / video:
Console error:
Network / API error:         (route, HTTP status, response body)
Payment ID / Order ID:       (if applicable)
Appointment ID:              (if applicable)
Purchase / Entitlement ID:   (if applicable)
Reproducible?:               (always / intermittently / once)
First or second consecutive run on this database?
Additional notes:
```

Before filing, rule out these five known-correct behaviours:

1. A booking refused as a clash on a **second consecutive run** — leftover state, not a bug.
2. A **server-side** time gate refusing an action under a simulated clock — the simulation is client-side by design.
3. A fixture missing from a **public page under `next start`** — ISR caches for 300 seconds.
4. A `?therapist=` chip missing in a **network-isolated sandbox** — the browser needs egress.
5. `/home-visit` returning **404** while the master switch is off — that is the feature.

---

27. Final release checklist

Sign off each line before release.

Data and environment

- [ ] `ALLOW_DEBUG_DATA_RESET` is **unset** in production, and `debug_reset_all_data()` is dropped
- [ ] `.env.production` does not exist in the repository
- [ ] The hosting dashboard's own environment variables have been checked (a deleted file cannot clear those)
- [ ] `RAZORPAY_WEBHOOK_SECRET` is set in production
- [ ] Razorpay is in **live** mode with live keys; no test key remains
- [ ] `supabase/schema.sql` has been applied, and applies **twice** cleanly
- [ ] Supabase Auth → **Confirm email is OFF**
- [ ] The `medical-reports` bucket is **private**; `avatars` is public
- [ ] At least two `full`-scope admins exist

Pre-launch removals

- [ ] **The Debug Bar is deleted** — not merely switched off. It is a public flag, and the bar names `/admin/login` and `/admin/dashboard`

- [ ] The five seeded testimonials are replaced with real, consented ones, or removed
- [ ] Stock photography under `public/photos/` is replaced with the clinic's own, keeping the aspect ratios

Functional sign-off

- [ ] `REG-J1` … `REG-J6` all pass on a fresh database
- [ ] Both money identities hold on the reference dataset
- [ ] Settings → System Health reports **no** accounting disagreements and **no** unresolved sync issues
- [ ] Every action in the audit vocabulary has been observed in **Logs → All Activity**, `payout.settle` included
- [ ] A clear at the shortest cutoff left the last 30 days intact and recorded itself (`ADM-LOG-002`)
- [ ] No generated password appears anywhere in the log
- [ ] `npm run verify` (lint + unit tests + build) passes
- [ ] `npm run lint` passes, including the Realtime publication coverage check
- [ ] The Playwright suite passes against a test project (`workers: 1`, against `next dev`)

Security sign-off

- [ ] Every §18 case passes, each at the route level as well as in the UI
- [ ] No admin path appears in any public client bundle (once the debug bar is deleted)
- [ ] No stack trace, column name or row id is reachable on any error screen
- [ ] Append-only tables (`session_credit_ledger`, `care_plan_versions`, `communication_flags`, `contact_reveal_log`, `admin_activity_log`, `risk_reviews`, `session_note_revisions`) all refuse update and delete

UX sign-off

- [ ] The full booking + payment flow passes at 390 × 844 with no horizontal scrolling
- [ ] Keyboard-only completion of the booking wizard succeeds
- [ ] No hydration warnings in the console on any page
- [ ] Every dashboard offers **Back to Home** in all three sidebar renders

Documentation

- [ ] `README.md`, `AGENTS.md` and `CLAUDE.md` describe the shipped behaviour — routes, roles, environment variables, npm scripts, and every documented rule (booking lead time, refund window, payment verification, Meet sync, payout maths)